



UNIVERSITAT
POLITÈCNICA
DE VALÈNCIA



UNIVERSITAT POLITÈCNICA DE VALÈNCIA

Escuela Técnica Superior de Ingeniería Informática

Implementación de actualizaciones OTA en dispositivos IoT
para la plataforma Thingsboard

Trabajo Fin de Grado

Grado en Ingeniería Informática

AUTOR/A: Gimeno Hermán, Pablo

Tutor/a: Manzoni, Pietro

CURSO ACADÉMICO: 2024/2025

A mis amigos Álvaro, Andrés y José Antonio.

Resum

Aquest Treball de Fi de Grau presenta el desenvolupament d'un sistema IoT amb la capacitat de realitzar actualitzacions OTA (*Over The Air*), concepte clau al camp de l'Internet de les Coses. Aquest tipus d'actualitzacions, que es produeixen de manera remota i sense fil, estan dirigides a resoldre vulnerabilitats, millorar característiques o fer front a canvis en les especificacions dels dispositius que formen part de la xarxa IoT.

Servint-nos de Thingsboard, una popular plataforma IoT de codi obert, implementarem l'actualització OTA per a uns dispositius que es comunicaran amb la plataforma en dos escenaris de connectivitat que fins ara no han estat contemplats. Concretament, connectarem la plataforma amb dispositius que podran utilitzar tres tecnologies de xarxa sense fils diferents: Wifi, BLE i LoRa, amb la particularitat que aquestes dues últimes no compten actualment amb suport per a OTAs a Thingsboard. Per resoldre'l, ampliarem les capacitats de la plataforma de manera que mitjançant un sistema d'intermediaris l'actualització OTA serà possible també a través de BLE i LoRa. El resultat final serà que, des d'un panell visual a la interfície de Thingsboard, podrem gestionar les versions dels dispositius independentment de quina tecnologia s'hagi fet servir per connectar-los.

Per dur a terme la implementació que ens hem proposat, també haurem d'escriure el codi necessari als dispositius que s'actualitzaran, desenvolupant una biblioteca que gestioni la comunicació a través de les tres tecnologies de xarxa sense fils (Wifi, BLE i LoRa), a més de rebre i instal·lar correctament els paquets d'actualització. Ho farem fent servir el llenguatge de programació Micropython, amb el qual programarem unes plaques de desenvolupament basades en microcontrolador i, addicionalment, plantejarem una línia de treball per portar aquesta implementació a dispositius Linux.

Paraules clau: OTA, Thingsboard, Micropython, ESP32, IoT, actualització, Linux, Docker, LoRa, BLE

Resumen

Este Trabajo de Fin de Grado presenta el desarrollo de un sistema IoT con la capacidad de realizar actualizaciones OTA (*Over The Air*), concepto clave en el campo del Internet de las Cosas. Este tipo de actualizaciones, que se producen de forma remota e inalámbrica, están dirigidas a resolver vulnerabilidades, mejorar características o hacer frente a cambios en las especificaciones de los dispositivos que forman parte de la red IoT.

Sirviéndonos de Thingsboard, una popular plataforma IoT de código abierto, implementaremos la actualización OTA para unos dispositivos que se comunicarán con la plataforma en dos escenarios de conectividad que hasta la fecha no han sido contemplados. Concretamente, conectaremos la plataforma con dispositivos que podrán utilizar tres tecnologías de red inalámbrica diferentes: Wifi, BLE y LoRa, con la particularidad de que estas dos últimas no cuentan actualmente con soporte para OTAs en Thingsboard. Para resolverlo, ampliaremos las capacidades de la plataforma de forma que, mediante un sistema de intermediarios, la actualización OTA será posible también a través de BLE y LoRa. El resultado final

será que, desde un panel visual en la interfaz de Thingsboard, podremos gestionar las versiones de los dispositivos independientemente de qué tecnología se haya usado para conectarlos.

Para llevar a cabo la implementación que nos hemos propuesto, también deberemos escribir el código necesario en los dispositivos que se actualizarán, desarrollando una biblioteca que gestione la comunicación a través de las tres tecnologías de red inalámbrica (Wifi, BLE y LoRa), además de recibir e instalar correctamente los paquetes de actualización. Lo haremos utilizando el lenguaje de programación Micropython, con el cual programaremos unas placas de desarrollo basadas en microcontrolador y, adicionalmente, plantearemos una línea de trabajo para llevar esta implementación a dispositivos Linux.

Palabras clave: OTA, Thingsboard, Micropython, ESP32, IoT, actualización, Linux, Docker, LoRa, BLE

Abstract

This Final Degree Project presents the development of an IoT system with the ability to perform OTA (Over The Air) updates, a key concept in the field of the Internet of Things. These types of updates, which are performed remotely and wirelessly, are intended to resolve vulnerabilities, improve features, or address changes in the specifications of devices that are part of the IoT network.

Using Thingsboard, a popular open-source IoT platform, we will implement OTA updates for devices that will communicate with the platform in two connectivity scenarios that have not been considered to date. Specifically, we will connect the platform with devices that can use three different wireless network technologies: Wi-Fi, BLE, and LoRa, with the particularity that the latter two do not currently support OTAs in Thingsboard. To solve this, we will expand the platform's capabilities so that, through a system of intermediaries, OTA updates will also be possible via BLE and LoRa. The end result will be that, from a visual panel in the Thingsboard interface, we will be able to manage device versions regardless of the technology used to connect them.

To carry out the implementation we have proposed, we will also need to write the necessary code on the devices to be updated, developing a library that manages communication through the three wireless network technologies (Wi-Fi, BLE, and LoRa), as well as receiving and correctly installing the update packages. We will do this using the Micropython programming language, with which we will program microcontroller-based development boards and, additionally, we will propose a line of work to bring this implementation to Linux devices.

Key words: OTA, Thingsboard, Micropython, ESP32, IoT, update, Linux, Docker, LoRa, BLE

Índice general

Índice general	VII
Índice de figuras	IX
Glosario	XI

1	Introducción	1
1.1	Motivación	2
1.2	Objetivos	3
1.3	Metodología	5
1.4	Estructura de la memoria y convenciones	6
2	Estado del arte	9
2.1	Enfoques habituales en las actualizaciones OTA	10
2.2	OTAs en los dispositivos del borde	11
2.2.1	Placas basadas en microcontrolador	12
2.2.2	Placas basadas en microprocesador	14
2.3	Plataformas frente a la gestión de OTAs	16
2.4	Consideraciones sobre las OTAs a través de diferentes protocolos de red	18
3	Propuesta	21
3.1	Análisis de las limitaciones actuales	21
3.2	Estudio de posibles soluciones	22
3.2.1	Elección de una plataforma	22
3.2.2	Elección de una tecnología para programar dispositivos	23
3.3	Propuesta final	24
4	Diseño de la solución	27
4.1	Arquitectura del Sistema	27
4.2	Tecnologías utilizadas	30
4.3	Diseño detallado	33
4.3.1	Diseño de las comunicaciones	33
4.3.2	Diseño del software de los dispositivos	40
5	Desarrollo de la solución propuesta	49
5.1	Estructura del proyecto	49
5.1.1	Árbol de ramas	49
5.1.2	Árbol de archivos	50
5.2	Características relevantes	50
5.2.1	Desarrollo en Micropython	50
5.2.2	Desarrollo de recursos en Thingsboard	56
5.2.3	Creación de paquetes de actualización	58
5.2.4	Infraestructura e intermediarios de comunicación	60
6	Implantación	65

6.1	Despliegue de la infraestructura	65
6.1.1	Despliegue del servidor Thingsboard	66
6.1.2	Despliegue del Thingsboard Gateway y componentes auxiliares	67
6.1.3	Despliegue del gateway LoRa-MQTT	70
6.2	Configuración de la plataforma	72
6.2.1	Creación de perfiles	73
6.2.2	Dashboard	73
6.2.3	Cadena de reglas	74
6.2.4	Conector MQTT	75
6.2.5	Conector BLE	76
6.2.6	Creación de dispositivos	76
6.3	Instalación en los dispositivos del borde	78
6.3.1	Dispositivo conectado por Wifi	79
6.3.2	Dispositivo conectado por BLE	80
6.3.3	Dispositivo conectado por LoRa	80
7	Pruebas	83
7.1	Técnicas de depuración	83
7.1.1	Logging	83
7.1.2	REPL de Micropython	83
7.1.3	Pruebas de conectividad	84
7.1.4	Depuración de la cadena de reglas en Thingsboard	85
7.2	Prueba de integración	86
8	Conclusiones	91
8.1	Consecución de los objetivos	91
8.2	Trabajos futuros	93
8.3	Relación del trabajo desarrollado con los estudios cursados	94
	Bibliografía	97
	Apéndices	
A	Archivos de interés	101
A.1	platform/tb-server/docker-compose.yml	101
A.2	platform/tb-gateway/docker-compose.yml	101
A.3	devices/micropython/mip_packages/thingsboard-ota-helpers.json	103
B	Objetivos de Desarrollo Sostenible (ODS)	105

Índice de figuras

2.1	Publicidad de Particle	17
4.1	Arquitectura inicial del sistema	28
4.2	Arquitectura final del sistema	29
4.3	Diseño de la comunicación por Wifi	35
4.4	Diseño de la comunicación por BLE	36
4.5	Diseño del servicio OTA Transfer	37
4.6	Diseño de la comunicación por LoRa	39
5.1	Captura extraída del archivo README.md en la raíz del proyecto .	50
5.3	Diagrama de flujo del script <i>boot.py</i>	51
5.4	Diagrama de flujo del script <i>main.py</i>	52
5.5	En este directorio se almacenan todos los archivos <i>.py</i>	53
5.6	En este directorio se almacenan todos los archivos de configuración	54
5.7	En este directorio, con los ficheros JSON que empiezan por...	54
5.8	Versión reducida del script <i>main.py</i> correspondiente a la versión para LoRa de la aplicación de ejemplo, donde se enfatiza su estruc- tura asíncrona.	56
5.10	Cadena de reglas <i>Updatable-Devices-rule-chain</i>	57
5.11	Dashboard titulado <i>Updatable-Devices-dashboard</i>	58
5.12	Directorio <i>tools/</i>	58
5.13	Ejecución del programa de línea de comandos llamado <i>gen_ota_package.py</i>	59
5.14	En esta imagen se muestran los directorios <i>tb-gateway</i> y <i>tb-server</i> .	60
5.2	Estructura de archivos general del proyecto	63
5.9	Directorio donde se almacenan los recursos importables	63
6.2	Creación de la entidad Gateway en Thingsboard	66
6.1	Inicio de sesión en Thingsboard	67
6.3	Configuración de una conexión MQTT en en Thingboard	69
6.4	Cuadro emergente que aparece en el navegador	72
6.5	Configuración del gateway LoRa-MQTT	72
6.6	Importación de perfiles de dispositivo.	73
6.7	Importacion de Dashboards.	74
6.8	Importación de una cadena de reglas.	74
6.9	<i>Updatable-Devices-rule-chain</i> como la cadena de reglas raíz.	75
6.10	Creación de un conector MQTT.	75
6.11	Seleccionaremos nuestro conector en la parte izquierda...	76
6.12	Configuración del conector BLE	77
6.13	Se selecciona el dispositivo recién creado y se copia su <i>access token</i> .	77
6.14	Descripción esquemática del fichero <i>ota_config.json</i>	79
6.15	Descripción esquemática del fichero <i>wifi_config.json</i>	80

6.16	Fichero de ejemplo lora_config.json	81
7.1	Esta captura muestra parte de la salida que emite...	84
7.2	Estructura de temas creados por el gateway Lora-MQTT	85
7.3	Estado visual del Dashboard	88
7.4	Cuadro emergente que aparece cuando se pulsa la opción <i>Añadir paquete</i>	88
7.5	Se indica el icono con la acción <i>Edit firmware</i>	89
7.6	Estado visual del Dashboard después de una actualización OTA exitosa	89
8.1	Tabla de objetivos cumplidos	92

Glosario

- BLE** (*Bluetooth Low Energy*) Tecnología de red de área personal inalámbrica que es una evolución del Bluetooth clásico y proporciona un menor consumo de energía junto a otras nuevas características. 6
- ESP32** Microcontrolador de bajo coste y alto rendimiento desarrollado por Espressif Systems. 11
- firmware** Software que establece la lógica de más bajo nivel que controla los circuitos electrónicos de un dispositivo. 1
- flashear** (del inglés *flash*) Instalar o reinstalar el firmware de un dispositivo. 31
- FOTA** (*Firmware OTA*) Clase de OTA en la que se actualiza el firmware del dispositivo. 10
- Gateway** Dispositivo de red que actúa como un punto de acceso entre dos redes diferentes, permitiendo la comunicación y la transferencia de datos entre ellas. 18
- IoT** (*Internet of Things*) Área de la informática vinculada a la interconexión de dispositivos físicos que reciben y transfieren datos a través de redes inalámbricas. 1
- LoRa** (*Long Range*) Tecnología de comunicación de largo alcance y bajo consumo, diseñada para aplicaciones del Internet de las Cosas. 6
- Micropython** Implementación del lenguaje de programación Python optimizada para poder ejecutarse en un microcontrolador. 5
- OTA** (*Over The Air*) Dicho de un método para distribuir e instalar actualizaciones de software, firmware o configuraciones a dispositivos conectados de forma remota. 1
- SOTA** (*Software OTA*) Clase de OTA en la que se actualizan elementos de software tales como aplicaciones, configuraciones, y funcionalidades del sistema a nivel de usuario. 10
- Thingsboard** Plataforma IoT de código abierto. 6

CAPÍTULO 1

Introducción

Las conocidas como **actualizaciones OTA**, por sus siglas en inglés *Over The Air* —por el aire— son aquellas que se producen de forma remota e inalámbrica, siendo su finalidad la de actualizar el firmware¹, las aplicaciones y/o la configuración de un dispositivo a través de la red. Éstas implican un proceso que comprende tanto la distribución de un paquete de actualización, sea de una naturaleza u otra, como la propia instalación en el dispositivo. Se trata de una funcionalidad extendida en numerosos entornos debido a sus grandes ventajas, estando presente en sistemas IoT, computación en la nube, dispositivos móviles, sistemas empotrados en general e incluso vehículos. [1]

Concretamente, dentro del campo del **IoT** (*Internet of Things*), las actualizaciones OTA suponen una ventaja decisiva en la **gestión de flotas**, pues nos permiten modificar el software —o incluso el firmware— de un dispositivo sin interactuar físicamente con él, de tal forma que no se interrumpe la función que este desempeña dentro del sistema IoT o, en caso de producirse una interrupción, esta es mínima comparativamente. Esto supone una diferencia fundamental respecto a las actualizaciones que se hacen manualmente, donde se accede al dispositivo físicamente y, por ejemplo, se sustituye la memoria flash donde almacena el sistema operativo, o se le carga nuevo código mediante una conexión USB y usando alguna herramienta de desarrollo. Aunque estas actualizaciones de naturaleza más manual son normalmente indispensables en las etapas de desarrollo previas al despliegue de un dispositivo, carecen de las ventajas que tienen las OTAs en dispositivos que ya están operativos dentro de una red IoT.

Sin actualizaciones OTA, dispositivos ya desplegados pueden convertirse fácilmente en fósiles tecnológicos, incapaces de resolver problemas inesperados o cambios en las especificaciones, mientras no reciban una intervención manual muchas veces inconveniente. Entre los escenarios para los cuales las actualizaciones OTA son útiles, se encuentran los siguientes:

- Resolución de vulnerabilidades mediante parches de seguridad.
- Mejoras de características a lo largo del ciclo de vida del dispositivo.
- Reducción de costes por medio de eliminar visitas de mantenimiento al entorno donde un dispositivo ha sido desplegado.

¹*firmware*: Software que establece la lógica de más bajo nivel que controla los circuitos electrónicos de un dispositivo.

En un sistema IoT que admita actualizaciones OTA es esencial su integración en una **plataforma** que actúe como servidor de actualizaciones, e incluya una consola de administración o dashboard, para proporcionar a los usuarios con los permisos adecuados una interfaz con la que manejar los procesos de actualización y conocer las versiones de los distintos dispositivos. La plataforma debe gestionar los paquetes de actualización y hacerlos fluir hacia los dispositivos a través la red, idealmente contemplando las distintas casuísticas que puedan surgir: compatibilidad o incompatibilidad de determinados dispositivos con un cierto paquete de actualización, actualizaciones en lote de múltiples dispositivos, conexión con los dispositivos a través de diferentes tecnologías de red inalámbricas, seguridad en la comunicación, etc. Es vital también la generación de paquetes de actualización, cómo son gestionados por la plataforma, y, sobre todo, cómo son instalados finalmente en los dispositivos.

Este proyecto propondrá la implementación de un sistema IoT capaz de realizar actualizaciones OTA. Sirviéndonos de una conocida plataforma IoT, desarrollaremos una solución dirigida a funcionar con un amplio rango de dispositivos IoT, y resolveremos escenarios de conectividad que hasta la fecha no han sido tratados extensamente en el mundo del código abierto.

1.1 Motivación

Los proyectos enmarcados en el denominado Internet de las Cosas a menudo usan un rango de tecnologías de red y hardware mucho más amplio en comparación a los sistemas distribuidos que podemos englobar en el Internet tradicional. Por un lado, en lo relativo a la conectividad, mientras en el Internet tradicional encontramos servicios en su mayoría conectados mediante protocolos sobre la pila TCP/IP, en las comunicaciones de dispositivos IoT encontramos una variedad de tecnologías inalámbricas que de entrada no funcionan con IP, tales como Zigbee, Bluetooth o LoRa². Por otro lado, mientras en casos como centros de datos, servicios Web, o aplicaciones móviles, que ocupan una grandísima parte de los desarrollos informáticos, existe un conjunto de sistemas operativos predecible (normalmente Linux y Windows en los servidores, con Android y Mac también en la parte de usuario final), en dispositivos IoT encontramos una mayor dispersión en cuanto a su firmware, incluyendo sistemas operativos como FreeRTOS, Zephyr, o frameworks de programación de microcontroladores sin sistema operativo como Arduino, Micropython³, ESP-IDF o PlatformIO. La razón de este último aspecto tiene su origen en el hardware de estos dispositivos, basados en microcontroladores en muchos casos, y diseñados para ajustarse a su propósito reduciendo todo lo posible el coste económico y la energía que consumen. Del mismo modo se explica la dispersión existente en las tecnologías de comunicación inalámbrica en IoT, que están pensadas para ajustarse de la forma más ceñida posible a los requisitos de distancia y ancho de banda, que atienden a los escenarios físicos donde se ubican las diversas *cosas* del Internet de las Cosas.

²LoRa (*Long Range*): tecnología de comunicación de largo alcance y bajo consumo, diseñada para aplicaciones de Internet de las Cosas. [2]

³Micropython: Implementación del lenguaje de programación Python optimizada para ejecutarse en microcontroladores. [6]

Esta dispersión en lo relacionado con la conectividad, el hardware y el firmware de dispositivos IoT ha hecho, posiblemente, que ciertas tecnologías y conceptos se desarrollen con más dificultad en este ámbito. En sistemas distribuidos donde se asume que los nodos pueden conectarse mediante TCP/IP y tienen la suficiente potencia de hardware como para ejecutar contenedores, existen herramientas como Kubernetes⁴, que hace tiempo solucionaron el problema de las actualizaciones remotas y la gestión remota de software de una forma muy elegante. Sin embargo, si ampliamos el espectro de dispositivos posibles, incluyendo microcontroladores, o incluimos diversas formas de conectividad más allá de TCP/IP, no existe una solución tan elegante.

Actualmente, las plataformas que permiten gestionar flotas de dispositivos IoT con actualizaciones OTA están limitadas a unos casos concretos, que no cubren muchos de los escenarios que podemos encontrar en sistemas IoT. Por ejemplo, existen soluciones basadas en contenedores, pero eso implica que los dispositivos han de tener los suficientes recursos para instalar Linux. En lo concerniente a la conectividad, difícilmente se pueden encontrar plataformas que gestionen actualizaciones OTA sobre nodos que no estén conectados directamente con IP.

Idealmente, imaginamos que podría existir una plataforma IoT que gestionase una **red de dispositivos híbrida**, incluyendo tanto microcontroladores como dispositivos más sofisticados, conectados potencialmente por tecnologías inalámbricas diferentes, y todos ellos pudiesen ser actualizados de forma remota en caso de poseer el firmware adecuado. Conseguir algo así, aunque escapa de las posibilidades de este trabajo, es la idea que guiará su dirección.

En esa línea, buscaremos una solución que permita la realización de actualizaciones OTA a través de **3 tecnologías de red inalámbrica** diferentes y que funcione en un **rango de dispositivos lo más amplio posible**. Dicha solución habrá de integrarse sin fisuras en una plataforma IoT de manera que se puedan gestionar los distintos dispositivos de forma transparente, desde una **interfaz común**.

1.2 Objetivos

En base a la motivación expuesta en la sección anterior, se propone desarrollar un proyecto guiado por los siguientes objetivos principales:

1. **Usando una plataforma IoT, establecer una interfaz común para gestionar actualizaciones OTA de naturaleza heterogénea.**

Se buscará una solución que permita gestionar las versiones de los dispositivos de la flota y lanzar actualizaciones OTA, a pesar de las diferencias subyacentes respecto al hardware de los dispositivos y el protocolo de red mediante la cual estén conectados. Para ello se intentará aprovechar una plataforma IoT existente, que será configurada adecuadamente.

⁴Kubernetes: plataforma de código abierto para automatizar la implementación, el escalado y la administración de aplicaciones en contenedores que pueden estar alojados en distintos nodos dentro de una red. <https://kubernetes.io/es/>

Se anticipa que la plataforma IoT elegida, como se justifica en un capítulo posterior, será **Thingsboard**⁵.

2. Implementar la actualización OTA para dos familias de dispositivos en el borde.

Dada la aspiración de interoperabilidad que tiene este trabajo, se pretende abarcar un rango amplio de dispositivos sobre los cuales se realizarán las OTAs, planteando dos familias de dispositivos que cubren gran parte de los empleados en el ámbito IoT:

a) **Dispositivos de bajos recursos con microcontrolador.**

Usados para realizar tareas relacionadas con sensores y actuadores conectados a su placa, tienen además un bajo consumo de energía. Se buscará una forma de programar estos dispositivos compatible con un buen número de ellos y se implementará la OTA sobre él.

La tecnología escogida para programar esta clase de dispositivos será **Micropython**, según se justifica más adelante en esta memoria.

b) **Dispositivos con microprocesador y capacidad para ejecutar Linux.**⁶

Empleados para realizar funciones que requieren más capacidad de procesamiento o un flujo de datos mayor. Se buscará la forma de crear imágenes de Linux que puedan ser instaladas y actualizadas mediante OTA.

3. Implementar la actualización OTA a través de distintos escenarios de conectividad.

Del mismo modo que se plantean varias clases de hardware en los dispositivos de borde, se plantean varias tecnologías de red mediante las cuales puedan estar conectados dichos dispositivos a la plataforma.

a) **Red IP**

Es el tipo de conexión más extendida y directa. La implementación de la OTA a través de IP incluye varias tecnologías de red tales como Wi-Fi, Ethernet o NB-IoT.

b) **Red WPAN**

Clase de red caracterizada por su corto alcance y bajo consumo de energía, apropiada para sensores y dispositivos portátiles. Para implementar la OTA sobre ella se seleccionará una de las tecnologías pertenecientes a esta categoría, dentro de la cual destacan BLE⁷ y ZigBee.

c) **Red LPWAN**

Ideal para escenarios que requieren comunicación a largas distancias y bajo consumo energético. También se elegirá una tecnología concre-

⁵*Thingsboard*: Plataforma IoT de código abierto, que se presenta como una solución para la administración de dispositivos, recolección de datos, procesamiento y visualización. [4]

⁶Finalmente, este objetivo no pudo satisfacerse en este trabajo debido a limitaciones temporales, si bien se aborda la forma de resolverlo y se propone una línea de trabajo para conseguirlo.

⁷BLE (*Bluetooth Low Energy*): tecnología de red de área personal inalámbrica que es una evolución del Bluetooth clásico y proporciona un menor consumo de energía junto a otras nuevas características. [3]

ta perteneciente a esta categoría para implementar la OTA sobre ella, como Sigfox, LoRa o LoRaWAN⁸.

Conforme más adelante se justifica, las tecnologías de red elegidas serán, por el orden expuesto: **MQTT sobre Wifi**, **BLE** (*Bluetooth Low Energy*) y **LoRa** (*Long Range*).

4. Desarrollar componentes reutilizables que permitan replicar el sistema de OTAs implementado.

Los desarrollos realizados procurarán la creación de componentes que podrán ser reutilizados, ya sea en forma de bibliotecas, programas de servidor, o scripts de utilidades, con su debida documentación. Siempre y cuando se ajuste a los escenarios contemplados, el usuario de estos componentes podrá integrar la gestión de OTAs en sus aplicaciones específicas.

1.3 Metodología

Este trabajo se materializó en la creación de un proyecto de código, versionado con git y alojado en Github, que se puede encontrar en el siguiente enlace:

<https://github.com/p4blo0S/thingsboard-updatable-devices.git> [5]

Para desarrollarlo, seguimos un enfoque de **desarrollo incremental**, el cual permite construir y mejorar el sistema planteado en iteraciones sucesivas. Este enfoque facilita la adaptación a cambios y la incorporación de nuevas funcionalidades de manera continua, asegurando que cada fase del desarrollo se base en los resultados y aprendizajes obtenidos en iteraciones anteriores. A continuación, se describen las fases seguidas en este proyecto:

- **Planificación:** En esta fase inicial, se definieron los objetivos del sistema y, tras una investigación previa sobre el estado del arte, se seleccionó un conjunto de tecnologías y herramientas para su desarrollo. Se establecieron las bases para las siguientes etapas.
- **Desarrollo Incremental:** El sistema se implementó en iteraciones, cada una de la cuales se centró en desarrollar y probar unas características específicas, integrando las anteriores y, en su caso, reajustándolas. Concretamente, las iteraciones del desarrollo incluyeron:⁹
 - **Iteración 1: Prueba de concepto**
A modo de prueba de concepto, en esta fase se desarrolló un sistema IoT minimalista que cumpliera la mínima funcionalidad para realizar una OTA en el escenario de conectividad más sencillo.
 - **Iteración 2: Actualización OTA sobre dispositivo Micropython conectado por MQTT**

⁸LoRaWAN (*Long Range Wide Area Network*): protocolo de red que utiliza la tecnología LoRa para gestionar la comunicación entre dispositivos IoT y gateways en redes de área amplia. [2]

⁹La elección concreta de estas iteraciones se justifica con la propuesta que se expone en el capítulo 3.

Usando como base el código desarrollado en la prueba de concepto, se implementó la OTA sobre Micropython en conjunto con una biblioteca y otras utilidades para gestionar el software del dispositivo. Además, en el lado de Thingsboard, se elaboró una primera versión de la interfaz común para gestión de OTAs. En las siguientes fases, los componentes mencionados serían refinados para integrar en ellos las nuevas características.

- **Iteración 3: Integración de un dispositivo conectado por BLE**

Para implementar esta forma de conexión, la infraestructura tuvo que ser ampliada, agregando algunos elementos de comunicación intermedios entre el servidor Thingsboard y el dispositivo. En lo concerniente a la programación del propio dispositivo, aparte de implementar la comunicación mediante BLE, se hizo una refactorización del código anterior en la línea de crear una biblioteca que pudiese manejar la comunicación con Thingsboard a través de los 3 escenarios de conectividad propuestos.

- **Iteración 4: Integración de un dispositivo conectado por LoRa**

Se consiguió la comunicación y la actualización OTA a través de LoRa gracias a introducir nuevos elementos de comunicación intermedios entre el dispositivo y la plataforma. En el código Micropython, se completó la biblioteca mencionada con las nuevas funciones capaces de manejar el driver de LoRa.

- **Iteración 5: Integración de un dispositivo Linux**

Desafortunadamente, por limitaciones de tiempo, esta iteración no pudo ser completada finalmente. No obstante, a lo largo de este trabajo queda expuesta la investigación que se ha hecho en relación a esta característica y la línea de trabajo que se propone.

- **Integración y evaluación:** Al final de cada iteración, se llevó a cabo la integración de las nuevas funcionalidades y se realizaron pruebas para asegurar que el sistema evolucionase de manera efectiva.

Este enfoque inspirado en el concepto de desarrollo incremental, aunque no se adhiere a una metodología formal, permitió una gran flexibilidad y adaptabilidad, cosa fundamental para abordar los desafíos que surgieron durante el proceso de desarrollo.

1.4 Estructura de la memoria y convenciones

Esta memoria se organiza en los siguientes capítulos:

- **Capítulo 1: Introducción.** Presenta la base conceptual de este trabajo, con la motivación que lo ha inspirado, los objetivos que persigue y la metodología planteada para alcanzarlos.
- **Capítulo 2: Estado del arte.** Describe en profundidad el estado tecnológico actual relativo a las actualizaciones OTA, explicando en primer lugar los

distintos enfoques que se practican, continuando con un análisis de las distintas tecnologías a nuestro alcance, tanto en las plataformas IoT como en los dispositivos del borde, y finalmente revisando las posibilidades de las OTAs a través de diferentes tecnologías de red.

- **Capítulo 3: Propuesta.** En base al panorama tecnológico expuesto, se realiza una propuesta, que pasa por analizar las limitaciones actuales y plantear el desarrollo de un proyecto que contribuya a mitigar algunas de ellas, no sin antes hacer un balance de las distintas alternativas que tenemos a nuestra disposición.
- **Capítulo 4: Diseño de la solución.** Se hace una descripción del sistema que hemos ideado para abordar la propuesta, incluyendo su arquitectura, las tecnologías utilizadas y el diseño detallado de cada uno de sus componentes.
- **Capítulo 5: Desarrollo de la solución propuesta.** Documenta el proceso de desarrollo de la solución a través de sus distintas iteraciones, plasmando las decisiones tomadas, las dificultades encontradas y cómo se resolvieron.
- **Capítulo 6: Implantación.** Presenta la puesta en práctica del sistema desarrollado, con un método exhaustivo para desplegar todos los componentes desarrollados y así llevar el sistema creado a distintos escenarios.
- **Capítulo 7: Pruebas.** Reúne las distintas pruebas que se han realizado para comprobar el correcto funcionamiento del sistema, tanto desde el punto de vista de cada componente individual como de su integración conjunta, así como se observa la experiencia de usuario y el rendimiento del sistema.
- **Capítulo 8: Conclusiones.** Finaliza este trabajo revisando el éxito o fracaso obtenidos en la consecución de cada objetivo planteado inicialmente, proponiendo trabajos futuros en la línea de éste y justificando su relación con los estudios cursados.
- **Apéndices:** Lo forman algunos archivos, o fragmentos de archivos, que se han empleado en el desarrollo del proyecto, junto con los denominados *Objetivos de Desarrollo Sostenible*.

Para concluir con esta introducción, aclararemos a continuación algunas cuestiones relativas la redacción del texto por si no resultasen del todo evidentes:

- Usamos el término *OTA* indistintamente al de *actualización OTA* como suele hacerse por economía verbal cuando se trata este tema.
- Empleamos el concepto de *borde* (*edge*, en inglés) en alusión a los dispositivos que forman parte de un sistema IoT y se encuentran cerca de la fuente de datos —por ejemplo, sensores—, en contraposición a la *plataforma* donde esos datos son enviados.
- Respecto a las referencias a páginas web, en los casos donde se referencie una subpágina dentro de una página que ya haya sido citada con una entrada en el índice bibliográfico, se optará por indicar la URL sencillamente

con una nota al pie, con el fin de evitar sobrecargar el índice bibliográfico y aportar comodidad.

CAPÍTULO 2

Estado del arte

Para poder llevar a cabo la contribución que nos hemos propuesto realizar al mundo de los proyectos IoT, primero hemos de analizar en profundidad el estado tecnológico en que este se encuentra. Lo haremos en este capítulo, con especial énfasis en la temática de las actualizaciones OTA, pero sin obviar todas las perspectivas que nos afectan, ya que este análisis será fundamental para posteriormente seleccionar con buen criterio las tecnologías que usaremos.

En primer lugar, debemos tener en cuenta los diferentes enfoques con que se aborda habitualmente la problemática de las OTA, por ejemplo, para conocer las clases de OTA existentes, cuál nos conviene más en base a nuestra motivación, y si uno u otro enfoque es capaz de integrarse bien con determinadas plataformas y determinados dispositivos. A ese respecto, la tecnología que elijamos para programar los dispositivos será crucial, pues no solo influye en la naturaleza de las OTAs que podamos realizar, sino también en la eficiencia, la facilidad de programación, o el número de dispositivos soportados. Por el otro lado, las plataformas también pueden limitarnos a un tipo concreto de dispositivos, y determinarán, no solo la lógica de comunicación para transferir los paquetes de actualización, sino cualquier intercambio de datos con los dispositivos. Una plataforma puede facilitar la conexión de los dispositivos a través de distintos protocolos de red, limitarlos a uno sólo, o dejar en manos del desarrollador la integración con las distintas formas de conectividad.

Trataremos todas estas cuestiones con una atención especial a las tecnologías de **código abierto**, ya que son múltiples las ventajas que ofrecen para un trabajo como este. En primer lugar, su accesibilidad, ya que en la inmensa mayoría de los casos son gratuitas. En segundo lugar, porque muchas veces tienen grandes comunidades detrás, con usuarios que a su vez publican otros proyectos basados en dicha tecnología, los cuales podemos aprovechar o tomar como ejemplo; además, suele haber en Internet información publicada relativa a dudas y problemas que se encuentran esos usuarios. En tercer lugar, permiten una flexibilidad de personalización que no es posible con el código propietario, cosa que, anticipamos, nos será especialmente necesaria en el desarrollo de este trabajo. Por último, fomentar el código abierto es algo que beneficia a todos, ya que contribuye a la colaboración entre desarrolladores de todo el mundo, acelerándose así la innovación.

2.1 Enfoques habituales en las actualizaciones OTA

A la hora de investigar las posibilidades para la realización de actualizaciones OTA en contextos determinados, a menudo encontramos casos donde ciertos enfoques o conceptos se dan por hecho, pero no son triviales, ya que la actualización OTA es un término amplio y, en especial para un proyecto como el nuestro, elegir un enfoque u otro supone una gran diferencia.

1. FOTA *vs* SOTA [8]

Se considera FOTA (firmware OTA) a la clase de OTA en la que se actualiza el firmware, es decir, el código que controla directamente el hardware del dispositivo. Suponen la actualización del sistema operativo si el dispositivo objetivo posee uno. Cuando se habla de OTA en microcontroladores, a menudo se asume que se trata de esta clase concreta, aunque no necesariamente tiene que ser así.

Se considera SOTA (software OTA) a la clase de OTA en la que se actualizan elementos de software que se sitúan por encima del núcleo del sistema operativo, incluyendo aplicaciones, configuraciones, y funcionalidades del sistema a nivel de usuario.

2. Transferencia del paquete de actualización:

Descarga desde un repositorio *vs* Envío activo

Es común una práctica relacionada con la integración continua¹ basada en que un dispositivo se descargue la última versión de su propio software o firmware desde un repositorio para posteriormente auto instalarse la actualización. A menudo, el dispositivo comprueba periódicamente si ha habido cambios para saber cuándo iniciar la descarga, aunque también se puede dar el caso en el que la plataforma le indique al dispositivo, bajo ciertas condiciones, el momento en que debe iniciar la actualización, junto a un enlace hacia la nueva versión. Si bien tiene grandes ventajas esta aproximación, no siempre es posible, ya que asume que el dispositivo puede conectarse mediante TCP/IP al servidor desde donde descargarse el paquete, normalmente mediante HTTP(s).

Para los casos donde no sea posible o, sencillamente, no sea conveniente ese método, se puede realizar un envío activo del paquete al dispositivo del borde, ya sea directamente desde una plataforma IoT, a través de algún intermediario con la plataforma IoT, o bien sin contar con dicha plataforma, por ejemplo, desde un teléfono móvil directamente hacia el dispositivo.

3. Prácticas para una actualización OTA profesional [9]

- a) Capacidad de *fallback* — Es la habilidad de un dispositivo para revertir a una versión anterior del firmware o software en caso de que la nueva actualización falle o cause problemas. En el caso de las actualizaciones FOTA, esta capacidad ha de residir en el *bootloader*².

¹*integración continua*: Práctica de desarrollo de software que implica la integración frecuente de cambios en el código en un repositorio compartido.

²*bootloader*: Programa esencial que se ejecuta al encender un dispositivo, responsable de cargar el sistema operativo o el firmware en la memoria del dispositivo.

- b) Reportes y monitorización — El dispositivo debe reportar el estado en que se encuentra (actualizando, éxito, fracaso, etc...), permitiendo la monitorización remota.
- c) Verificación — El dispositivo ha de comprobar la integridad del paquete de actualización recibido, así como su correcta instalación.
- d) Administración de los paquetes de actualización — Este aspecto puede incluir el versionado, la validación de paquetes y la opción de crear actualizaciones incrementales³.

2.2 OTAs en los dispositivos del borde

Hoy en día, las actualizaciones OTA se han convertido en una práctica esencial en diversos ámbitos, abarcando una amplia gama de dispositivos que van desde teléfonos inteligentes y tabletas hasta electrodomésticos inteligentes y vehículos conectados. En el sector de los *smartphones*, tanto Android como iOS reciben actualizaciones de software y seguridad de manera regular, mejorando la experiencia del usuario y corrigiendo vulnerabilidades. En el ámbito del Internet de las Cosas, dispositivos como sensores, cámaras de seguridad o localizadores, se benefician de actualizaciones OTA para optimizar su rendimiento y añadir nuevas funcionalidades. Incluso en la industria automotriz, los vehículos más modernos reciben actualizaciones de firmware que mejoran sus sistemas de navegación, seguridad y rendimiento.

Podemos imaginar que, al abarcar dispositivos tan diferentes, cada sistema de actualizaciones OTA tendrá una naturaleza muy distinta, aunque puedan compartir algunas características comunes. Por lo tanto, es conveniente que acotemos el ámbito de nuestra investigación a unos escenarios específicos. Concretamente, lo haremos pensando en el ámbito del IoT y, en particular, nos centraremos en las llamadas **placas de desarrollo**⁴, pensando en el caso de uso en que un desarrollador prototipa o crea su proyecto IoT con estos dispositivos, empleando para ello tecnologías de código abierto —descartando así soluciones de terceros que solo valgan para cierta línea de productos. Arduino Nano, ESP32⁵, BeagleBone o Raspberry Pi son placas de desarrollo populares en el mercado que se enmarcarían dentro de nuestro ámbito de interés.

Sin embargo, el ámbito de las placas de desarrollo programables sigue siendo demasiado amplio, así que proponemos una división en dos grandes familias de placas, de forma que posteriormente buscaremos una solución específica para cada una de ellas. Estas dos familias vienen a ser, por un lado, las placas basadas en microcontrolador, y, por otro lado, las placas basadas en microprocesador.

³*actualización incremental*: Tipo de actualización que solo incluye los cambios o mejoras realizadas desde la última versión, en lugar de descargar y reinstalar todo el firmware/software completo.

⁴*placa de desarrollo*: plataforma de hardware que permite a ingenieros, desarrolladores y entusiastas de la electrónica crear, prototipar y experimentar con proyectos de hardware y software. Este tipo de placas suelen incluir un microcontrolador o microprocesador, así como una variedad de interfaces y conectores que facilitan la conexión de sensores, actuadores y otros componentes electrónicos.

⁵ESP32: Microcontrolador de bajo coste y alto rendimiento desarrollado por Espressif Systems.

2.2.1. Placas basadas en microcontrolador

Generalmente, este tipo de placas están diseñadas para tareas específicas y no ejecutan sistemas operativos complejos. Se programan directamente y tienen poca memoria RAM (del orden de los KB), lo que las hace ideales para aplicaciones simples y de bajo consumo energético. Las actualizaciones de firmware (FOTA) para estas placas dependen de su arquitectura interna, por lo que un determinado paquete de actualización únicamente serviría para un modelo de placa en específico. Por el contrario, si bien no es lo más común, una actualización de software (SOTA) podría realizarse para varios modelos de dispositivos siempre y cuando tuviesen un firmware subyacente capaz de ejecutar el software en cuestión.

Con la idea de abarcar un rango de dispositivos lo más amplio posible, deberemos elegir un **sistema operativo o framework de programación que soporte un buen número de estas placas**, de manera que podamos programar una solución compatible con todas ellas. Con esa condición, estas son algunas opciones populares de código abierto:

- **Zephyr:** Sistema operativo de tiempo real (RTOS) diseñado para dispositivos embebidos y el IoT. Es modular y escalable, lo que permite su uso en una amplia variedad de aplicaciones, desde dispositivos de bajo consumo hasta sistemas más complejos. Soporta múltiples arquitecturas de hardware, ofrece actualizaciones OTA, y tiene un enfoque en la seguridad y la conectividad. Aunque su curva de aprendizaje es bastante pronunciada, supera a sus competidores en flexibilidad y características avanzadas. [10]
- **FreeRTOS:** También se trata de un sistema operativo de tiempo real, diseñado para microcontroladores y sistemas embebidos. Ampliamente utilizado en aplicaciones de IoT y dispositivos de consumo, es conocido por su facilidad de uso y su capacidad para integrarse con otros sistemas y bibliotecas. [11]
- **Arduino:** Plataforma de hardware y software que permite a los usuarios crear proyectos interactivos y electrónicos. Incluye una serie de placas de desarrollo y un entorno de programación fácil de usar, ofreciendo además una amplia gama de bibliotecas y ejemplos, lo que facilita la programación de dispositivos. Es ideal para principiantes y se utiliza en educación, prototipado y proyectos de hobby. [12]
- **Micropython:** Implementación del lenguaje de programación Python diseñada para ejecutarse en microcontroladores y dispositivos embebidos. Permite a los desarrolladores utilizar Python para programar hardware de manera sencilla y eficiente, proporcionando un subconjunto del lenguaje optimizado para funcionar en entornos con recursos limitados. Es fácil de usar y permite la rápida creación de prototipos, siendo popular en proyectos de IoT y educación. [6] [7]

Dispositivos con Micropython

Dado que, tal como justificaremos en el capítulo siguiente, nos decidiremos por la opción de Micropython, esta tecnología merece un análisis más específico en lo relativo a las posibilidades que otorga para la realización de OTAs.

En primer lugar, se debe aclarar que Micropython no ofrece una forma de realizar **FOTAs** común a todas las placas soportadas. Por el contrario, para realizar actualizaciones de esta naturaleza, la documentación solo muestra métodos específicos para las placas ESP32⁶ y WiPy⁷. En el primer caso, la actualización FOTA se consigue escribiendo el nuevo firmware directamente sobre las particiones de la memoria flash, usando una biblioteca exclusiva de la ESP32, y efectuando luego un *hard reset*⁸. En el segundo caso, el método consiste en la escritura del archivo de firmware en un directorio específico del sistema de archivos (`/flash/sys/`), que en realidad es una pasarela a un sistema de archivos interno e invisible para el usuario, de manera que el nuevo firmware se instalará al reiniciar la placa.

Como vemos, la FOTA en principio no nos serviría para llegar a un amplio rango de dispositivos, así que exploraremos la **SOTA** en Micropython. Aunque no existe un método estipulado en la documentación oficial de Micropython, los desarrolladores suelen seguir una aproximación que aprovecha de una forma inteligente la naturaleza de Micropython. La clave está en que Micropython, aprovechando la memoria flash, expone un sistema de archivos donde, además de almacenarse los scripts y las bibliotecas que se pueden ejecutar, también se pueden guardar todo tipo de archivos y directorios como si se tratase de un sistema operativo. De esta forma, para efectuar una actualización de bibliotecas, configuraciones o la propia aplicación principal, bastaría con instalar una serie de ficheros, que sobrescriban a los que ya hay.

Estos son los proyectos de código abierto que, a nuestro parecer, mejor siguen esta aproximación entre los que hemos encontrado, con sus aspectos destacables:

- **micropython-ota**, del usuario *olivergregorius* — <https://pypi.org/project/micropython-ota/>
Vía HTTPs descarga los nuevos archivos de un servidor de *git* capaz de contener versiones sucesivas, y usa una lógica de reinicio interesante a la hora de aplicar la actualización, en la cual podemos inspirarnos.
- **cOTA**, del usuario *roehann* — <https://github.com/roehann/cota>
Desarrollado para la variante de Micropython llamada CircuitPython, este proyecto está hecho específicamente para lanzar la actualización desde la plataforma Thingsboard. El dispositivo participa en el intercambio de mensajes regido por Thingsboard y recibe un enlace a un repositorio de código, de donde descarga la nueva versión.
- **uOTA**, del usuario *mkomon* — <https://github.com/mkomon/uota>
Esta biblioteca tiene la particularidad de descargar la actualización en un único archivo comprimido en formato TAR GZ⁹, el cual descomprime en el

⁶Véase <https://docs.micropython.org/en/latest/library/esp32.html#flash-partitions>

⁷Véase <https://docs.micropython.org/en/latest/wipy/general.html#upgrading-the-firmware-over-the-air>

⁸*hard reset*: Reinicio completo del dispositivo que restablece todos sus registros y configuraciones a sus valores predeterminados. Se realiza generalmente mediante la desconexión de la alimentación o mediante un pin de reset específico en el microcontrolador.

⁹formato TAR GZ: Combinación de dos formatos de compresión y archivo: TAR (Tape Archive) y Gzip (GNU zip). Empleado comúnmente en sistemas Unix para agrupar y comprimir archivos y directorios.

momento de la actualización. Cuenta además con un archivo de configuración que permite excluir o borrar determinados archivos durante la actualización.

2.2.2. Placas basadas en microprocesador

Generalmente, este tipo de placas se utilizan para realizar funciones que requieren más capacidad de cómputo o un flujo de datos mayor. Suelen ejecutar **sistemas operativos basados en Linux**. Se programan utilizando entornos de desarrollo más complejos y cuentan con una mayor cantidad de memoria RAM (del orden de los MB, cuando no GB), lo que las hace adecuadas para aplicaciones más sofisticadas. Muchas veces, el firmware que usan estas placas —a menudo una distribución de Linux— ha sido compilado en específico para el propio modelo de la placa en cuestión, con el fin de aprovechar el hardware al máximo; pero también es común usar firmware genérico, que normalmente tiene el único requisito de ser compatible con la arquitectura ARM, la más popular en este tipo de placas de desarrollo.

Por lo tanto, en caso de elegir Linux para programar este tipo de placas, no deberíamos tener problema en realizar tanto SOTAs como FOTAs en un gran número de estos dispositivos, siempre y cuando asumamos, en este segundo caso, que dichos dispositivos tienen la arquitectura ARM. Podemos dar por descontado que habrá soluciones al respecto de las actualizaciones OTA en un ecosistema tan extenso como es Linux.

A continuación analizamos, desde dos enfoques diferentes, una selección de soluciones de código abierto para programar placas basadas en Linux.

- Herramientas para la construcción de imágenes de Linux personalizadas (ordenadas de menor a mayor facilidad de uso):
 - **Yocto** : Proyecto que usa *OpenEmbedded*¹⁰ como base para crear imágenes de Linux personalizadas, permite a los desarrolladores construir sistemas operativos completos y adaptados a sus necesidades, ofreciendo flexibilidad y control sobre el software y los paquetes incluidos. [13]
 - **PTXdist** : Herramienta de construcción de sistemas Linux embebidos que permite crear imágenes de sistema personalizadas. Se centra en la simplicidad y facilidad de uso, proporcionando un entorno de desarrollo que facilita la integración de software y la gestión de configuraciones. [14]
 - **Buildroot** : Herramienta que simplifica la creación de sistemas Linux embebidos. Permite a los desarrolladores generar imágenes de sistema completas de manera rápida y eficiente, con un enfoque en la simplicidad y facilidad de configuración. Es especialmente popular por su rapidez en la generación de imágenes. [15]

¹⁰*OpenEmbedded*: Framework que permite la creación de distribuciones Linux personalizadas para dispositivos embebidos. <https://www.openembedded.org>

- Distribuciones adecuadas para usar en placas de desarrollo, sobre las cuales podemos instalar nuestro propio software:
 - **Raspberry Pi OS:** Sistema operativo oficial para Raspberry Pi, basado en Debian y optimizado para el hardware de la Raspberry Pi, que proporciona un entorno fácil de usar. [16]
 - **Fedora IoT:** Versión de Fedora diseñada para dispositivos del Internet de las Cosas, que ofrece un entorno modular y seguro, permitiendo la creación de aplicaciones IoT. [17]
 - **Ubuntu Core:** Versión de Ubuntu enfocada en dispositivos embebidos e IoT, que utiliza un sistema de paquetes snap para la instalación y actualización de aplicaciones de manera segura y aislada. [18]
 - **Alpine:** Distribución ligera y segura, diseñada para ser eficiente en recursos y con un enfoque en la seguridad, permitiendo la personalización y optimización del sistema. Aunque su uso más extendido está en la construcción de imágenes para contenedores, por su extrema livianidad, también puede ser una buena opción para dispositivos embebidos. [19]

OTAs en dispositivos con Linux

Dado que, sea cual sea la opción que elijamos para la placa basada en Microprocesador, esta acabará con una imagen Linux instalada, exploraremos a continuación las soluciones que permiten realizar actualizaciones OTA en cualquier dispositivo con Linux embebido.

- **Mender** [20]
Plataforma de gestión de actualizaciones OTA que permite a los desarrolladores implementar actualizaciones de software de manera segura y eficiente. Ofrece características como la verificación de integridad de las actualizaciones, la posibilidad de realizar *rollbacks* en caso de fallos y una interfaz web para gestionar el proceso de actualización. Funciona tanto para Linux como para sistemas operativos de tiempo real como Zephyr.
- **RAUC (Robust Auto-Update Controller)** [21]
Es a la vez un cliente de actualizaciones para Linux embebido y una herramienta que permite crear, inspeccionar y modificar distintas clases de artefactos de actualización para un dispositivo. Cuenta con soporte por defecto para Yocto, Buildroot, PTXdist, y distribuciones como Arch Linux, Debian o Nix OS, lo cual lo hace extremadamente versátil en diferentes entornos. Además se centra especialmente en la seguridad y la robustez, permitiendo la creación de paquetes de actualización que pueden ser firmados y verificados.
- **SWupdate** [22]
Framework que soporta múltiples métodos de actualización OTA, incluyendo actualizaciones de firmware y de aplicaciones, y permite la personalización de los procesos de actualización mediante scripts. Permite realizar

OTAs desde múltiples interfaces, ya sea a través de un USB, un servidor Web o un gestor de flotas. Además, tiene soporte oficial en Yocto, Buildroot y Debian. También ofrece características de seguridad, como la verificación de firmas digitales.

2.3 Plataformas frente a la gestión de OTAs

A continuación se expone un conjunto de plataformas que hemos encontrado interesantes en relación con este proyecto. Si bien existen muchas más plataformas IoT en el mercado, hemos hecho esta selección procurando que tuviesen uno o más aspectos destacables en el contexto de este trabajo, tales como: que sean de uso general, que faciliten la gestión de flotas con alguna interfaz, que admitan variedad de dispositivos, que faciliten la comunicación a través de distintos protocolos, que sean de código abierto y que den cabida a las actualizaciones OTA —ya sea mediante alguna solución predefinida o personalizando su funcionamiento.

- **Mainflux** [23]

Plataforma IoT de código abierto que ofrece una suite de herramientas para la conectividad, gestión de dispositivos, y análisis de datos. Soporta una variedad de protocolos de comunicación y está diseñado para ser altamente modular, permitiendo a los desarrolladores personalizar y extender sus funcionalidades según sus necesidades.

Sin embargo, no cuenta con una interfaz de usuario para la gestión de flotas ni visualización de datos. Tampoco parece aportar una solución predefinida para gestionar actualizaciones OTA.

- **Kaa IoT** [24]

Plataforma IoT de código abierto que proporciona una infraestructura completa para la gestión y administración de dispositivos IoT. Ofrece recopilación y análisis de datos, y soporte para protocolos de comunicación como MQTT y CoAP. Destaca por su escalabilidad y capacidad para manejar grandes volúmenes de datos en tiempo real.

Cuenta con una interfaz de usuario donde se pueden gestionar dispositivos a varios niveles y cuenta con paneles para la visualización de datos. Aporta una solución para realizar OTAs basada en archivos de actualización que se almacenan en la plataforma y se distribuyen a los dispositivos indicados, con la alternativa de distribuir también un enlace de descarga en lugar de un archivo.

- **Thingsboard** [4]

Plataforma IoT de código abierto que se centra en la visualización y gestión de dispositivos IoT. Ofrece una interfaz de usuario intuitiva y poderosa que permite a los usuarios monitorizar y controlar dispositivos en tiempo real. Thingsboard soporta una variedad de protocolos de comunicación y proporciona herramientas avanzadas para el análisis de datos y la creación de dashboards personalizados.

Cuenta con una solución para realizar OTAs que, al igual que Kaa, permite

difundir archivos de actualización o enlaces de descarga para distribuir las nuevas versiones de firmware.

■ Particle [25]

Plataforma IoT que proporciona hardware y software para el desarrollo de dispositivos conectados. Ofrece una gama de módulos y placas de desarrollo a la vez que proporciona una plataforma en la nube que facilita la gestión de dispositivos, la recopilación de datos, y la integración con servicios de terceros.

Aunque mayoritariamente no es de código abierto, creemos que Particle merece una mención especial, ya que se vende como un producto que por defecto soluciona escenarios como los que planteamos en este trabajo. Asegura proveer de «una capa de abstracción en la comunicación que funciona de forma fiable sobre un amplio rango de radios: LTE, 5G, Wifi, satélite y LoRaWAN». Además, ofrece un sistema capaz de realizar actualizaciones OTA de software («Application OTA»), de firmware («Device OTA») y otros componentes («Asset OTA»). Incluso dice dar soporte a actualizaciones tanto en dispositivos Linux como en microcontroladores, para los cuales tiene su propio sistema operativo llamado Device OS. Véase cómo se presenta Particle en la figura 2.1.

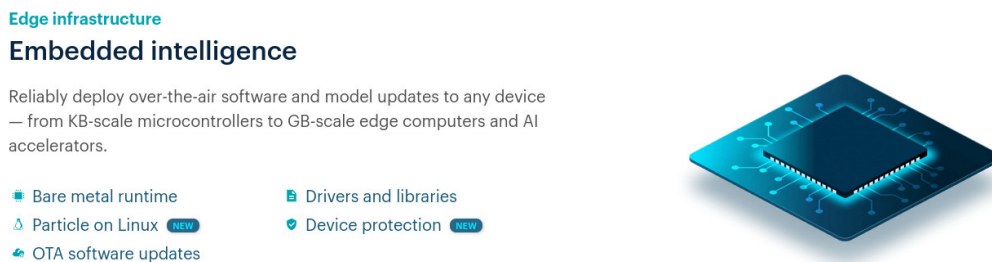


Figura 2.1: Captura hecha en la página web de Particle. Se presenta como una solución capaz de realizar OTAs en un rango de dispositivos muy amplio.

■ Balena [26]

Se trata de una tecnología basada en contenedores para la orquestación de flotas de dispositivos. Su infraestructura en la nube permite desarrollar, desplegar y gestionar flotas de dispositivos IoT con Linux, proporcionando actualizaciones OTA, una API y una SDK. No es de código abierto, aunque ofrece una prueba gratuita para usar hasta 10 dispositivos. Balena utiliza contenedores Docker para encapsular aplicaciones, lo que facilita su despliegue y escalabilidad.

■ Toit [27]

Solución de código abierto para gestionar dispositivos ESP32 con un enfoque de alto nivel basado, según dice su página web, en contenedores. Aunque no es estrictamente una plataforma, cuenta con una herramienta de gestión de flotas, y permite la realización de OTAs, permitiendo conectar y actualizar el dispositivo a través de Wifi, NB-IoT or LTE-M. Su paradigma de *live reloading* permite cargar una aplicación en el microcontrolador

a medida que el programador la escribe. Además, tiene su propio lenguaje orientado a objetos para programar las placas, que permite controlar su hardware. Como desventajas, sólo funciona en la ESP32.

2.4 Consideraciones sobre las OTAs a través de diferentes protocolos de red

Como las plataformas IoT son servidores que han de ser accesibles a través de Internet, están pensadas principalmente para la comunicación sobre el protocolo IP, de manera que, para comunicarse con los dispositivos, ofrecen APIs que funcionan con protocolos que se sitúan por encima de IP. En el caso de Thingsboard y Kaa —que, entre las plataformas de código abierto que hemos visto, son las que más parecen preocuparse por ofrecer distintas formas de conectividad— se exponen APIs que usan los protocolos MQTT, CoAP, HTTP, LwM2M y SNMP, todos los cuales funcionan, o bien sobre la pila TCP/IP, o bien sobre UDP/IP. Cuando un dispositivo está conectado a la red mediante tecnologías como Ethernet o Wi-fi, que funcionan sobre IP, esto no supone ningún problema. Sin embargo, no es posible usar los protocolos mencionados cuando entran en juego tecnologías de red inalámbrica como BLE, Zigbee o LoRa; al menos no directamente.

Es por eso que estas plataformas hacen uso de *gateways*¹¹ que intentan poner en contacto a la plataforma con el dispositivo que se comunica a través de esas tecnologías inalámbricas no IP. Sin embargo, que una determinada plataforma consiga establecer conexión con un dispositivo a través de un Gateway no implica necesariamente que las actualizaciones OTA vayan a funcionar del mismo modo que funcionan con los dispositivos que se comunican directamente con la plataforma. Esto se debe a que la comunicación a través de tecnologías inalámbricas no IP requiere una programación a más bajo nivel, para tratar con las particularidades de cada una, y una actualización OTA implica un desarrollo extra. Desarrollar una OTA en estos casos no es trivial, ya que requiere la transferencia fiable de un archivo que potencialmente puede ser bastante grande, caso que no es habitual en tecnologías como BLE, Zigbee o LoRa.

Con todo, es posible encontrar ejemplos de OTAs a través de varias tecnologías de red inalámbrica no IP, tal y como muestran estos proyectos:

- **LoraWAN OTA update** — tutorial de Pycom.
<https://docs.pycom.io/tutorials/networkprotocols/ota-lorawan/>
Este ejemplo demuestra cómo realizar una OTA a través de LoRaWAN sobre un dispositivo del ecosistema Pycom, con la ayuda de un Gateway LoRaWAN que hace de servidor.
- **nRF Device Firmware Update** — aplicación móvil.
<https://www.nordicsemi.com/Products/Development-tools/nRF-Device-Firmware-Update>
Esta es una aplicación móvil que, usando Bluetooth Low Energy como transporte, es capaz de actualizar el firmware de dispositivos que usen chips de la serie nRF51 o nRF52 del fabricante Nordic Semiconductor.

¹¹*gateway*: Dispositivo de red que actúa como un punto de acceso entre dos redes diferentes, permitiendo la comunicación y la transferencia de datos entre ellas.

- **OTAs a través de Zigbee2MQTT** — documentación del gateway Zigbee2MQTT.

https://www.zigbee2mqtt.io/guide/usage/ota_updates.html

En este artículo encontrado en la documentación del gateway llamado Zigbee2MQTT, cuya función es deducible por su nombre, se explica el método para realizar actualizaciones OTA a través de este gateway sobre uno de sus dispositivos compatibles.

Estos ejemplos demuestran que realizar OTAs a través de estas tecnologías es posible, aunque más difícil es ver estas OTAs integradas en una plataforma IoT de uso general.

CAPÍTULO 3

Propuesta

En este capítulo proponemos el desarrollo de un sistema que sirva para cumplir los objetivos que planteamos inicialmente, intentando así aportar un avance en la gestión de actualizaciones OTA.

Esencialmente la meta es conseguir un sistema IoT que, desde una interfaz proporcionada por la plataforma, permita una gestión de actualizaciones OTA lo más **interoperable** posible, ofreciendo compatibilidad con un rango amplio de dispositivos, y que soporte una red **híbrida**, donde los dispositivos se puedan conectar mediante varias tecnologías de red inalámbricas.

A la hora de hacer la propuesta se seleccionará justificadamente un conjunto de las tecnologías vistas en el capítulo anterior, teniendo en cuenta su viabilidad en el sistema que proponemos. Además, en el caso del software y el firmware a emplear, procuraremos que sean de código abierto, así como también elegiremos tecnologías de red que sigan estándares de comunicación abiertos.

3.1 Análisis de las limitaciones actuales

Tras haber analizado el estado del arte, podemos afirmar que el tema de las actualizaciones OTA se ha desarrollado con bastante profundidad tanto en el mundo del código abierto como en el del código propietario. Esto es especialmente cierto cuando hablamos de redes IP y dispositivos con Linux. En ese escenario se mueve la plataforma Balena que, con una elegante solución basada en contenedores, permite gestionar flotas de dispositivos IoT. Si preferimos usar código abierto, Mender, RAUC o SWUpdate nos proporcionan soluciones robustas para actualizar de forma remota nuestros dispositivos. Y si queremos contar con capacidades comunicación más propias de aplicaciones IoT —como es nuestro caso— podemos combinar alguna de las herramientas mencionadas con plataformas IoT de código abierto como Thingsboard.

Sin embargo, fuera del escenario típico basado en una red IP y dispositivos Linux, las soluciones no son tan directas. Por ejemplo, la cómoda solución basada contenedores Docker no es posible en microcontroladores, que no cuentan con los suficientes recursos. Por otra parte, respecto a la realización de OTAs a través de tecnologías inalámbricas no IP, si bien son teóricamente posibles y existen ejemplos que lo demuestran, es difícil verlas integradas en cualquier plataforma

que gestione OTAs, donde no se suelen tener en cuenta. Dicho sea esto con la salvedad de Particle que, según se deduce de su publicidad, es capaz de realizar OTAs sobre LoRaWAN, pero se trata de una solución de pago. En el caso de Thingsboard, aunque permite la comunicación con varias tecnologías de red inalámbricas a través de Gateways, las OTAs no están actualmente implementadas para esos escenarios, según se infiere de este hilo de Github¹.

3.2 Estudio de posibles soluciones

Un sistema IoT completo se compone, como mínimo, de una plataforma y uno o varios dispositivos. Por ende, es indispensable estudiar las posibilidades disponibles para implementar cada uno de estos dos aspectos.

3.2.1. Elección de una plataforma

La plataforma es el servicio que se encarga principalmente de procesar y almacenar los datos de los dispositivos, junto con otras muchas posibles características, tales como la visualización de datos, la monitorización o la gestión de flotas. Se nos presentan dos alternativas: programar nuestra propia plataforma desde cero, o aprovechar una plataforma ya existente que nos permita desarrollar sobre ella las soluciones que nos interesen. A este respecto nos decidimos por la segunda opción dados los siguientes motivos:

- a) El coste de programar una plataforma desde cero sería muy alto teniendo en cuenta que este trabajo pretende enfocarse en unas características en específico de la plataforma (las OTAs).
- b) Existen plataformas de código abierto que ya cuentan con una estructura base desarrollada (bases de datos, visualización de datos, protocolos de comunicación implementados, etc).
- c) Algunas plataformas de código abierto facilitan su extensión por diseño, como es el caso de Thingsboard o Mainflux.

Entre las plataformas que analizamos en el capítulo anterior, descartaremos Balena porque su modelo basado en contenedores no nos permitiría gestionar dispositivos basados en microcontrolador. Descartaremos Particle por no ser de código abierto y también Kaa, cuya versión de código abierto fue descontinuada². Entre las restantes, la mejor opción parece ser **Thingsboard**, cuya versión Community (la de código abierto), parece muy completa, y da soporte a distintas tecnologías de comunicación a través de sus Gateways. Además, tiene potentes herramientas para la visualización de datos, y un método predefinido para realizar actualizaciones OTA.

¹<https://github.com/thingsboard/thingsboard-gateway/issues/953>

²En 2016, «Kaa open source» terminó su desarrollo con la entrada de «Kaa Enterprise»: <https://www.kaaiot.com/kaa-open-source>

3.2.2. Elección de una tecnología para programar dispositivos

Como se ha mencionado anteriormente, dividiremos los dispositivos en dos familias y aportaremos una solución particular en cada una de ellas.

Dispositivo basado en microcontrolador

Entre las tecnologías vistas para programar estos dispositivos, las cuatro cuentan con un gran número de dispositivos soportados, y cualquiera de ellas podría ser válida. Sin embargo, nos decidiremos por **Micropython**, que si bien puede no ser la mejor opción en términos de rendimiento, nos proporciona estas ventajas:

- a) Teniendo experiencia con Python, la programación resulta comparativamente sencilla. Arduino se programa usando un estilo simplificado de C/C++, y tanto Zephyr como FreeRTOS funcionan con C, lo cual puede exigir un desarrollo algo más concienzudo.
- b) Tiene un sistema de paquetes llamado *mip* que nos permitirá reutilizar y distribuir nuestro código de forma muy directa.
- c) Aprovechando su diseño, podemos implementar un proceso de actualización SOTA, tal y como explicamos en el capítulo anterior. De esta manera, un mismo paquete de actualización podrá ser válido en cualquier dispositivo con el firmware de Micropython, ya que gracias a esta capa de firmware podemos abstraernos completamente del hardware en la capa de aplicación. Tal cosa no es posible en sistemas como Zephyr, donde las aplicaciones han de compilarse para cada dispositivo.

Dispositivo basado en microprocesador con Linux

3

Entre las soluciones basadas en Linux vistas en el capítulo anterior, consideramos que una buena opción sería Buildroot. Buildroot es, aparentemente, la herramienta más sencilla para crear imágenes Linux. En comparación a usar una distribución ya compilada, como Ubuntu Core o Fedora IoT, una imagen personalizada permite reducir al mínimo los componentes instalados, para así ahorrar recursos. De esta forma, podríamos instalar en el dispositivo solo lo necesario para ejecutar su aplicación y comunicarse con Thingsboard en paralelo. En un ecosistema como el de Linux, no debería suponer demasiados problemas encontrar bibliotecas y SDKs que nos ayuden con ese desarrollo.

En lo que se refiere a la implementación de la OTA, sugerimos que se utilice RAUC, que cuenta con soporte en Buildroot. RAUC permite la generación de paquetes de actualización de software, los cuales se podrían enviar al dispositivo desde la plataforma y, una vez recibidos, se podrían instalar gracias a las utilidades de RAUC integradas en el dispositivo.

³Por limitaciones en el tiempo desarrollo, los dispositivos basados en microprocesador con Linux no serán implementados en este trabajo, por lo que no se hace referencia a ellos en la propuesta final.

3.3 Propuesta final

En consonancia con los objetivos planteados para este trabajo, se propone el desarrollo de un sistema IoT, con **Thingsboard** como plataforma, que permita a los dispositivos conectarse y actualizarse a través de las siguientes tecnologías de red inalámbricas, que se han seleccionado intentando representar una buena parte de los casos de uso habituales en IoT:

- **Wifi**

Tecnología de red inalámbrica que opera en la banda de 2.4 GHz y 5 GHz, usada en multitud de contextos para proporcionar acceso a internet sin cable. En el campo del IoT, se usa en ámbitos como la domótica u oficinas inteligentes. Al ser un protocolo que funciona sobre IP, será previsiblemente el caso más sencillo de implementar.

- **BLE**

Evolución del estándar Bluetooth clásico, es una tecnología que opera en la banda de 2.4 GHz y está optimizada para aplicaciones que necesitan transferir pequeñas cantidades de datos con baja latencia, corto alcance y un consumo de energía mínimo. En el campo del IoT, se utiliza en *wearables*⁴, localización basada en proximidad, o control remoto de electrodomésticos.

- **LoRa**

Tecnología de comunicación de larga distancia y bajo consumo de energía que opera en la banda de frecuencia sub-GHz. Es ideal para aplicaciones que requieren una cobertura extensa y una vida útil de batería prolongada. Se emplea en contextos como ciudades inteligentes, agricultura inteligente o industria.

En aras de trabajar con una infraestructura más simple, decidimos usar LoRa en lugar de LoRaWAN, que es un protocolo que ofrece gestión de red y seguridad sobre la modulación LoRa.

La integración de la **actualización OTA en Thingsboard mediante BLE y LoRa** se trata de un aspecto novedoso que aporta nuestra propuesta, pues actualmente la plataforma no cuenta con ningún soporte para tal fin, y no se ha encontrado en Internet ninguna solución que aborde esta funcionalidad.

Para los dispositivos con microcontrolador que se conecten a la plataforma, se desarrollará un proyecto de código en **Micropython**, que ofrecerá una biblioteca para la comunicación y la actualización remota de software (**SOTA**) a través de las tres tecnologías de red inalámbrica elegidas. Como hardware para poner a prueba la solución, se emplearán placas de desarrollo modelo **ESP32**, conocidas por su gran potencia y bajo coste.

En lo concerniente a la plataforma Thingsboard, esta deberá ser configurada o incluso extendida adecuadamente para soportar la gestión de dispositivos que puedan conectarse por medio de Wifi, BLE y Lora. Se aprovecharán los Gateways

⁴*wearables*: Dispositivos portátiles como relojes inteligentes, pulseras de actividad, y monitores de salud.

de Thingsboard y se desarrollarán otros **intermediarios de comunicación** necesarios.

Los dispositivos no solo podrán actualizarse a través de Thingsboard sino que conservarán las funcionalidades de comunicación que tiene Thingsboard con sus dispositivos. Concretamente, y como ejemplo de demostración, se elaborará un panel visual o **dashboard** donde se puedan comprobar estas funcionalidades:

- Actualización OTA.
- Envío de telemetría desde el dispositivo hacia la plataforma.
- Comandos RPC (*Remote Procedure Call*) desde la plataforma al dispositivo.

Por último, el sistema se desarrollará mediante la creación de **componentes reutilizables** que podrán ser empleados por otros desarrolladores en sus propios proyectos.

CAPÍTULO 4

Diseño de la solución

En este capítulo se expone el diseño que se ha elaborado para abordar la propuesta. En primer lugar, se ilustrará la arquitectura del sistema, para comprender la interacción entre los distintos componentes y la función que desempeña cada uno. En segundo lugar, se presentarán las tecnologías que se han empleado para hacer frente a la implementación de todos esos componentes. Después, se realizará una exposición más detallada, en la que primeramente se analizará el diseño de las comunicaciones entre Thingsboard y los dispositivos, incluyendo los intermediarios que la hacen posible, y pasando por todos los escenarios de conectividad (Wifi, BLE y LoRa). Por último, se mostrará en detalle el diseño del software en Micropython que se ha escrito para los dispositivos que formen parte de este sistema.

4.1 Arquitectura del Sistema

La figura 4.1 muestra la arquitectura inicial del sistema planteado, que se explica a continuación.

La arquitectura de este sistema tiene como punto de partida y eje central el **servidor Thingsboard**, que es una plataforma IoT en sí mismo. Realiza las funcionalidades que se esperarían de una plataforma IoT, como recolección de datos, visualización de dichos datos a través de una interfaz de usuario, análisis de telemetría con procesamiento de eventos, lanzamiento de procedimientos remotos (RPC), entre otras funcionalidades.

El servidor Thingsboard por sí mismo es capaz de comunicarse con los dispositivos directamente a través una API que expone para que los dispositivos interesados en conectarse la usen. Existe una versión de esta API para cada protocolo de transporte soportado, concretamente: MQTT, MQTT Sparkplug, CoAP, HTTP, LwM2M y SNMP.

En nuestro sistema, el dispositivo que se conecte a través de **Wifi** lo hará usando la API MQTT del servidor Thingsboard. Entre todas las APIs, elegimos MQTT por su sencillez, su eficiencia y el hecho de que es el protocolo más usado en IoT. En este escenario de conectividad, la infraestructura que necesita el dispositivo es un punto de acceso Wifi capaz de conectarle con el servidor, cosa que se dará en el caso de que dispositivo y servidor compartan la misma red local, o en el

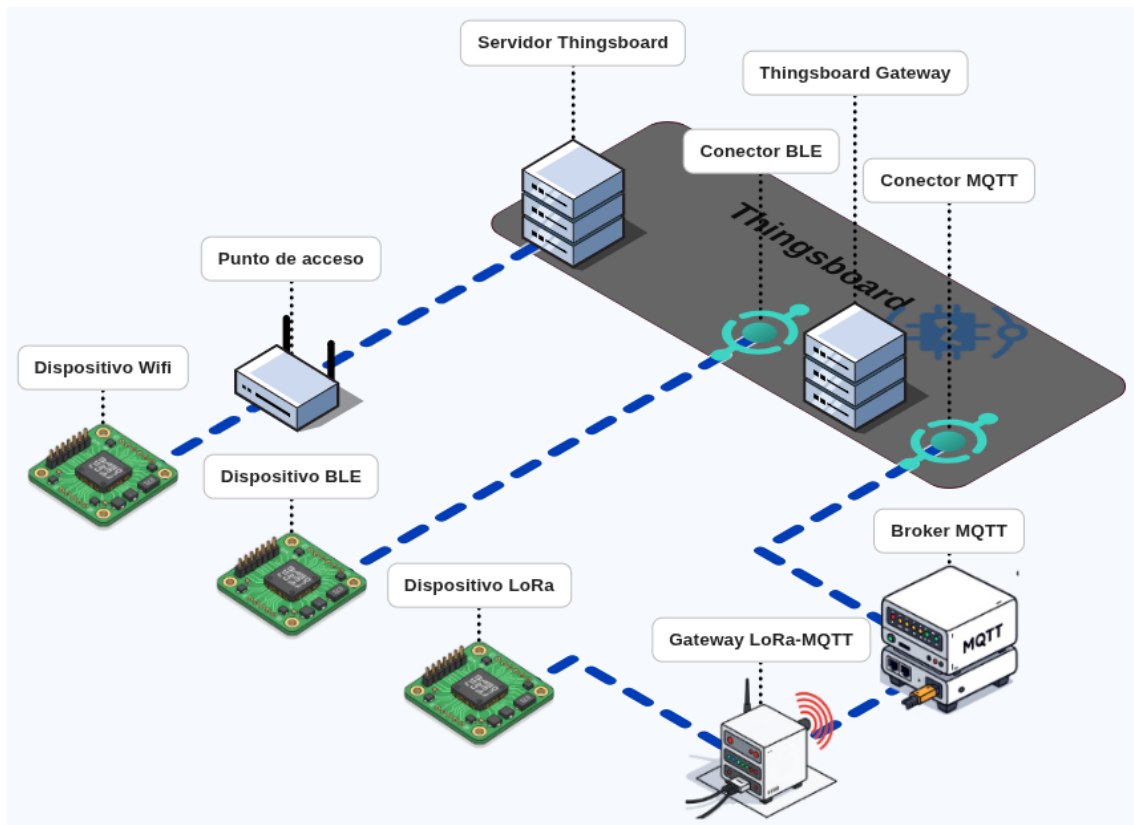


Figura 4.1: Primera aproximación a la arquitectura del sistema, dirigida a establecer comunicación con Thingsboard desde tres dispositivos conectados respectivamente mediante Wifi, BLE y LoRa.

caso de que el punto de acceso dé salida a Internet y el servidor esté expuesto en Internet.

El **Thingsboard Gateway** es un componente que también forma parte del ecosistema de Thingsboard Community y cumple la función de integrar dispositivos conectados a sistemas heredados y de terceros. Se conecta al servidor Thingsboard a través de MQTT, aunque esa comunicación le es transparente al administrador del sistema. La arquitectura propia de este componente incluye los llamados conectores, que son los encargados de establecer la conexión con sistemas externos o dispositivos a través de diferentes protocolos. El Thingsboard Gateway dispone de un conector Modbus, un conector HTTP, un conector FTP, un conector CAN... y así hasta 16 conectores distintos, sumados a la posibilidad de crear uno personalizado.

El dispositivo que se conecte a través de **BLE** no lo hará directamente con el servidor Thingsboard, sino en conexión con el **conector BLE** del Thingsboard Gateway, que configuraremos adecuadamente para que pueda establecer comunicación con el dispositivo deseado.

El dispositivo que se conecte por **LoRa**, en cambio, no puede conectarse a tal cosa como un conector LoRa, ni un conector LoRaWAN, pues dichos conectores no existen a la fecha en que se redacta esta memoria. Sin embargo, ideamos una solución en la que la comunicación se produce indirectamente a través de un

conector MQTT. Este conector se conecta a un **broker MQTT** que hará de enlace entre la plataforma y el dispositivo de la siguiente forma.

El dispositivo LoRa se comunica con un **gateway LoRa-MQTT**, el cual se comunica a su vez con el broker. Este gateway LoRa-MQTT hace traducciones entre paquetes LoRa y mensajes MQTT que recibe, reenviando después esa traducción, actuando de pasarela entre dispositivo y el broker.

Lo expuesto hasta aquí comprende la arquitectura planteada inicialmente. Sin embargo, la información que aportaron las fases de desarrollo nos obligó a añadir nuevos componentes, como se aprecia en la figura 4.2.

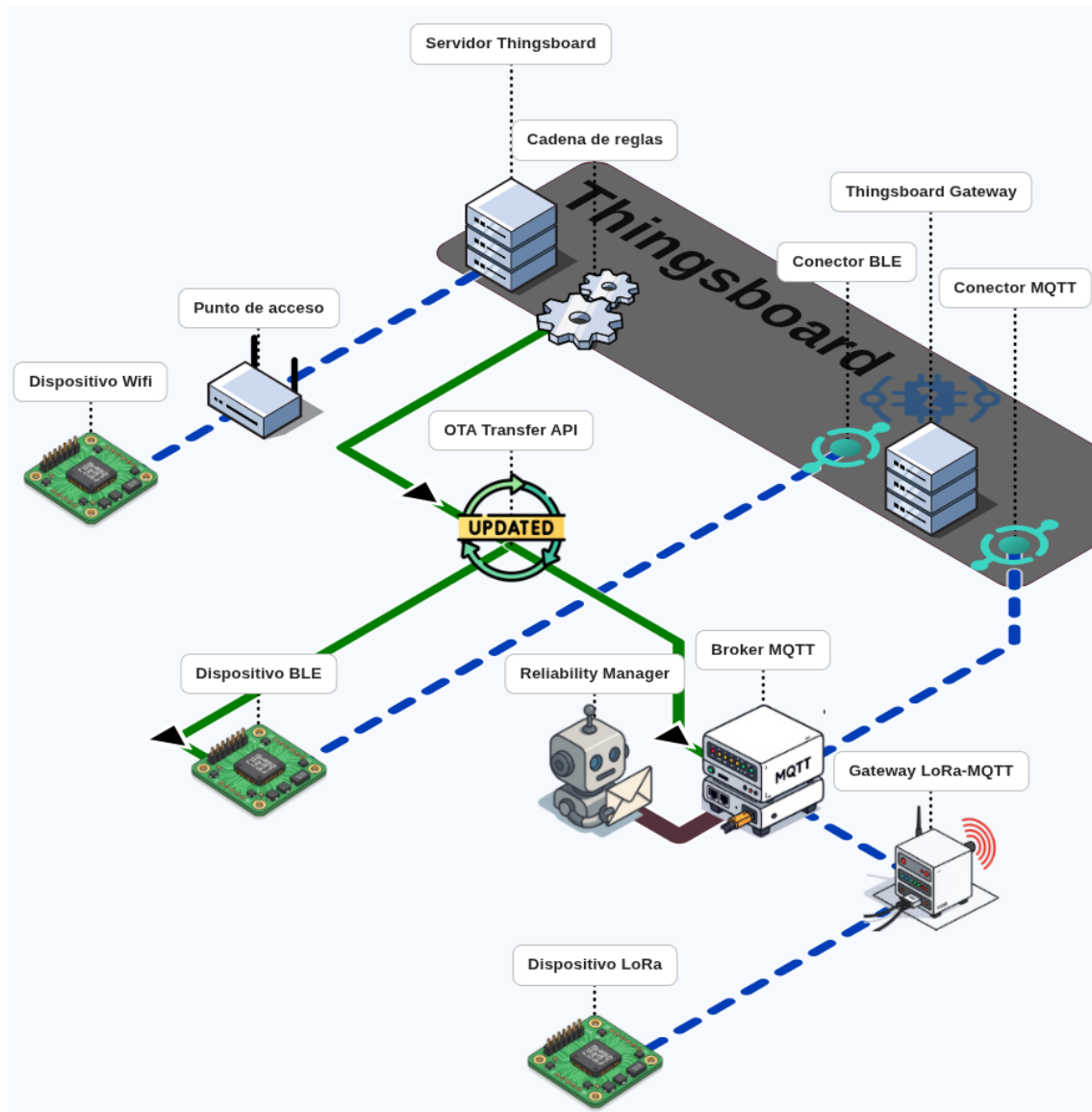


Figura 4.2: Arquitectura final del sistema, con nuevos componentes añadidos para asegurar tanto el intercambio de datos como el de paquetes de actualización.

A la hora de implementar la comunicación relativa a las actualizaciones OTA en BLE y LoRa, descubrimos que las actualizaciones OTA en Thingsboard no cuentan con soporte en el Thingsboard Gateway. Es decir, mediante los conectores pueden pasar datos de telemetría, atributos compartidos o llamadas RPC,

pero no paquetes de actualización. Por este motivo, tuvimos que introducir un nuevo componente, el OTA Transfer Service, que sirve una API HTTP llamada **OTA Transfer API**, con métodos que permiten transferir el paquete OTA correspondiente a los dispositivos conectados por BLE o LoRa. En el caso de BLE, la transferencia es directa, mientras que en el caso de LoRa, se hace a través del broker MQTT.

La integración del OTA Transfer Service con la plataforma se hace a través de la **cadena de reglas** que tiene el servidor Thingsboard. La cadena de reglas es un mecanismo personalizable de procesamiento de datos central que puede recibir, transformar, enrutar y reaccionar a telemetría y eventos que llegan a la plataforma. Nuestra cadena de reglas personalizada detecta cuándo se ha solicitado una actualización OTA y hace la debida llamada a la OTA Transfer API.

Por último, introducimos otro nuevo componente al que llamamos **Reliability Manager**, encargado de asegurar una transmisión fiable entre el dispositivo LoRa y el broker MQTT, puesto que LoRa no es un protocolo de red con garantías de entrega. Este componente es un servicio que lee y escribe en determinadas colas del broker MQTT y, en coordinación con el dispositivo LoRa, gestiona un procedimiento basado en ACKs para intercambiar mensajes de forma fiable.

4.2 Tecnologías utilizadas

A continuación se muestran las principales tecnologías que se han elegido para abordar la implementación de los componentes del sistema.

- **Docker y Docker Compose** [29]

Docker automatiza el despliegue de aplicaciones dentro de contenedores de software, proporcionando una capa adicional de abstracción con un paradigma de virtualización ligera. Docker Compose, por su parte, permite definir y ejecutar aplicaciones multi-contenedor.

Ejecutaremos los componentes de Thingsboard usando sus versiones en forma de imágenes Docker, y también desarrollaremos imágenes para nuestros componentes propios, unificando toda la solución con Docker Compose.

- **JSON (*JavaScript Object Notation*)** [30]

Es un popular formato de intercambio de datos, ligero y basado en un subconjunto del lenguaje de programación JavaScript. Leerlo y escribirlo es simple para humanos, mientras que para las máquinas es simple interpretarlo y generarlo.

Thingsboard usa JSON (aunque también admite Protobuf) en el cuerpo de los mensajes que intercambia con los dispositivos a través de su API MQTT, así que deberemos usar bibliotecas que nos permitan interpretarlo. Como decisión de diseño también usaremos JSON en la comunicación por LoRa.

- **Hardware:**

- **Placa de desarrollo ESP32 WROOM** [31]

Placa con microcontrolador de la serie ESP32, desarrollada por Espressif Systems, que cuenta con capacidades Wifi y BLE. Tiene un procesa-

por Xtensa Dual Core LX6 de 32-bit con frecuencia de 160 o 240MHz, 520KB de SRAM y varios periféricos.

La usaremos para implementar los dispositivos conectados por Wifi y BLE.

- **Placa de desarrollo ESP32 TTGO LoRa32** [32]

Placa de desarrollo para LoRa basada en la ESP32-PICO-D4 (procesador Dual core a 240MHz y 520KB de SRAM). La versión concreta que usaremos (T3_V1.6.1) equipa el transceptor de modulación LoRa SX1276. También cuenta con Wifi y BLE.

Utilizaremos dos placas idénticas de este modelo: una para implementar el dispositivo conectado por LoRa, y otra que desempeñará la función de Gateway LoRa-MQTT.

- Intermediarios de comunicación:

- **LoRa Gateway del proyecto Theengs OpenMQTTGateway** [33]

Por un lado, recibe señales de LoRa en bruto, las procesa y publica los datos a un tema MQTT. Por otro lado, puede suscribirse a temas de MQTT y recibir comandos para enviar mensajes a los dispositivos LoRa.

Instalaremos el firmware que realiza estas funciones en la TTGO LoRa32 como implementación del Gateway LoRa-MQTT.

- **Eclipse Mosquitto** [34]

Broker de mensajería MQTT ligero y de código abierto, idóneo para el IoT.

Su versión dockerizada nos servirá de implementación para el broker MQTT necesario en el escenario de conectividad con LoRa.

- Características de Thingsboard:

- **Rule Chain**¹

Mecanismo personalizable de procesamiento de datos central que puede recibir, transformar, enrutar y reaccionar a telemetría y eventos que llegan a la plataforma.

Personalizaremos este mecanismo para disparar, en el momento oportuno, la transferencia del paquete de actualización al dispositivo.

- **Dashboards**²

Los *dashboards* (paneles) son visualizaciones interactivas que sirven para monitorizar y manejar eficientemente datos y dispositivos.

Nos servirán para crear la interfaz gráfica que permitirá gestionar y actualizar los dispositivos desde un mismo punto, independientemente de la tecnología inalámbrica con la que estén conectados.

- Utilidades:

- **esptool** [35]

Herramienta de línea de comandos de código abierto, basada en Python, para flashear³, aprovisionar e interactuar con dispositivos de Espressif

¹Véase <https://thingsboard.io/docs/user-guide/rule-engine-2-0/overview/>

²Véase <https://thingsboard.io/docs/user-guide/dashboards/>

³flashear: (del inglés *flash*) Instalar o reinstalar el firmware de un dispositivo.

por medio de una conexión serial.

Se empleará para instalar el firmware de Micropython en los dispositivos que programaremos, y también para instalar el firmware de Theengs OpenMQTTGateway en la TTGO LoRa32.

- **mpremote** ⁴

Herramienta de línea de comandos para interactuar de forma remota, administrar el sistema de archivos y automatizar un dispositivo Micropython a través de una conexión serial.

Se empleará para instalar las bibliotecas y aplicaciones Micropython en las placas de desarrollo, ver la salida de los programas que se ejecuten en ellas, y hacer depuración con el entorno REPL (*Read-Eval-Print Loop*) que proporciona.

- **mip** («*mip installs packages*») ⁵

Es a la vez un módulo de Micropython y una funcionalidad de la herramienta *mpremote*, que permite instalar paquetes de distintos orígenes, como la *micropython-lib*, repositorios alojados en sitios como Github, o proyectos locales.

Lo emplearemos para instalar dependencias de software en las placas, y para crear nuestros propios paquetes, que nos permitirán distribuir nuestro software hecho en Micropython.

- **MQTT Explorer** [36]

Aplicación de escritorio que actúa como cliente MQTT para proporcionar una vista estructurada de los temas de un servidor MQTT. Sirve, entre otras cosas, para visualizar y publicar mensajes usando una interfaz gráfica.

La usaremos para monitorizar el estado del broker MQTT de nuestra arquitectura y así depurar la comunicación en el escenario del dispositivo conectado por LoRa.

- **HTTPIe** [37]

Aplicación de escritorio que actúa como cliente HTTP para testear APIs con una interfaz gráfica.

La usaremos para depurar a la API HTTP del componente OTA Transfer API.

- Bibliotecas de Micropython para desarrollar el software de los dispositivos:

- **thingsboard-micropython-client-sdk** [38]

Proporciona un conveniente SDK de cliente para que el dispositivo interactúe con la API MQTT del servidor Thingsboard.

La funcionalidad para el escenario de conectividad Wifi se implementará haciendo uso de esta biblioteca.

- **aioble** [39]

Envoltorio del módulo bluetooth de bajo nivel, orientado a objetos y basado en «asyncio».

La funcionalidad para el escenario de conectividad BLE se implementará haciendo uso de esta biblioteca.

⁴Véase <https://docs.micropython.org/en/latest/reference/mpremote.html>

⁵Véase <https://docs.micropython.org/en/latest/reference/packages.html>

- **LoRa driver** [40]

Proporciona controladores inalámbricos sincrónicos y asincrónicos para dispositivos modernos Lora de Semtech.

La funcionalidad para el escenario de conectividad LoRa se implementará haciendo uso de esta biblioteca.

- Bibliotecas de Python:

- **fastapi** [41] y **uvicorn** [42]

Uvicorn es un servidor web y *fastapi* es un framework web para construir APIs.

Con la combinación de estas dos bibliotecas implementaremos la API que expone el componente OTA Transfer Service.

- **bleak** (*Bluetooth Low Energy platform Agnostic Klient*) [43]

Proporciona un cliente GATT capaz de conectarse a dispositivos que actúen como servidores GATT.

Con esta biblioteca implementaremos la transferencia por BLE del paquete de actualización, dentro del OTA Transfer Service.

- **paho-mqtt** [44]

Proporciona un cliente capaz de conectarse a un broker MQTT para publicar mensajes o suscribirse a topics y recibir mensajes publicados. Con esta biblioteca implementaremos, dentro del OTA Transfer Service, una transferencia por MQTT del paquete de actualización, que indirectamente llegará al dispositivo conectado por LoRa. Además, también implementaremos con esta biblioteca el componente Reliability Manager.

4.3 Diseño detallado

En esta sección analizaremos más en detalle el diseño del sistema, explicando los principales aspectos bajo dos categorías: el diseño de las comunicaciones y el diseño del software de los dispositivos.

4.3.1. Diseño de las comunicaciones

Para comprender el diseño de las comunicaciones, primeramente hemos de conocer cómo se produce el intercambio de datos entre Thingsboard y los dispositivos. Tres aspectos importantes al respecto son los atributos, la telemetría y las llamadas RPC. Los **atributos**⁶ son pares clave-valor que pueden compartirse entre la plataforma y el dispositivo. La **telemetría**⁷ son series temporales de valores relativos a una clave, que se envían desde el dispositivo a la plataforma. Las llamadas **RPC** (*Remote Procedure Call*)⁸ son funciones que se pueden invocar remotamente, desde la plataforma al dispositivo y viceversa. El intercambio de

⁶Véase <https://thingsboard.io/docs/user-guide/attributes/>

⁷Véase <https://thingsboard.io/docs/user-guide/telemetry/>

⁸Véase <https://thingsboard.io/docs/user-guide/rpc/>

datos necesario relativo a estos aspectos se hará de una forma u otra dependiendo del protocolo de comunicación que se use.

El proceso en detalle de la OTA se puede consultar en la documentación de Thingsboard⁹, pero brevemente lo describiremos aquí. Una actualización OTA se activa desde la plataforma y, en primer lugar, supone la actualización de un conjunto de atributos compartidos por parte de la plataforma: «fw_title», «fw_version», «fw_checksum» y «fw_checksum_algorithm». El dispositivo lee los valores de estos atributos y decide si, a juzgar por el título y la versión del nuevo firmware (o software) es realmente necesaria la actualización, conociendo lo que actualmente ya tiene instalado. Si lo es, puede solicitar la transferencia del paquete de OTA con un procedimiento que depende de cada protocolo de comunicación empleado. Después de recibir dicho paquete, el dispositivo puede comprobar su integridad con los valores de los atributos «fw_checksum» y «fw_checksum_algorithm» que había leído antes. Además, el dispositivo puede reportar el estado de la actualización en curso enviando telemetría con las claves «current_fw_title», «current_fw_version», «fw_state» y «fw_error».

Conexión por Wifi

Esta es la forma de conectividad más simple, como se puede apreciar en la figura 4.3. La primera condición para que puedan comunicarse plataforma y dispositivo es que la IP y puerto del servidor Thingsboard sean accesibles desde el dispositivo Wifi. Para permitir un despliegue simple con esta condición, tanto el servidor Thingsboard como el dispositivo se conectarán a la misma red Wifi (siendo una red Wifi doméstica perfectamente válida al respecto).

En cuanto a la comunicación a más alto nivel, el dispositivo actuará como un cliente MQTT de la API MQTT¹⁰ expuesta por la plataforma. La autenticación se producirá gracias a un *access token* que se obtiene al crear una entidad de dispositivo en Thingsboard. El dispositivo tendrá que conocer este *access token* para usarlo como usuario al iniciar la conexión MQTT. El intercambio de datos con la plataforma —en especial de telemetría, atributos, llamadas RPC y actualización OTA— se hará a través de mensajes en formato JSON, usando las colas apropiadas para cada caso indicado en la documentación de la API.

Conexión por BLE

La comunicación por BLE se realiza teniendo en mente las especificaciones de esta tecnología, concretamente: GAP (*Generic Access Profile*), que define cómo se establecen, mantienen y terminan las conexiones entre dispositivos; y GATT (*Generic Attribute Profile*), que define cómo se estructuran y transmiten los datos dentro de una conexión. Desde la perspectiva de GAP, el dispositivo sigue el rol de *Peripheral*, mientras que la plataforma sigue el rol de *Central*. Desde la perspectiva de GATT, el dispositivo es el servidor y la plataforma es el cliente.

De acuerdo a esta configuración, siguiendo la arquitectura de BLE, el dispositivo expondrá unas **características**, agrupadas en servicios, que serán descubrier-

⁹Véase <https://thingsboard.io/docs/user-guide/ota-updates/>

¹⁰Véase <https://thingsboard.io/docs/reference/mqtt-api/>

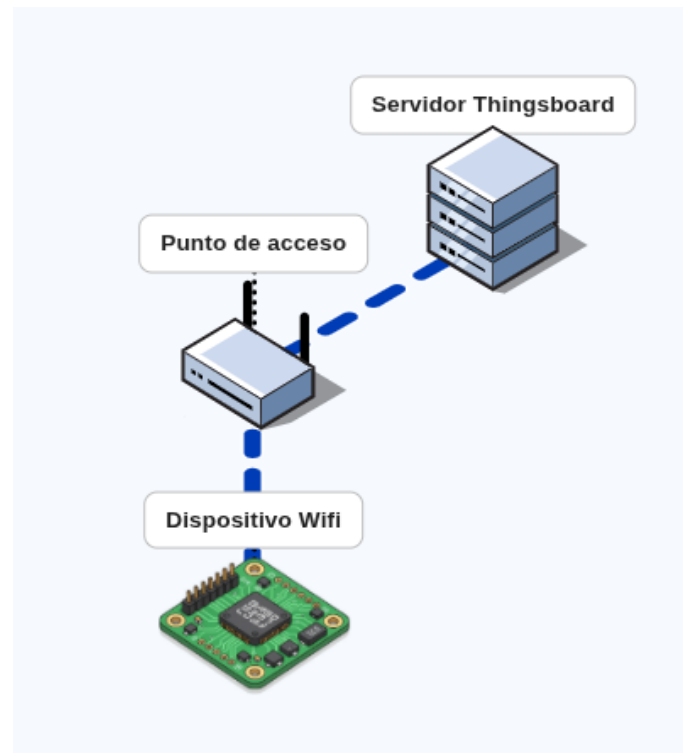


Figura 4.3: Diseño de la comunicación por Wifi, sobre la cual fluirán tanto paquetes de actualización como otros datos de plataforma a través de MQTT.

tos por la plataforma cuando se produzca la conexión. Cada característica representa una unidad de datos y, una vez conectados, ambos interlocutores pueden leer y escribir sobre dichas características.

La conexión por BLE, como se muestra en la figura 4.4, se realiza a través del conector BLE¹¹ del Thingsboard Gateway. Dicho conector actúa como servidor GATT, y se ha de configurar adecuadamente para que relacione correctamente las características BLE del dispositivo con sus respectivos datos, atributos, telemetría, y llamadas RPC en el lado de Thingsboard. Además, el conector identificará al dispositivo de interés por su dirección MAC.

El conector tiene unos componentes llamados *Downlink Converter* y *Uplink Converter*, que convierten, respectivamente, los datos de Thingsboard en cadenas de bytes para ser transportadas, y las cadenas de bytes provenientes de los dispositivos en datos para Thingsboard. A causa de que el Uplink Converter por defecto no satisfacía nuestras necesidades, decidimos desarrollar el *UTF-8 Uplink Converter* para sustituirlo.

El conector, a pesar de todo, no cuenta con una funcionalidad para transferir el paquete de actualización.

OTA Transfer Service

El servicio OTA Transfer nace para cubrir la funcionalidad, ausente en el Thingsboard Gateway, de transferir el paquete de actualización cuando se activa una

¹¹Véase <https://thingsboard.io/docs/iot-gateway/config/ble/>

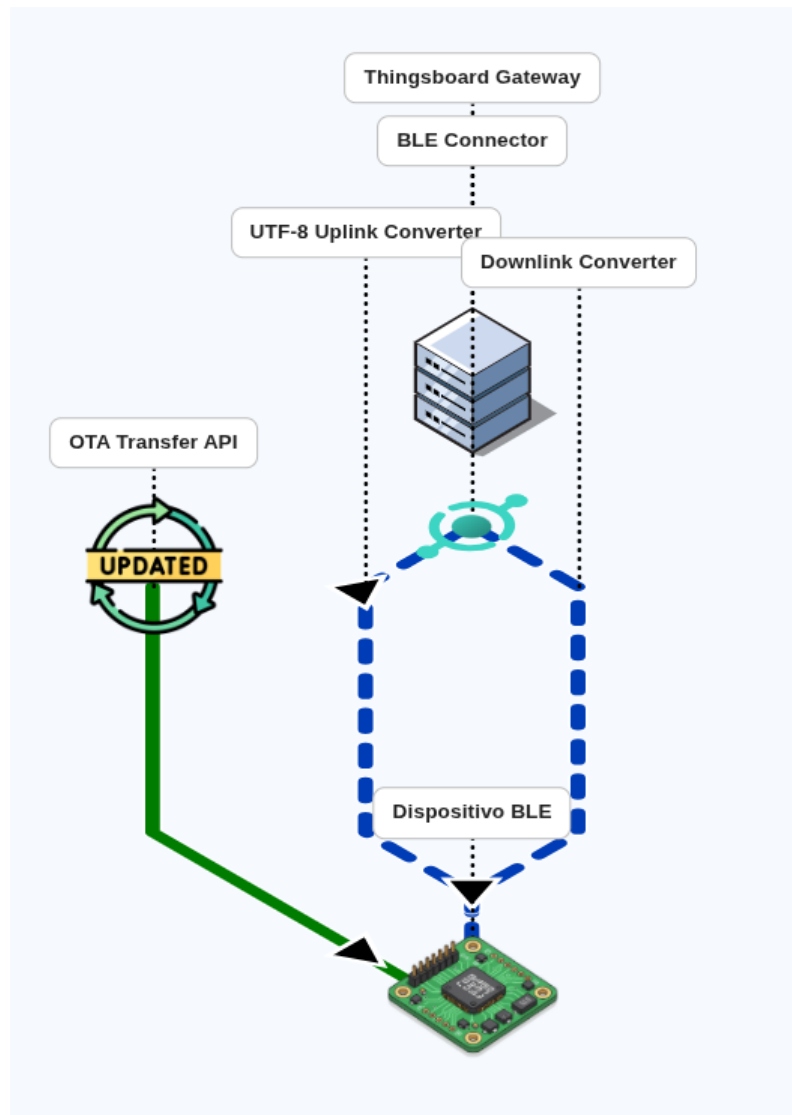


Figura 4.4: Diseño de la comunicación por BLE, donde se muestra cómo fluirán los datos de plataforma directamente entre el conector y el dispositivo, mientras la transferencia del paquete de actualización se realizará mediante el servicio *OTA Transfer*.

OTA. Servirá, por ende, para transmitir el paquete de actualización tanto al dispositivo conectado por BLE como al conectado por LoRa.

Como muestra la figura 4.5, el servicio OTA Transfer está fuertemente relacionado con la cadena de reglas —o *Rule Chain*— del servidor Thingsboard. Personalizaremos esta cadena de reglas para detectar el momento en que se activa una actualización OTA y solicitar la transferencia correspondiente al servicio OTA Transfer, usando su API. Detectaremos ese momento monitorizando la telemetría que llega al servidor, porque dicha telemetría sí llega a través del Gateway. En el momento en que la variable «fw_state» de un dispositivo pase a tener el valor DOWNLOADING, significará que el dispositivo en cuestión ha comprobado que necesita una actualización y espera descargarla. Si dicho dispositivo está conectado por BLE o LoRa, cosa que comprobaremos leyendo su atributo «ota_connectivity» (definido por nosotros), haremos la llamada adecuada a la API.

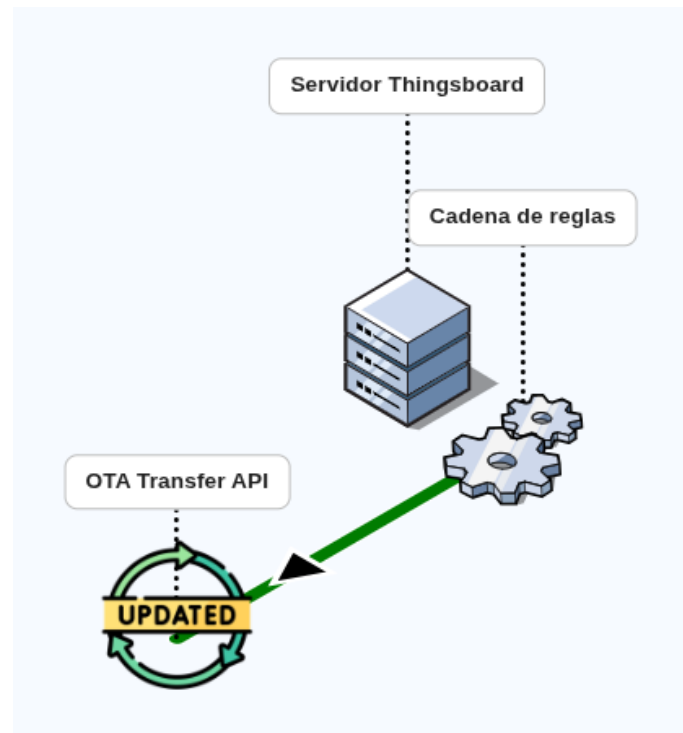


Figura 4.5: Diseño del servicio *OTA Transfer*, que realiza una transferencia del paquete de actualización a demanda, sin pasar por los conectores de Thingsboard, válido tanto en el caso de BLE como en el de LoRa.

El servicio OTA Transfer expone una API HTTP para ofrecer la funcionalidad que hemos explicado. Concretamente, proporciona estos *endpoints*:

- **POST /trigger_ble_ota_transfer**

Dispara la transferencia de un paquete de actualización hacia un dispositivo conectado por BLE. Se esperan los parámetros de consulta «device_name», «fw_title», «fw_version» y «access_token».

Ejemplo:

```
http://127.0.0.1:5000/trigger_ble_ota_transfer?
device_name=ESP32-Micropython-BLE-device&
fw_title=micropython-BLE-Peripheral&
fw_version=v1&
access_token=ikZwFlLeQxFlerRQd6SJ
```

- **POST /trigger_lora_ota_transfer**

Dispara la transferencia de un paquete de actualización hacia un dispositivo conectado por LoRa. Se esperan los parámetros de consulta «device_name», «fw_title», «fw_version», «access_token» y «lora_id».

Ejemplo:

```
http://127.0.0.1:5000/trigger_lora_ota_transfer?
device_name=ESP32-Micropython-LoRa-device&
fw_title=micropython-LoRa-Node&
fw_version=v1&
access_token=KIqU4a0FjcrhR1GTJbyq&
lora_id=f024f9b18b08
```

En su funcionamiento interno, el servicio OTA Transfer, para poder recuperar el paquete de actualización, hará a su vez una llamada a una API del servidor Thingsboard no mencionada hasta ahora: su API REST de administración¹². Esta API proporciona, para los clientes autenticados correctamente, una multitud de métodos para leer y modificar datos de la plataforma eludiendo la interfaz de usuario. Se solicitará a dicha API el paquete de actualización determinado por los parámetros de consulta «fw_title» y «fw_version». Una vez recuperado el paquete, el servicio OTA Transfer lo transferirá al dispositivo BLE o al dispositivo LoRa.

Conexión por LoRa

La comunicación que se da puramente por LoRa, prescindiendo de un protocolo a más alto nivel como LoRaWAN, tiene lugar en la capa física. Los dispositivos reciben y transmiten señales de LoRa en bruto, sin un protocolo de red o un manejo de datos propio de capas superiores. Eligiendo LoRa frente a LoRaWAN optamos por una infraestructura más ligera, sobre todo al evitar el gateway requerido en el caso de LoRaWAN, que necesita antenas avanzadas y un hardware potente. Como contrapartida, se sacrifica la fiabilidad y la manera definida en LoRaWAN para transportar mensajes entre una red IP y una red LoRa.

En la figura 4.6, la única línea de comunicación que realmente simboliza un flujo de señales LoRa es la que se observa entre el «Dispositivo LoRa» y el «Gateway LoRa-MQTT».

De un modo que explicaremos más adelante, en el broker MQTT aparecerá un mensaje equivalente a cada paquete LoRa enviado o recibido por el dispositivo. Partiendo de esa base, el conector MQTT del Thingsboard Gateway será configurado para suscribirse al broker MQTT e interpretar sus mensajes como atributos, telemetría o llamadas RPC operables en Thingsboard.

Gateway LoRa-MQTT

Este gateway, que en la práctica será una placa TTGO LoRa32 con el firmware del proyecto Theengs OpenMQTT Gateway, actúa como puerta de enlace entre el dispositivo conectado por LoRa y el broker MQTT. Para cumplir su función principal, este gateway interactuará con los siguientes temas del broker MQTT:

- `thingsboard/OMG_ESP32_LORA/LORAtoMQTT`

El gateway enviará aquí los mensajes con el contenido de los paquetes LoRa que reciba. Además, si el paquete LoRa recibido (una vez decodificado) sigue el formato JSON, y lleva dentro un campo «id», el gateway lo dirigirá un subtema definido por el texto de ese campo. Por ejemplo:

Si el paquete LoRa que llega tiene este contenido:

```
{"id": "f024f9b18b08/telemetry", "msg": {"memory_free": 72720} }
```

El mensaje aparecerá en el tema:

`thingsboard/OMG_ESP32_LORA/LORAtoMQTT/f024f9b18b08/telemetry`.

¹²Véase <https://thingsboard.io/docs/reference/rest-api/>

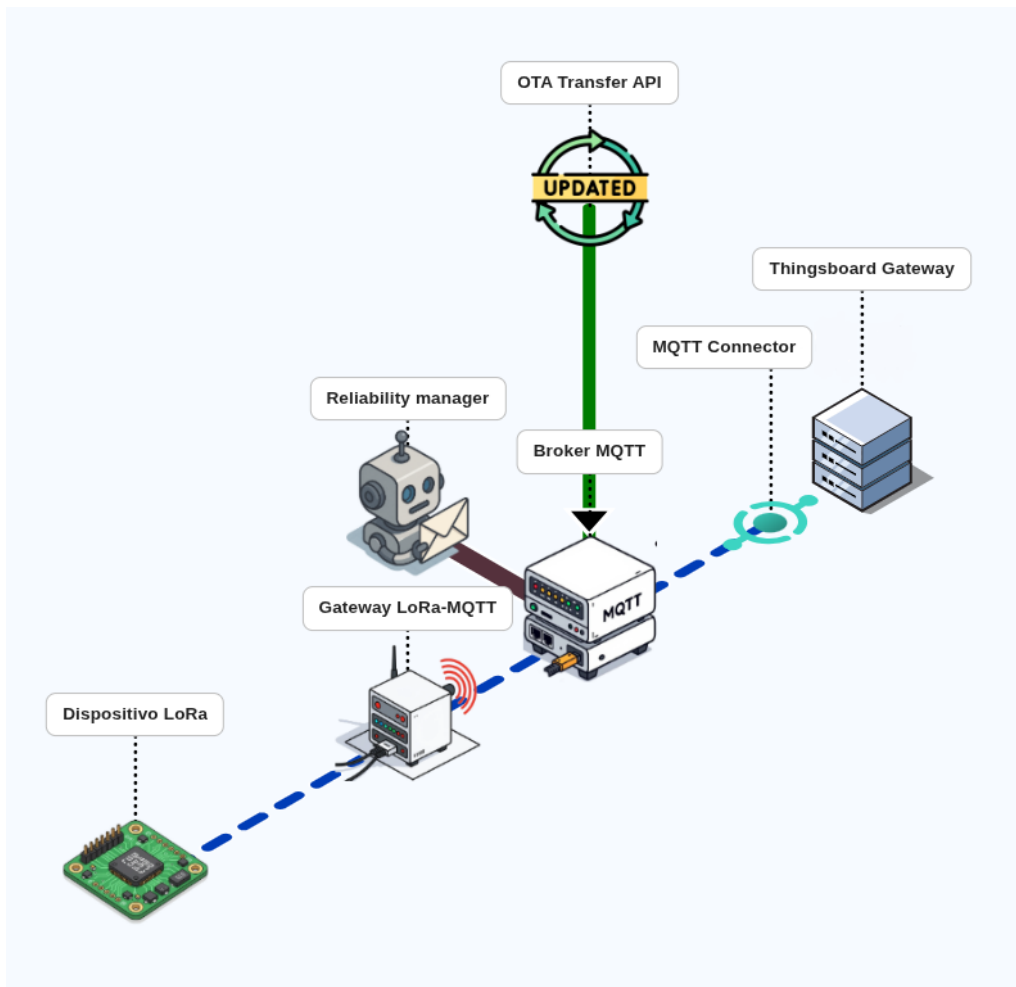


Figura 4.6: Diseño de la comunicación por LoRa, donde se muestra el *broker* MQTT en comunicación con el conector, el *gateway* LoRa-MQTT, el servicio *OTA Transfer* y el componente *Reliability Manager*, el cual asegura un intercambio de mensajes fiable entre dispositivo y *broker* mediante un sistema ACKs.

- `thingsboard/OMG_ESP32_LORA/commands/MQTTtoLORA`

El Gateway leerá los mensajes que lleguen a este tema y emitirá un paquete LoRa con su contenido, que podrá ser recibido por el dispositivo que esté a la escucha. El mensaje debe seguir un formato JSON y tener la siguiente estructura: `{"message": "XXYYZZ"}`, donde XXYYZZ es el mensaje que finalmente se codificará y se emitirá.

Reliability Manager

Un porcentaje considerable de los paquetes LoRa que emiten el dispositivo y el Gateway LoRa-MQTT no llegan a su destino. Por eso, es necesario este componente, que se coordina con el dispositivo conectado por LoRa para asegurar la fiabilidad de los mensajes, usando un procedimiento basado en ACKs que hemos diseñado.

Para asegurar la llegada de los mensajes que viajan **desde el dispositivo hasta el broker MQTT** —y por extensión, hasta la plataforma—, el dispositivo espera-

rá recibir un ACK de vuelta para cada mensaje fiable que quiera enviar. Para ello, el Reliability Manager:

1. Observa al tema donde el dispositivo hace llegar sus mensajes de categoría fiable.
(thingsboard/OMG_ESP32_LORA/LORAtoMQTT/DEVICE-ID/reliable/#)¹³
2. Cuando llegue un mensaje, envía un ACK al tema donde los mensajes se emiten como paquetes LoRa.
(thingsboard/OMG_ESP32_LORA/commands/MQTTtoLORA)

Para asegurar la llegada de los mensajes que viajan **desde el broker MQTT** —por extensión, desde la plataforma— **hasta el dispositivo**, el Reliability Manager hace lo siguiente:

1. Observa el tema donde la plataforma deja los mensajes fiables.¹⁴
(thingsboard/OMG_ESP32_LORA/commands/MQTTtoLORA/reliable/DEVICE-ID)
2. Redirige el mensaje recibido en ese tema al tema donde los mensajes se emiten como paquetes LoRa.
(thingsboard/OMG_ESP32_LORA/commands/MQTTtoLORA)
3. Espera a recibir un ACK en otro tema.
(thingsboard/OMG_ESP32_LORA/LORAtoMQTT/DEVICE-ID/ack)
4. Si no llega el ACK, vuelve al paso 2 para forzar una nueva emisión del paquete LoRa.

4.3.2. Diseño del software de los dispositivos

El software diseñado para los dispositivos con Micropython tiene el fin de manejar adecuadamente la comunicación con la plataforma e implementar una actualización SOTA, que se lanzará en coordinación con Thingsboard. Se hará permitiendo al mismo tiempo la ejecución concurrente de otras tareas, para que el dispositivo pueda realizar su interacción con el mundo físico y cumplir su papel dentro del sistema IoT.

La biblioteca diseñada para cumplir estos propósitos, que llamaremos **thingsboard-ota-helpers**, contendrá la lógica de actualización y las utilidades de comunicación con la plataforma a través de Wifi, BLE y LoRa. Se trata de una biblioteca transversal a todos los escenarios de conectividad y proporciona 4 clases, distribuidas en 4 submódulos que describiremos a lo largo de esta sección:

- ota_installer.py
- updatable_mqtt_client.py
- updatable_ble_peripheral.py
- updatable_lora_node.py

¹³DEVICE-ID es un identificador de dispositivo que usamos para cada dispositivo conectado por LoRa. Es una cadena basada en la MAC del módulo Wifi del dispositivo.

¹⁴Para atributos compartidos y llamadas RPC, establecemos este tema en la configuración del conector MQTT.

Además, proporcionaremos unos programas de ejemplo que harán uso de estas clases para conformar aplicaciones funcionales.

Lógica de actualización

En un dispositivo Micropython, el firmware espera encontrar dos archivos de código fuente en su sistema de ficheros interno, los cuales podemos escribir y nos permiten determinar el comportamiento de la placa:

- **boot.py:** Este script opcional, si está presente, se ejecuta nada más arrancar el sistema. Se usa para realizar una inicialización única personalizada, por ejemplo, para configurar el hardware de la placa o para establecer la conexión de red.

Nuestro `boot.py` comprobará si existe un determinado fichero, correspondiente al paquete de actualización, y, si existe, intentará instalarlo reportando el resultado a Thingsboard. Después, borrará dicho fichero para que no se repita el proceso en la próxima actualización, y dará paso a `main.py`.

- **main.py:** Este script se ejecuta también cada vez que se inicia el sistema, a continuación de `boot.py`, y, como su nombre indica, en su código debe residir la aplicación principal. No tiene por qué tener terminar en un tiempo limitado como `boot.py`, sino que lo más común es que permanezca indefinidamente en ejecución.

Nuestro `main.py`, en paralelo a otras tareas, manejará la recepción de posibles paquetes de actualización provenientes de Thingsboard. Si recibe uno de ellos, tras comprobar su integridad, lo guardará en el mismo archivo donde `boot.py` espera encontrarlo. Seguidamente ejecutará un *reset*.

Aunque estos archivos sirvan como punto de entrada, podemos crear otros, los cuales pueden ser importados como módulos, o abiertos como ficheros de configuración. Un fichero de configuración que obligatoriamente estará en todos nuestros dispositivos será uno llamado «FW_METADATA.json», que almacenará el título y versión actual con este formato:

```
{"title": "micropython-Wifi-MQTT-client", "version": "v0"}
```

Para posibilitar la lógica de actualización descrita anteriormente, la biblioteca `thingsboard-ota-helpers` cuenta con el submódulo **`ota_installer.py`**. Véase el siguiente bloque de código sintetizado, donde se muestra su clase principal, **`OTAInstaller`**, con las cabeceras de sus métodos públicos¹⁵:

```
1 # ota_installer.py
2
3 class OTAInstaller():
4     """
```

¹⁵Aunque Python y Micropython no distinguen entre métodos públicos y privados, a la hora de implementar las clases seguiremos la práctica generalizada de diferenciar los métodos de uso privado nombrándolos con un guion bajo («_») delante.

```

5     Clase manejadora de la instalación de un paquete OTA recibido, con
        métodos para comprobar el correcto formato del archivo, su
        coherencia con los datos reportados por la plataforma y su
        instalación sobre el sistema de ficheros con diferentes
        parámetros de personalización. Está pensada para ser usada en
        la rutina de inicio del dispositivo, tras comprobar que existe
        un nuevo paquete OTA listo para instalarse.
6     """
7
8     def __init__(self, ota_package_path: str, quiet=False)
9         """
10        Contruye un objeto de tipo OTAInstaller.
11        Parámetros:
12            ota_package_path: ruta del paquete de OTA que se pretende
                instalar
13            quiet: si es True, no se emitirá ningún mensaje de log
14        """
15
16    def check_tar_gz_format(self)
17        """
18        Comprueba que el paquete OTA es un fichero con el formato TAR
            GZ, lanzando una excepción en caso negativo.
19        """
20
21
22    def check_metadata_in_package(self)
23        """
24        Inspecciona como un TAR.GZ el fichero de OTA y comprueba que
            contenga dentro el fichero FW_METADATA.json.
25        Los campos 'title' y 'version' de FW_METADATA.json deberán
            coincidir con los reportados con la plataforma antes del
            reinicio, almacenados un fichero
            '<ota_file_name>.metadata.json'.
26        Lanza una excepción si no se superan las comprobaciones.
27        """
28
29
30    def delete_ota_package(self)
31        """
32        Elimina el paquete OTA y su fichero de metadatos asociado.
33        """
34
35
36    def install_firmware(self, excluded_files: list, cleanup: bool)
37        """
38        Aplica el paquete OTA sobrescribiendo el sistema de ficheros.
39        Parámetros:
40            excluded_files: lista de rutas excluidas (no se
                modificarán ni borrarán en ningún caso)
41            cleanup: si es True, se realizará un borrado de todos los
                archivos (menos los excluidos) antes de instalar los
                nuevos archivos.
42        """

```

Como se puede deducir de los métodos que ofrece la clase OTAInstaller, el paquete de actualización esperado es un fichero comprimido con formato TAR GZ, el cual contendrá el árbol de archivos que se desea instalar en el sistema de Micropython.

Dispositivo conectado por Wifi

Como se ha dicho, para este escenario de conectividad haremos uso del SDK **thingsboard-micropython-client-sdk**, el cual nos proporciona una clase denominada **TBDeviceMqttClient**. Esta nos brinda una serie de métodos bastante convenientes para manejar la comunicación con el servidor Thingsboard a través de MQTT, los cuales abstraen funcionalidades como el envío de telemetría, la suscripción a atributos o el establecimiento de manejadores para RPCs. Esta clase implementa incluso la recepción del paquete de actualización.

No obstante, no podemos usar directamente esta clase porque, aunque recibe el paquete de actualización, guardándolo en una variable y verificando su integridad, omite cualquier acción más allá. Por tanto, extenderemos esta clase llamada **TBDeviceMqttClient**, y sobrescribiremos algunos de sus métodos para que se adapte completamente a nuestra lógica de actualización.

Para ello, la biblioteca **thingsboard-ota-helpers** cuenta con el submódulo **updatable_mqtt_client.py**, el cual contiene la clase **UpdatableMqttClient** que extiende a **TBDeviceMqttClient**. En el código que se muestra a continuación, aparece de forma sintética la clase **UpdatableMqttClient** y los métodos que sobrescribe de su clase padre:

```

1 # updatable_mqtt_client.py
2
3 class UpdatableMqttClient(TBDeviceMqttClient):
4     """
5     Clase derivada de TBDeviceMqttClient con las siguientes
6     modificaciones:
7     — Constructor con parámetros opcionales adicionales:
8         — fw_current_title      (título del firmware actual)
9         — fw_current_version    (versión actual del firmware)
10        — fw_filename           (nombre de fichero para almacenar el
11                               paquete OTA que se reciba)
12    — Guarda el firmware recibido en un fichero <fw_filename>, para
13        posibilitar la actualización después de reiniciar.
14    — Guarda el título y versión notificados del nuevo firmware en un
15        fichero <fw_filename>.metadata.json, para poder realizar una
16        comprobación tras el reinicio.
17    — Pospone la notificación del estado UPDATED hasta que el
18        dispositivo aplique la actualización después del reinicio.
19    """
20
21    def __init__(
22        self, host, port=1883, access_token=None,
23        quality_of_service=None, client_id=None, chunk_size=0,
24        fw_current_title="Initial",
25        fw_current_version="v0",
26        fw_filename="new_firmware.tar.gz"
27    ):
28        super().__init__(
29            host, port, access_token, quality_of_service, client_id,
30            chunk_size)
31        ...
32
33    def _on_decode_message(self, topic, msg):

```

```

27         super()._on_decode_message(topic, msg)
28         ...
29
30
31     def __process_firmware(self):
32         ...

```

Usada correctamente, esta clase permitirá realizar todo intercambio de datos con el servidor Thingsboard, incluido el relativo a la actualización OTA con el correspondiente reinicio si se da el caso.

Dispositivo conectado por BLE

Para manejar la comunicación a través de BLE, como se ha mencionado, se usará la biblioteca **aioble**. En este caso, no existe una implementación que facilite el intercambio de datos específico para Thingsboard, así que diseñamos una clase que utilice aioble para trabajar directamente con características de BLE.

Esta clase implementará el rol de Peripheral, actuando como servidor de una serie de características BLE, manejando adecuadamente la actualización OTA y permitiendo también el intercambio de otros datos con el conector BLE de Thingsboard, que actúa con el rol de Central.

La biblioteca thingsboard-ota-helpers cuenta con el submódulo **updatable_ble_peripheral.py**, cuya clase principal, **UpdatableBLEPeripheral**, se muestra de forma sintética en el siguiente código:

```

1 # updatable_ble_peripheral.py
2
3 class UpdatableBLEPeripheral():
4     """
5     Clase que implementa un servidor BLE con el rol Peripheral,
6     conectado con Thingsboard a través del conector BLE, y
7     gestiona la actualización OTA en coordinación con la
8     plataforma.
9
10    Permite exponer un servicio BLE, el cual usa una serie de
11    características para manejar la comunicación con Thingsboard
12    relativa a las OTAs. Además, permite vincular características
13    extra, que pueden ser gestionadas desde fuera de la clase, y
14    pueden ser usadas para intercambiar otro tipo de datos con
15    Thingsboard (telemetría, comandos RPC, etc).
16
17    """
18
19    def __init__(self, fw_current_title="Initial",
20                 fw_current_version="v0", fw_filename="new_firmware.tar.gz"):
21        """
22        Constructor que toma los parámetros indicados para gestionar
23        la OTA con el procedimiento propio de esta biblioteca
24        (thingsboard_ota_helpers).
25
26        """
27
28    def register_service(self)
29        """

```



```
18     Registra el servicio BLE que podrá ser publicitado. Es posible
19     agregar características extra al servicio usando el
20     atributo 'ble_service' antes de llamar a este método.
21     """
22     async def run_advertising(self)
23     """
24     Publicita continuamente el servicio BLE.
25     """
26
27
28     async def run_OTA_manager(self)
29     """
30     Mantiene activa la funcionalidad de la actualización remota:
31     — Monitoriza la comunicación con Thingsboard relativa a la OTA.
32     — Recibe y guarda el paquete de actualización si es necesario,
33       reiniciando el dispositivo en ese caso para que sea
34       instalado al próximo arranque.
35     — Reporta adecuadamente a Thingsboard el estado durante este
36       proceso.
37     """
38
39     async def disconnect(self)
40     """
41     Desconexión del cliente BLE.
42     """
```

Como se puede deducir del código, el uso de esta biblioteca exige la ejecución de tareas concurrentes con *asyncio*. Concretamente, las rutinas `run_OTA_manager` y `run_advertising` deberán permanecer activas para mantener la escucha de la OTA asegurando la conexión. Demostraremos este modo de uso en el programa de ejemplo.

Dispositivo conectado por LoRa

El diseño de la comunicación en el dispositivo LoRa cuenta con una dificultad añadida, y esa es la gestión de la fiabilidad de los mensajes con el intercambio de ACKs expuesto en la descripción del Reliability Manager. Además, trabajaremos con una biblioteca a bastante bajo nivel como es LoRa Driver. Tendremos que definir un formato de mensaje basado en JSON y un identificador obligatorio para intercambiar datos con el gateway LoRa-MQTT.

Si bien JSON puede no ser lo más eficiente en casos donde el tamaño de paquete es bastante reducido y la velocidad de transmisión es muy baja —como es el caso de LoRa—, creemos que nos compensa usarlo en este caso por la agilidad que proporciona.

Diseñaremos otra clase con un comportamiento similar a la de BLE, con métodos asíncronos, pero se encargará de manejar envíos y recepciones de mensajes en lugar de características BLE. Proporcionará métodos para el envío y recepción de mensajes, abstrayendo el tratamiento de los paquetes LoRa de forma que el usuario podrá trabajar directamente con diccionarios, que expresan pares clave-valor

de forma interoperable con JSON. Esos pares clave-valor dentro de un JSON será también la lógica que tendrá el conector MQTT para hacer la equivalencia entre los mensajes de sus temas y los datos de Thingsboard (tales como telemetría, atributos o RPCs).

La clase descrita, llamada **UpdatableLoRaNode**, forma también parte de la biblioteca `thingsboard-ota-helpers`, dentro del submódulo `updatable_lora_node.py`. En el siguiente código se muestra una síntesis de su interfaz con una descripción de cada uno de sus métodos:

```

1 # updatable_lora_node.py
2
3 class UpdatableLoRaNode():
4     """
5     Clase que implementa un nodo LoRa en comunicación con el gateway
6     LoRa-MQTT para comunicarse con la plataforma Thingsboard a
7     través del conector MQTT. Gestiona automáticamente las OTAs
8     lanzadas desde Thingsboard. Permite el envío de mensajes al
9     gateway, y el manejo personalizado de mensajes entrantes.
10    """
11
12    def __init__(self, lora_modem, fw_current_title="Initial",
13                  fw_current_version="v0", fw_filename="new_firmware.tar.gz"):
14        """
15        Este constructor toma como 1r parámetro un modem lora
16        asíncrono creado con la biblioteca lora
17        (https://github.com/micropython/micropython-lib/tree/master/micropython/lora)
18        además de los parámetros indicados para gestionar la OTA
19        con el procedimiento propio de esta biblioteca
20        (thingsboard_ota_helpers).
21        """
22
23    async def connect(self):
24        """
25        Realiza unos envíos preliminares a Thingsboard para empezar a
26        interactuar con seguridad.
27        (indicación 'connect' + atributos 'ota_connectivity'='LoRa',
28         'lora_id'=? )
29        """
30
31    async def send(self, subtopic, msg):
32        """
33        Envía un paquete LoRa correctamente estructurado al gateway
34        para que viaje al tema indicado.
35        """
36
37    async def reliable_send(self, subtopic, msg):
38        """
39        Encapsulación del método send() para asegurar la recepción del
40        mensaje, reenviándolo hasta recibir un ACK de vuelta. El
41        Reliability Manager debe estar activo.
42        """

```

```
34 def set_callback(self, callback)
35     """
36     Establece una función de callback para recibir mensajes. Dicha
        función debe recibir un parámetro de tipo 'dict', pues los
        mensajes a tratar serán JSONs decodificados.
37     """
38
39
40 async def listen(self)
41     """
42     Permanece a la escucha de paquetes LoRa entrantes que estén en
        formato JSON y lleven un campo 'id' con identificación de
        este dispositivo, de esta forma: {"id": "LORA_ID", ...}
43     Siendo LORA_ID el identificador de este dispositivo en el
        atributo de instancia <device_id>.
44     Gestiona automáticamente la recepción de ACKs, reacciona
        adecuadamente a los mensajes relativos a la actualización
        OTA y usa la función de callback definida para recibir el
        resto de los mensajes.
45     """
```

El correcto uso de esta clase implicará la llamada inicial al método `connect`, y la ejecución continua del método `listen` para, entre otras cosas, manejar el proceso de la OTA. En el programa de ejemplo se mostrará cómo hacer esto de forma concurrente con otras tareas que pueda desempeñar el dispositivo, además de cómo crear ese objeto «`lora_modem`» que recibe el constructor, teniendo en cuenta las características de nuestro hardware.

CAPÍTULO 5

Desarrollo de la solución propuesta

En este capítulo se expone cómo este trabajo ha pasado del diseño al resultado final. Esencialmente, se describirá el proyecto de código llamado **thingsboard-updatable-devices** [5] que se ha desarrollado para implementar el sistema propuesto y cumplir así con los objetivos planteados.

En primer lugar, se presentará la estructura del proyecto, para tener una visión general y dar una idea clara de qué partes implementan cada funcionalidad del diseño. Después, se comentará una selección de características que se han materializado durante el desarrollo del proyecto y consideramos especialmente relevantes. Estas características harán referencia a aspectos del desarrollo en Micropython, el desarrollo de recursos en Thingsboard, la creación de paquetes de actualización y la infraestructura de comunicación.

5.1 Estructura del proyecto

A continuación se describe el árbol de ramas y el árbol de archivos del proyecto.

5.1.1. Árbol de ramas

Este proyecto, como repositorio de git, se ha desarrollado con una rama principal llamada *máster*, que solo ha avanzado mediante la fusión de otras ramas de características sobre ella. Estas ramas de características se han ido desarrollando y fusionando sobre *master* de una en una. En la figura 5.1 podemos ver las ramas que se planificaron en total:

Siguiendo el enfoque de desarrollo incremental, cada rama no se limitó solo a su característica principal, sino que también implicó la integración y ajustes de características anteriores.

Ramas de características desarrolladas hasta la fecha:

- ☒ 01-minimum_iot_system
- ☒ 02-basic_ota_update
- ☒ 03-improved_ota_update
- ☒ 04-ble_support
- ☒ 05-lora_support
- ☐ 06-usability-improvements
- ☐ 07-adaptation_to_linux
- ☐ 08-final_revision

Figura 5.1: Captura extraída del archivo README.md en la raíz del proyecto. En las ramas 01 y 02 se desarrolló la prueba de concepto. En la rama 03 se dio por finalizada la integración del dispositivo Wifi. En las ramas 04 y 05 se desarrolló la integración de los dispositivos BLE y LoRa, respectivamente. Nótese que aún quedan ramas de características por desarrollar, lo que da a entender la determinación de seguir avanzando este trabajo en el futuro.

5.1.2. Árbol de archivos

En la figura 5.2 se muestra la estructura de archivos general del proyecto, sin entrar al máximo nivel de profundidad de los directorios, para obtener una visión general.

Más adelante veremos algunos de estos directorios más en profundidad para describir algunas características relevantes.

5.2 Características relevantes

A continuación se descubrirán las características que hemos considerado más relevantes de entre las que se han determinado y concretado en la etapa de desarrollo, siendo estas esenciales para entender plenamente el funcionamiento de este proyecto.

5.2.1. Desarrollo en Micropython

Funcionamiento general

Todo el código para Micropython que se ha programado podría resumirse en el propósito de cumplir con un determinado comportamiento bajo las tres formas de conectividad propuestas (Wifi, BLE y LoRa). Este comportamiento se puede reducir a los flujos de ejecución que provoquen los dos archivos *boot.py* y *main.py*, que se lanzan de forma consecutiva. Siguiendo esta lógica, hemos elaborado dos diagramas de flujo que resumen el comportamiento base que deben cumplir los dispositivos, y se muestran en la figura 5.3 y la figura 5.4.

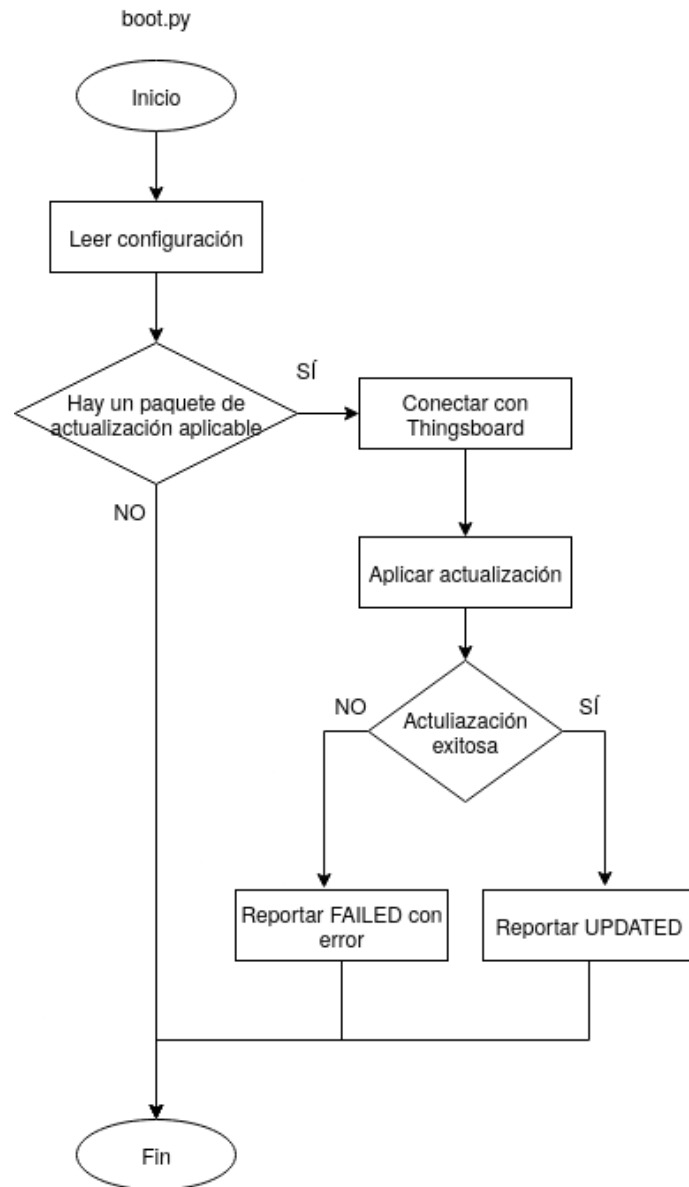


Figura 5.3: Diagrama de flujo del script *boot.py*.

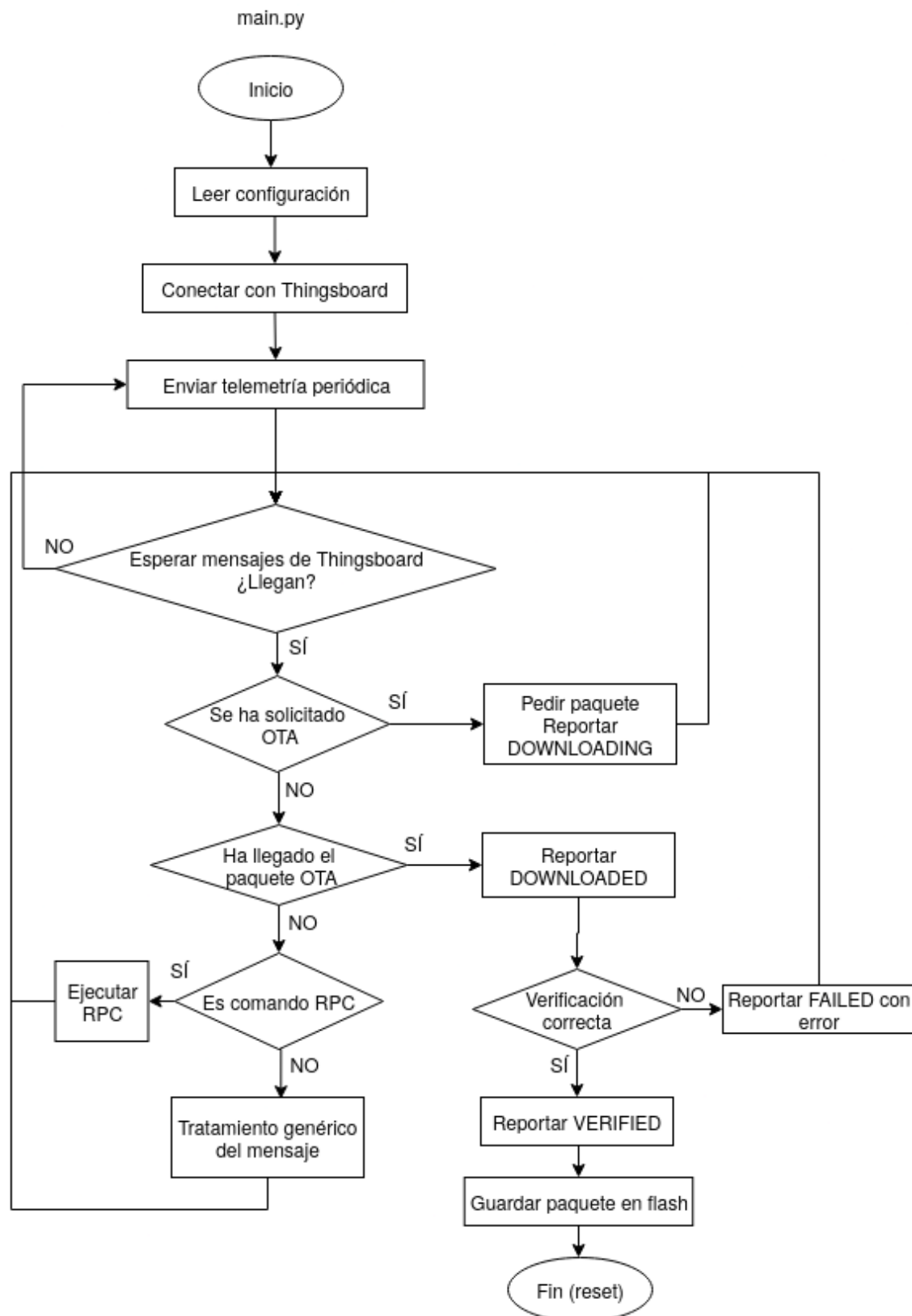


Figura 5.4: Diagrama de flujo del script *main.py*.

Estructura de archivos

Con el fin de cumplir el funcionamiento expuesto, se ha organizado el código con una estructura que tenga en cuenta qué aspectos son comunes entre dispositivos de distinta clase y cuáles son específicos de cada forma de conectividad. En esta estructura se ha desarrollado tanto código de biblioteca (en especial la biblioteca ya explicada *thingsboard-ota-helpers*) como código para las aplicaciones de ejemplo (que son tres, para cubrir los escenarios de Wifi, BLE y LoRa). En la figura 5.5 se muestra la estructura del directorio *src* (dentro de *devices/micropython/*), y en la figura 5.6 se muestra la del directorio *config* (también dentro del mencionado directorio).

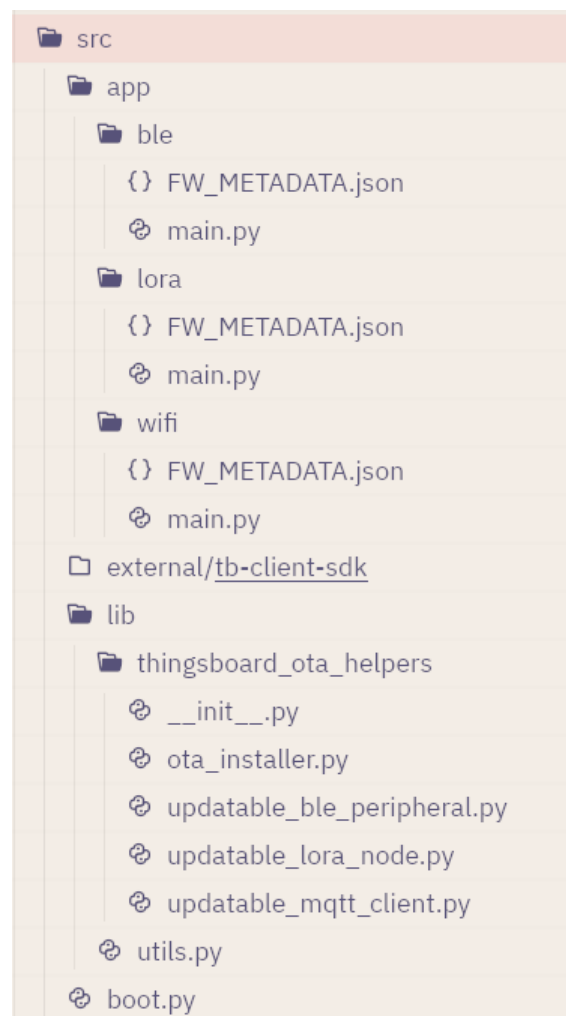


Figura 5.5: En este directorio se almacenan todos los archivos *.py*. En primer lugar, aparece el directorio *app/*, que contiene algunos archivos que son distintos para cada tipo de dispositivo: el *main.py* para cada aplicación de ejemplo y su correspondiente *FW_METADATA.json*. Después se observa el directorio *external/tb-client-sdk*, que almacena la biblioteca externa *thingsboard-micropython-client-sdk*. Más abajo aparece *lib/*, que contiene la biblioteca más importante de este proyecto junto a un archivo de utilidades *utils.py*, que es común a todos los programas de ejemplo y nos sirve para gestionar varios aspectos comunes sin sobrecargar los scripts *boot.py* y *main.py*. Por último, se muestra *boot.py*, que es común a todos los dispositivos porque gracias a la configuración averigua dinámicamente qué forma de conectividad necesita y usa un helper de comunicación u otro.

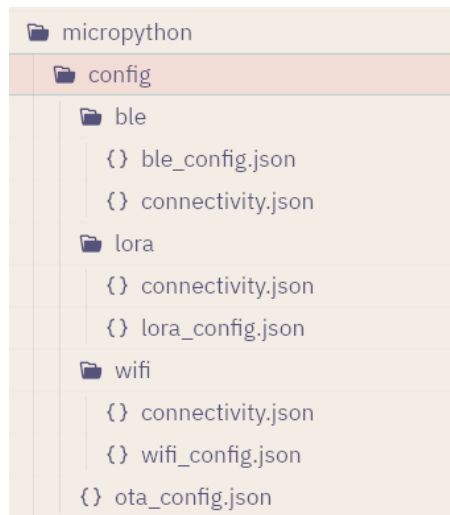


Figura 5.6: En este directorio se almacenan todos los archivos de configuración que usan las aplicaciones de ejemplo. Con la información de estos ficheros, cada aplicación configurará adecuadamente tanto el *OTAInstaller* (con *ota_config.json*) como el *helper* de comunicación específico que le corresponda (con *wifi_config.json* o *ble_config.json*). El fichero *connectivity.json* sirve para que *boot.py* (común a todos los dispositivos) averigüe dinámicamente qué forma de conectividad necesita.

Paquetes *mip*

Los paquetes *mip*, en su versión local, se implementan mediante ficheros JSON que hacen una equivalencia entre rutas de la máquina local y rutas del sistema de ficheros del dispositivo Micropython, donde se copiarán los archivos locales; incluso también pueden especificar dependencias con bibliotecas de Micropython que se pueden bajar de Internet al momento de instalar el paquete en cuestión.

La figura 5.7 muestra el directorio *devices/micropython/mip_packages/* donde se han desarrollado estos ficheros.

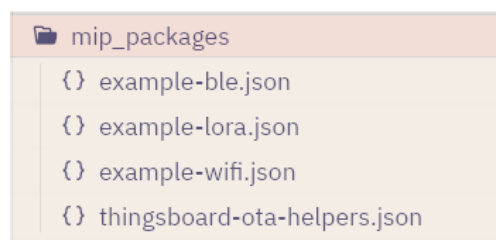


Figura 5.7: En este directorio, con los ficheros JSON que empiezan por *example*, se definen paquetes capaces de instalar las aplicaciones de ejemplo; y, lo que es más importante, con el archivo *thingsboard-ota-helpers.json* se define el paquete *mip* de la biblioteca *thingsboard-ota-helpers*.

Por su importancia, se adjunta el fichero *thingsboard-ota-helpers.json* en el apéndice A.

Fork del repositorio *thingsboard-micropython-client-sdk*

Una de las peculiaridades que tiene nuestro repositorio *thingsboard-updatable-devices* es que incluye como un submódulo de *git* el repositorio del SDK *thingsboard-micropython-client-sdk*. Recordemos que este SDK se usa como ayuda para resolver el caso de MQTT sobre Wifi en nuestra biblioteca *thingsboard-ota-helpers*. La razón de incluirlo como un submódulo, y no como una dependencia externa del paquete *mip*, radica en que actualmente este SDK no está disponible para ser descargado con el sistema *mip*, así que sus archivos Python han de estar presentes dentro de nuestro proyecto.

Sin embargo, al empezar a desarrollar código con este SDK, dimos con un problema, y es que, a pesar de resolver con eficacia casi todas las funcionalidades para las que ha sido desarrollado, contenía un inesperado *bug*. Este aparecía a veces en la rutina de recepción de los mensajes de Thingsboard, y estaba relacionado con el uso incorrecto de una variable (al parecer, por confundir el tipo que tomaba en un cierto escenario).

Así pues, decidimos hacer un *fork* del repositorio donde se aloja *thingsboard-micropython-client-sdk* para corregir dicho *bug*, creándole una nueva rama llamada *patch/functional_improvements*¹ en la que se corrigió. Después cambiamos la referencia del submódulo para que apuntase a nuestro *fork*, y más específicamente a la rama mencionada.

Modelo de concurrencia

Concluiremos la exposición de características relevantes sobre el desarrollo en Micropython refiriéndonos al modelo de concurrencia que hemos seguido para programar los dispositivos. Este ha sido un modelo de eventos con programación asíncrona basada en la biblioteca *asyncio* de Micropython, que es una adaptación de su homónima en Python. Esta biblioteca permite la ejecución de múltiples tareas de manera concurrente sin bloquear el hilo principal, cosa especialmente útil en dispositivos con recursos limitados, donde la eficiencia es crucial. Es fundamental cuando interviene cualquier tipo de comunicación asíncrona, como es el caso, y el dispositivo requiere hacer varias tareas de forma concurrente sin tener que bloquearse para recibir mensajes.

Las aplicaciones usarán este modelo para llevar a cabo todas sus tareas de forma concurrente, las cuales incluyen, en el caso de las aplicaciones de ejemplo: manejar la comunicación entrante de Thingsboard relativa a las OTAs y los comandos RPC, hacer parpadear un LED de *heartbeat*, y enviar periódicamente telemetría hacia Thingsboard (concretamente, telemetría que indica el estado de la memoria del dispositivo). Como muestra, la figura 5.8 enseña una versión resumida del programa principal de ejemplo en el escenario de LoRa.

¹Véase https://github.com/p4blo0S/thingsboard-micropython-client-sdk/commits/patch/functional_improvements/

Figura 5.8: Versión reducida del script main.py correspondiente a la versión para LoRa de la aplicación de ejemplo, donde se enfatiza su estructura asíncrona.

```

1 async def memory_report(lora_node, period_s):
2     ...
3
4 async def heartbeat_LED():
5     ...
6
7 def on_message_callback(recv_data):
8     if recv_data.get("rpc") == "garbage_collection":
9         log.info("Garbage Collection invocado desde la plataforma")
10        gc_collect()
11
12
13 async def main():
14     """
15     Ejecuta concurrentemente las tareas asíncronas definidas.
16     """
17     lora_node = utils.get_updatable_lora_node()
18     lora_node.set_callback(on_message_callback)
19     await asyncio_gather(
20         lora_node.connect(),
21         lora_node.listen(),
22         heartbeat_LED(),
23         memory_report(lora_node, 2)
24     )
25
26
27 if __name__ == "__main__":
28     """
29     Punto de entrada al programa principal
30     """
31     log.info("Iniciando programa principal")
32     asyncio_run(main())

```

5.2.2. Desarrollo de recursos en Thingsboard

Usando las herramientas que nos proporciona la interfaz web de Thingsboard, hemos desarrollado un conjunto de recursos pensados para ser importados, de forma que cualquier usuario de Thingsboard podrá usarlos si cuenta con los archivos adecuados. En el directorio *platform/resources/* del repositorio, que se muestra en la figura 5.9, hemos almacenado esos archivos.

A continuación describiremos los más destacables de dichos recursos.

Cadena de reglas

Para cumplir con las funciones de la cadena de reglas descritas en el diseño, fue necesario emplear una cantidad considerable de nodos conectados entre sí de una forma nada trivial. Además, cada nodo está configurado internamente, algunos incluso con código Javascript, para personalizar su función, ya sea esta recuperar algún atributo del dispositivo cuyo mensaje ha llegado, filtrar el dis-

positivo por la forma en que se ha conectado, realizar la llamada a la API del servicio OTA Transfer, etc. La figura 5.10 puede dar una idea de la lógica que hemos seguido para obtener el funcionamiento esperado, especialmente si se presta atención a las pequeñas descripciones de cada nodo.

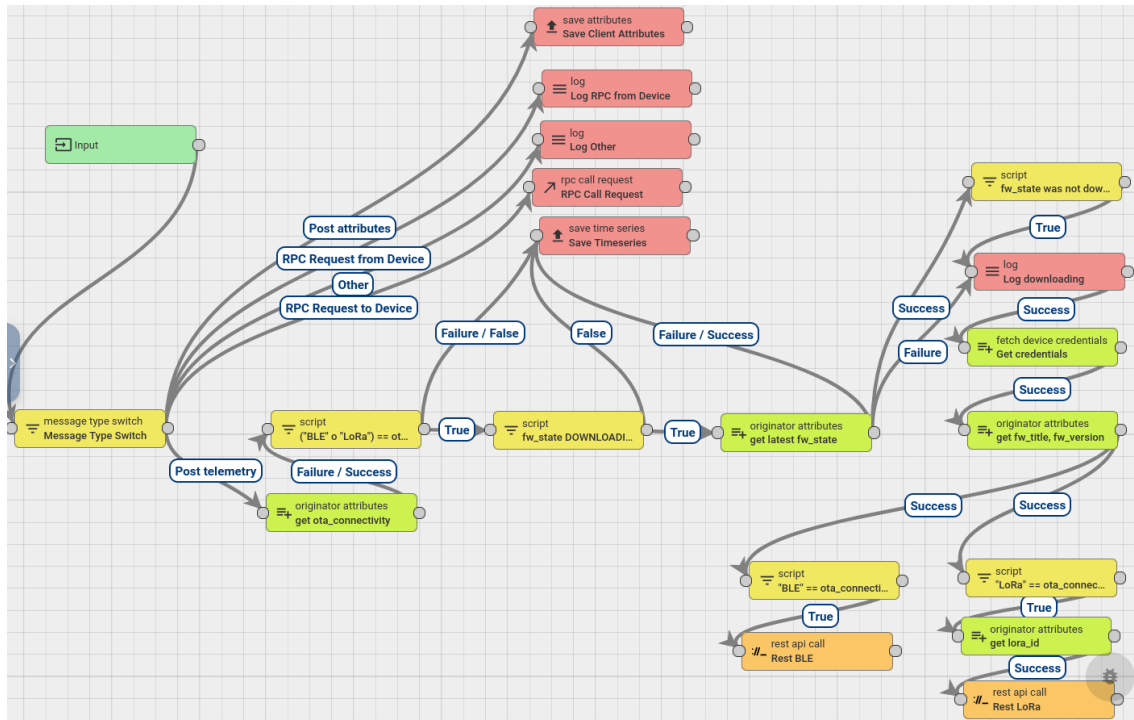


Figura 5.10: Cadena de reglas *Updatable-Devices-rule-chain*, diseñada para disparar el servicio OTA Transfer en el momento oportuno y así posibilitar la OTA a través de BLE y LoRa. Es un recurso importable con el archivo *updatable_devices_rule_chain.json* y está pensado para comportarse como la cadena de reglas raíz del servidor Thingsboard.

Dashboard

El Dashboard elaborado es el elemento más visible entre los que nos ayudan a cumplir nuestro objetivo de aportar una interfaz común para la gestión de los distintos dispositivos. La figura 5.11 nos muestra su versión final en funcionamiento.

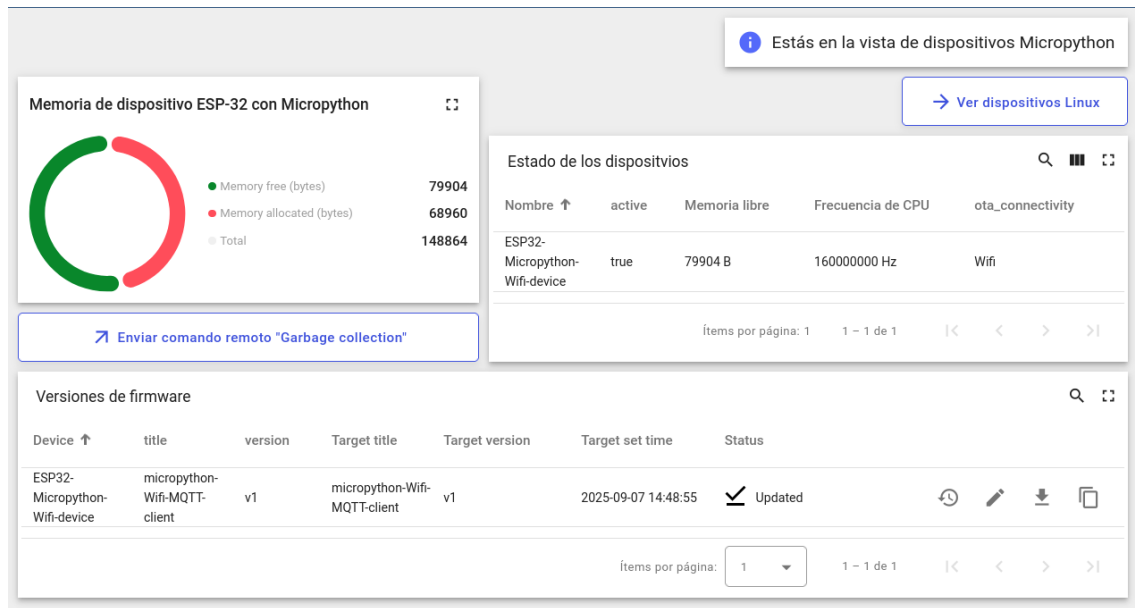


Figura 5.11: Dashboard titulado *Updatable-Devices-dashboard*, importable con el fichero *updatable_devices_dashboard.json*. Está pensado para gestionar dispositivos Micropython como los de las aplicaciones de ejemplo. Además, cuenta con una segunda pantalla desde donde se podrán gestionar los futuros dispositivos con Linux. Ofrece widgets con las siguientes funcionalidades: monitorización del estado de memoria de un dispositivo; envío del comando RPC "Garbage collection"; monitorización de algunos atributos de varios dispositivos (incluyendo el atributo *ota_connectivity*); y, por último, gestión de las versiones de firmware (para nuestro caso, software, en realidad) de varios dispositivos conectados, incluyendo la posibilidad de actualizarlos.

5.2.3. Creación de paquetes de actualización

Las herramientas que se exponen a continuación se encuentran en el directorio *devices/micropython/tools/*, cuyo contenido se muestra en la figura 5.12.

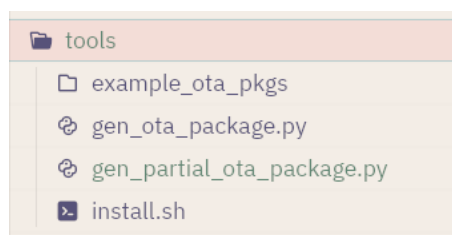


Figura 5.12: Directorio *tools/*, donde se almacenan paquetes de actualización de ejemplo en su subdirectorio *example_ota_pkgs/*, y tres scripts de utilidades: *gen_ota_package.py*, *gen_partial_ota_package.py* e *install.sh* (este último automatiza la instalación completa de las aplicaciones de ejemplo recibiendo como argumento *wifi*, *ble* o *lora*)

`gen_ota_package.py`

Como se ha mencionado anteriormente en este trabajo, nuestro sistema usa, como paquetes de actualización de software, archivos comprimidos en formato TAR GZ, que contienen un árbol de archivos para sobrescribir al que hay en el

sistema de ficheros del dispositivo. Con el fin de automatizar la creación de estos paquetes, se desarrolló el script llamado *gen_ota_package.py*, cuyo funcionamiento puede entenderse observando la figura 5.13.

```
(venv) [maxdmian@pavilion-convertible micropython]$ ./tools/gen_ota_package.py --help
usage: gen_ota_package.py [-h] [-n NAME]

Genera, en el directorio tools/generated, un paquete OTA para MicroPython a partir del estado actual del dispositivo conectado con mpremote.

options:
  -h, --help            show this help message and exit
  -n, --name NAME       Nombre del archivo de salida (por defecto se forma a partir de la info. encontrada en FW_METADATA.json)
(venv) [maxdmian@pavilion-convertible micropython]$ ./tools/gen_ota_package.py
Copiando sistema de ficheros del dispositivo a un directorio temporal
cp :/ /tmp/tmp44glutd9
Creando archivo comprimido TAR GZ
Archivo de salida creado: /home/maxdmian/codigo/prueba_tfg/thingsboard-updatable-devices/devices/micropython/tools/generated/micropython-Wifi-MQTT-client_v1.tar.gz
```

Figura 5.13: Ejecución del programa de línea de comandos llamado *gen_ota_package.py*. Con un dispositivo Micropython conectado, primero se lanza el mensaje de ayuda y después se ejecuta sin argumentos para lanzar su función principal.

gen_partial_ota_package.py

En un determinado momento del desarrollo, cuando la biblioteca *thingsboard-ota-helpers* había crecido considerablemente en tamaño, se empezaron a producir errores en el proceso de actualización del dispositivo, con logs que indicaban lo siguiente:

```
MemoryError: memory allocation failed, allocating 17152 bytes
```

El problema era que el dispositivo se quedaba sin memoria al intentar guardar en memoria principal todo el paquete de actualización proveniente de Thingsboard, y es que la biblioteca *thingsboard-ota-helpers* había hecho crecer también a los paquetes de actualización. Aunque este problema tenía varias soluciones, decidimos atajarlo reduciendo el tamaño del paquete.

Es por ello que se diseñó el script *gen_partial_ota_package.py*, cuya funcionalidad será crear un paquete de actualización parcial a partir de dos paquetes de actualización completos. El resultado será un paquete de actualización con un tamaño mucho más reducido, porque únicamente incluirá dentro los archivos del nuevo paquete que difieran con los del antiguo. Para usar este tipo de paquetes con el sistema de ejemplo, habrá que asegurarse de configurar el archivo *ota_config.json* con su parámetro *clear_filesystem* a *false*.

5.2.4. Infraestructura e intermediarios de comunicación

Estructura del despliegue

El despliegue de la infraestructura de nuestro sistema se ha desarrollado usando Docker Compose como principal tecnología. Véase la figura 5.14 para obtener una vista general de los archivos del proyecto destinados a este despliegue.

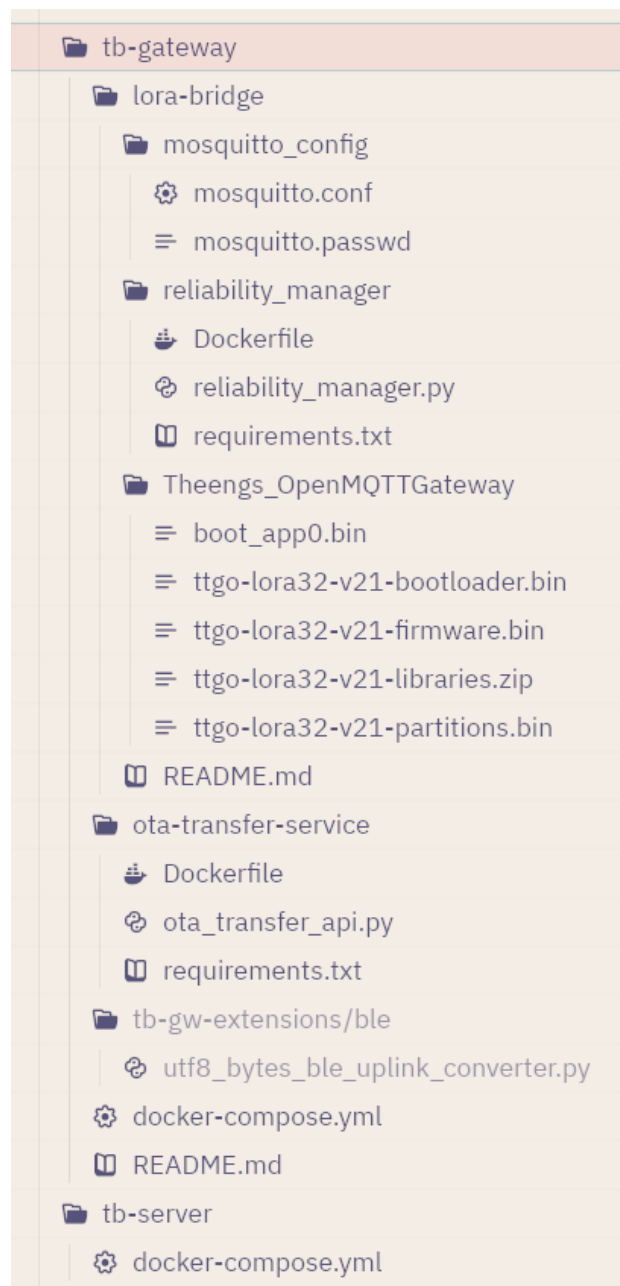


Figura 5.14: En esta imagen se muestran los directorios `tb-gateway` y `tb-server`, (ubicados en su directorio padre `platform/`) desplegados hasta su último nivel.

El directorio `tb-server` contiene un archivo de Docker Compose (`docker-compose.yml`) que sirve únicamente para desplegar el servidor Thingsboard.

El directorio *tb-gateway* contiene otro archivo de Docker Compose, que no solo sirve para desplegar el gateway de Thingsboard, sino también los servicios que consideramos auxiliares porque complementan su funcionalidad: OTA Transfer Service, broker MQTT y Reliability Manager.²

En el subdirectorio *tb-gateway/lora-bridge/* se agrupan los archivos que sirven específicamente para el caso de comunicación por LoRa: *mosquitto_config/* es un volumen que sirve para configurar la imagen Docker externa del broker MQTT; en el directorio *reliability_manager* se encuentra definida la imagen Docker del Reliability Manager; y, aunque no está gestionado por Docker Compose, se incluye también el directorio *Theengs_OpenMTTGateway*, que contiene firmware listo para instalarse en la placa TTGO LoRa32 (con esto último se desplegaría el gateway LoRa-MQTT).

En el subdirectorio *tb-gateway/ota-transfer-service* se define la imagen Docker relativa al servicio OTA Transfer.

En el capítulo 6 (Implantación) se describirán los pasos para realizar el despliegue usando todos estos archivos.

Converter personalizado *utf8_bytes_ble_uplink_converter.py*

A continuación se explica la existencia del archivo con ruta en *platform/tb-gateway/tb-gw-extensions/ble/utf8_bytes_ble_uplink_converter.py*.

Al configurar el conector BLE del Thingsboard Gateway para que pudiese recibir telemetría del dispositivo a través de características de BLE, nos dimos cuenta de que aparecían valores de telemetría en la plataforma, pero estos no eran los valores originales; eran extrañas cadenas formadas por números del 0 al 255. El problema era que el *Uplink Converter*³ por defecto tenía ese comportamiento. Este elemento recibe una cadena de bytes y convierte cada byte por separado a su interpretación como *string* (que viene a ser el código ASCII en decimal del carácter representado en ese byte). Con lo cual, siempre se obtienen cadenas que son concatenaciones de números en el rango de los códigos ASCII (de 0 a 255), lo cual nos resultó extremadamente inconveniente.

En Thingsboard, un *converter*, en la práctica, es simplemente un archivo de Python; y Thingsboard facilita las cosas para poder usar uno propio. Así que, partiendo del *converter* por defecto⁴, desarrollamos el nuestro propio (llamado *utf8_bytes_ble_uplink_converter.py*), y lo insertamos en la plataforma usando el volumen que la imagen Docker del Thingsboard Gateway proporciona para extender su funcionalidad: *tb-gw-extensions*. Desde el momento en que el *converter* personalizado se encuentra dentro de ese volumen, podemos usarlo en la configuración del conector BLE.

²En el apéndice A se adjuntan los archivos de Docker Compose con las rutas *platform/tb-server/docker-compose.yml* y *platform/tb-gateway/docker-compose.yml*.

³O sea, el elemento que procesa los datos que llegan por Bluetooth como *buffers* de bytes para convertirlos a un formato de datos que tenga sentido en la plataforma.

⁴Véase el archivo *bytes_ble_uplink_converter.py*, *Uplink Converter* por defecto en el repositorio de Thingsboard: https://github.com/thingsboard/thingsboard-gateway/blob/master/thingsboard_gateway/connectors/ble/bytes_ble_uplink_converter.py

En este *utf8_bytes_ble_uplink_converter.py* empleamos una decodificación que, a nuestro parecer, tiene mucho más sentido: una que usa UTF8. De esta forma, desde el lado del dispositivo, se pueden enviar cadenas codificadas con UTF8, que serán decodificadas en la plataforma para dar valores a datos provenientes del dispositivo como la telemetría.

Reliability Manager y manejo de ACKs

La lógica del componente Reliability Manager, adelantada en el diseño explicado en el capítulo anterior, se materializa en el programa de Python *tb-gateway/lorabridge/reliability_manager/reliability_manager.py*. Este programa es de vital importancia, ya que permite el intercambio de mensajes fiables con el dispositivo conectado por LoRa, entre los que se incluyen los mensajes relativos a la OTA.

Los ACKs que manejan ambos componentes usan identificadores, implementados como una cuenta incremental, los cuales sirven para detectar mensajes duplicados, aspecto crucial a la hora de transmitir por fragmentos el paquete de actualización (se ha de hacer por orden y sin duplicidades). Sin embargo, todo esto exige una especial atención a la concurrencia, cosa que se ha tenido en cuenta al programar estos intercambios de ACKs.

Además, una particularidad del Reliability Manager es que optimiza dinámicamente el tiempo de espera para reintentar un envío si no se ha recibido un ACK. Lo hace teniendo en cuenta el tiempo mínimo que alguna vez ha tardado en recibir el ACK de vuelta, y esto lo considera el RTT (*Round-Trip Time* o tiempo de ida y vuelta), que en este caso es el tiempo que tarda en ir y volver un mensaje desde el Reliability Manager al dispositivo LoRa, pasando por el broker y el gateway LoRa-MQTT. Con esta optimización, el Reliability Manager se adapta a las diferentes velocidades que puede tener la comunicación por LoRa dependiendo del entorno y la configuración. Así, puede reducir el margen que pasa en tiempo de espera y transmitir el paquete de actualización en mucho menos tiempo.

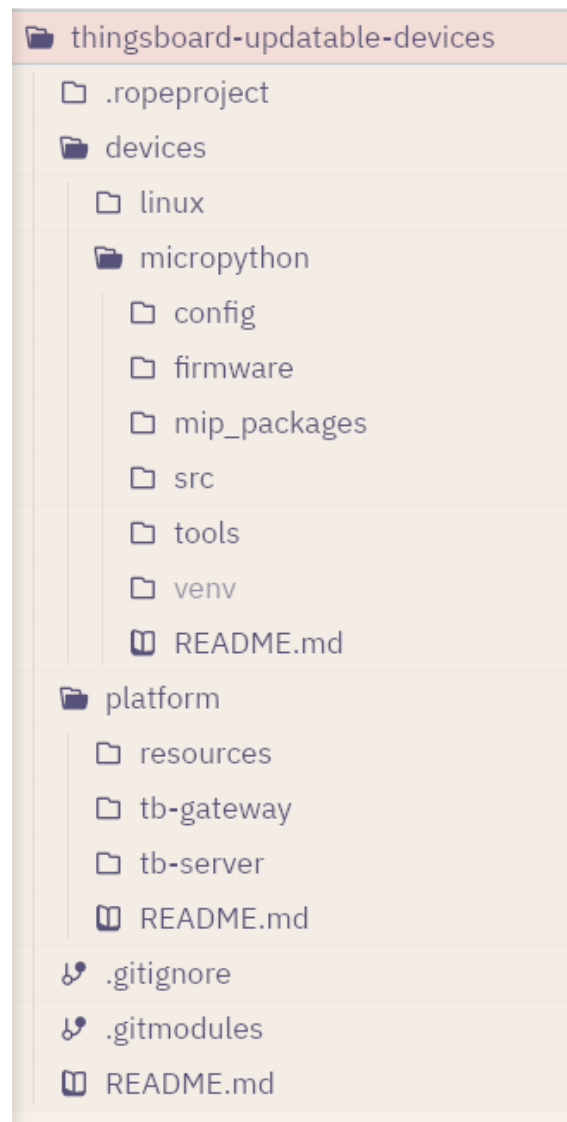


Figura 5.2: Estructura de archivos general del proyecto. Como se puede observar, este proyecto se divide en dos grandes directorios: *devices* y *platform*. *devices* contiene todo el código hecho para ser instalado en los dispositivos del borde, mientras que *platform* contiene todos los archivos necesarios para desplegar y configurar la infraestructura (no solo la plataforma, sino también sus componentes auxiliares que harán posible la comunicación con ella en todos los escenarios propuestos).



Figura 5.9: Directorio donde se almacenan los recursos importables desde Thingsboard en forma de archivos JSON.

CAPÍTULO 6

Implantación

En este capítulo se presenta el proceso de implantación del sistema desarrollado, con una descripción punto por punto de cada acción necesaria para ponerlo en funcionamiento. De esta forma, establecemos un método por el cual se puede reproducir desde cero la solución creada, usando los componentes creados por nosotros mismos y también otros de terceros, todos los cuales habrán de ser configurados debidamente para adaptarse al entorno donde se requieran.

El proceso se describirá asumiendo el uso de al menos una estación de trabajo con Linux y una placa de desarrollo TTGO LoRa32 para desplegar toda la infraestructura. En el lado de los dispositivos del borde, las instalaciones descritas han sido probadas con la placa ESP32 WROOM para los escenarios de Wifi y BLE, y con otra placa TTGO LoRa 32 para el escenario LoRa. Para hacer dichas instalaciones en los dispositivos, se asume el uso de un equipo de desarrollo que tendrá las herramientas *esptool* y *mpremote* instaladas.

La implantación que se presenta a continuación se plantea como la de un sistema prototipo, en el cual se ponen en marcha los programas de ejemplo y se usan configuraciones predeterminadas. No obstante, se da a entender en cada punto cómo podríamos adaptar estas configuraciones para encajar el sistema en otros entornos.

6.1 Despliegue de la infraestructura

En esta sección se describe el despliegue de la plataforma Thingsboard y todos los componentes de comunicación que hacen posible su conexión con dispositivos actualizables, contemplando los tres escenarios de conectividad implementados.

En un principio, nuestra estación de trabajo con Linux tan sólo requerirá **docker** y **git** como dependencias.

Como punto de partida para desplegar estos componentes, deberemos clonar el repositorio donde se ha desarrollado este trabajo, teniendo en cuenta que contiene un submódulo que se debe inicializar:

```
1 git clone https://github.com/p4blo0S/thingsboard-updatable-devices.  
   git  
2 cd thingsboard-updatable-devices/
```

```
3 | git submodule update --init --recursive
```

6.1.1. Despliegue del servidor Thingsboard

Para levantar el servidor Thingsboard por primera vez, levantaremos su servicio de Docker Compose¹ como se muestra a continuación:

```
1 | cd platform/tb-server # Aquí se encuentra el archivo docker-compose.
   | yml
2 | docker compose up -d # Se lanza el contenedor en segundo plano
```

A partir de este momento, estos son algunos comandos útiles que nos pueden servir para monitorizar y controlar el contenedor (se tendrán que ejecutar desde el mismo directorio donde se encuentra `docker-compose.yml`):

```
1 | docker compose logs -f tb-server # Consultar los logs
2 | docker compose stop tb-server    # Detener el contenedor
3 | docker compose start tb-server    # Reanudar el contenedor
```

Una vez terminado todo el proceso de inicialización de la plataforma (cosa que puede tardar varios minutos), la interfaz web permanecerá a la escucha en <http://localhost:8080/>. Para acceder a ella, es necesario identificarse con un usuario y contraseña, pues Thingsboard cuenta con un sistema de roles y permisos bastante sofisticado. Si bien es cierto, para el caso que nos ocupa y mientras no requiramos de una gestión de usuarios más profesional, podemos hacer uso del usuario y contraseña por defecto que se muestra en la figura 6.1.

Una vez dentro, deberemos dejar el servidor listo para permitir el despliegue de un Gateway, cosa que implica añadir en Thingsboard una entidad Gateway usando la interfaz web. Usando el panel izquierdo, iremos a *Entidades > Gateways* y añadiremos un Gateway con el nombre *Multi-Connection-gateway* y el perfil *default*, tal como se muestra en la figura 6.2:

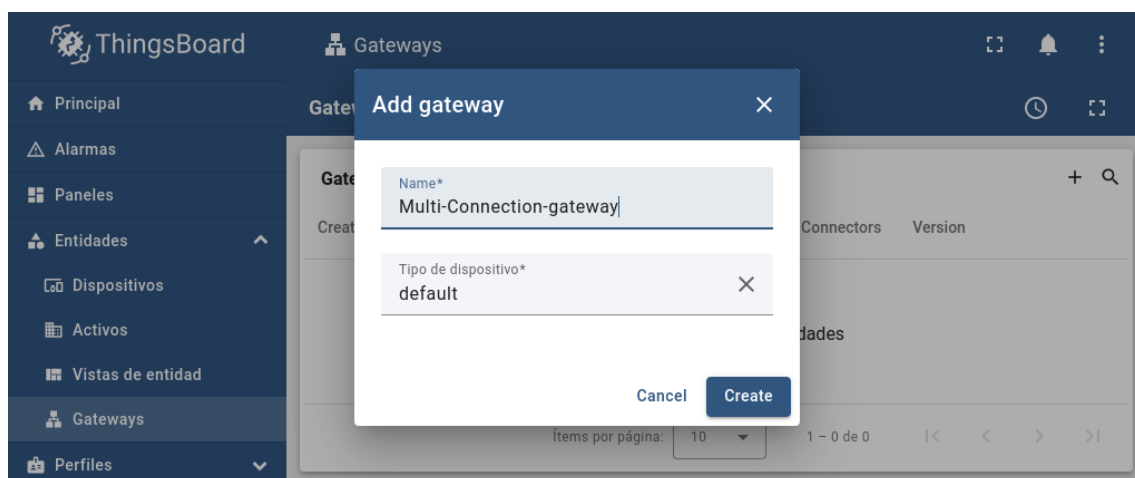


Figura 6.2: Creación de la entidad Gateway en Thingsboard. Elegimos el nombre *Multi-Connection-gateway* y el perfil *default*.

¹ El archivo donde se define el servicio mencionado (`platform/tb-server/docker-compose.yml`) se puede encontrar en el apéndice A.

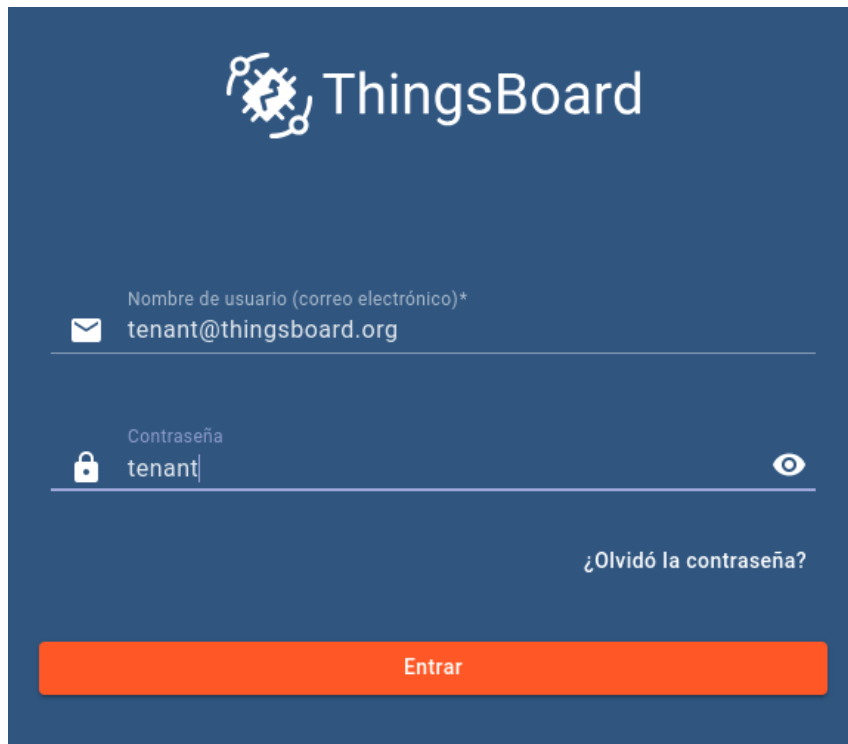


Figura 6.1: Inicio de sesión en la interfaz web del servidor Thingsboard. Se muestra el panel que veremos dentro de nuestro navegador, en la URL `http://localhost:8080/`, cuando el contenedor haya iniciado correctamente. Por defecto, podemos autenticarnos con el usuario `tenant@thingsboard.org` y contraseña `tenant`.

Después, sin salir de la sección Gateways, entraremos al Gateway recién creado, y pulsaremos la opción *General Configuration* (apareceremos en la pantalla titulada *Gateway List > Multi-Connection-gateway > Configuration*). En esa pantalla encontraremos fácilmente el llamado *access Token*, que tendrá un aspecto similar a este: `0IjZaSwakGFSw50ZVUBI`. Lo guardaremos para emplearlo en el despliegue del Thingsboard Gateway, que se describirá a continuación.

6.1.2. Despliegue del Thingsboard Gateway y componentes auxiliares

Desplegaremos en una única unidad tanto el Thingsboard Gateway como los componentes OTA Transfer Service, Broker MQTT y Reliability Manager. Lo haremos de esta manera con la idea de que, en conjunto, formen un gateway con funciones extendidas (actualizaciones OTA con BLE y LoRa). Además, al ser una unidad independiente del servidor, podremos colocarla en otro nodo físico, más cerca de los dispositivos del borde (aunque en este caso prototipo hemos usado el mismo nodo: nuestra estación Linux).

Levantaremos los servicios como un conjunto de contenedores con Docker Compose, definidos en el archivo con ruta en `platform/tb-gateway/docker-compose.yml`². En él, se definen los cuatro servicios, llamados **tb-gateway**, **ota-transfer**, **mqtt-broker** y **lora-reliability-manager**.

²Este archivo se encuentra adjunto en el apéndice A.

Sin embargo, antes de nada, deberemos editar ese archivo de Docker Compose para establecer correctamente algunas de sus variables. Obligatoriamente habremos de sobrescribir la variable de entorno *accessToken* del servicio *tb-gateway* con el valor que obtuvimos en la subsección anterior. De esta forma el servidor autenticará correctamente a su Gateway.

Del mismo modo, deberíamos modificar las variables de entorno *host* y *port* si quisiéramos desplegar este gateway en un lugar distinto al servidor. No obstante, al estar en la misma máquina, el gateway encontrará el servidor con los valores *host.docker.internal* y *1883* en esas variables.

```
1 cd platform/tb-gateway      # Nos movemos al directorio platform/tb-
                             gateway desde la raíz del repo
2 nano docker-compose.yml     # Editamos el access token
3 docker compose up -d        # Levantamos la composición de contenedores
```

Ahora, desde ese mismo directorio, podremos monitorizar y controlar cualquiera de los 4 servicios de esta forma:

```
1 # Consultar los logs
2 docker compose logs -f tb-gateway|ota-transfer|mqtt-broker|lora-
  reliability-manager
3
4 # Detener un contenedor
5 docker compose stop tb-gateway|ota-transfer|mqtt-broker|lora-
  reliability-manager
6
7 # Reanudar un contenedor
8 docker compose start tb-gateway|ota-transfer|mqtt-broker|lora-
  reliability-manager
```

Para direccionar en la red estos componentes desplegados, recomendamos asociar la máquina con un nombre, de manera que no sea necesario referenciarla desde fuera con una IP (que habitualmente es dinámica). Para ello, si no va a producirse comunicación desde fuera de la red local, podemos usar **mDNS** (*Multicast DNS*), un protocolo de resolución de nombres que permite a dispositivos en una red local resolver nombres de host en direcciones IP, sin necesidad de un servidor DNS central. Normalmente, en Linux es posible establecer un nombre mDNS para nuestra máquina configurando el servicio de *avahi*³. En la mayoría de Linux, lo conseguiremos editando el archivo */etc/avahi/avahi-daemon.conf*. En nuestro caso, lo editaremos para obtener el nombre **my-iot-station.local** en la red local accesible desde nuestra interfaz *wlo1*, estableciendo los siguientes parámetros dentro del archivo:

```
[server]
host-name=my-iot-station
domain-name=local
allow-interfaces=wlo1
use-ipv4=yes
```

Después, tan solo tendremos que reiniciar el servicio de *avahi*:

³Avahi es un sistema que implementa mDNS (Multicast DNS) y DNS-SD (DNS Service Discovery) en sistemas operativos basados en Linux.


```
1 systemctl restart avahi-daemon
```

Thingsboard Gateway

Cuando el servicio de Docker Compose *tb-gateway*, asociado al Thingsboard Gateway, arranque correctamente, podremos ver que su atributo *Status* cambia a *Active* en la lista de gateways de Thingsboard (<http://localhost:8080/entities/gateways>).

Servicio OTA Transfer

Cuando el servicio de Docker Compose *ota-transfer*, asociado al servicio OTA Transfer, arranque correctamente, podremos comprobar que el servidor HTTP está a la escucha en el puerto 5000 de la máquina:

```
1 wget http://my-iot-station.local:5000
```

Si recibimos una respuesta del servidor, aunque esta sea de error (pues no recibe el tipo de solicitud que espera), significará que el servicio está operativo.

Broker MQTT

Cuando el servicio de Docker Compose *mqtt-broker*, asociado a nuestro componente Broker MQTT, arranque correctamente, podemos utilizar la aplicación de escritorio MQTT Explorer para conectarnos a él y monitorizar el estado de sus temas. Para ello, podemos abrir una conexión usando su dirección (*my-iot-station.local*, en nuestro caso), puerto 1884, usuario *device* y contraseña *updatable*, tal y como se muestra en la figura 6.3:

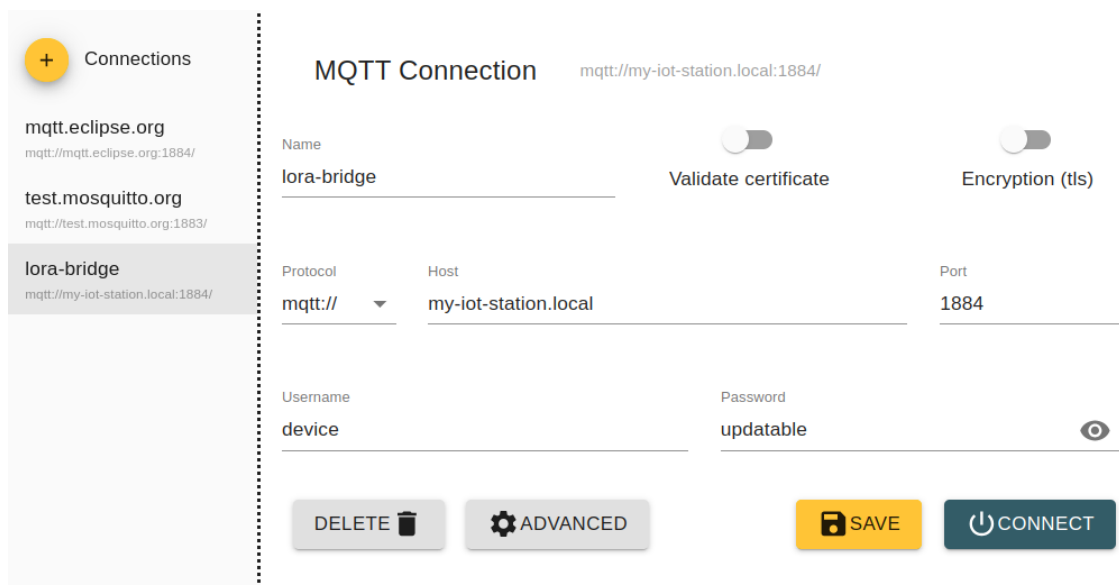


Figura 6.3: Configuración de una conexión MQTT en en Thingboard.

Las credenciales mencionadas son las que hemos configurado por defecto, y pueden cambiarse modificando el directorio `platform/tb-gateway/lora-bridge/mosquitto_config` que está configurado como un volumen asociado al directorio `/mosquitto/config` de la imagen de Docker `eclipse-mosquitto`. También es posible cambiar el puerto de MQTT modificando el servicio `mqtt-broker` del `docker-compose.yml`.

Reliability Manager

Para que este componente funcione como se espera, sería necesario, antes de arrancar su servicio, indicarle los identificadores de los dispositivos LoRa que espera gestionar. Lo haremos editando el `docker-compose.yml`, concretamente la variable `device_id_list` en la sección `environment`. Más tarde, expondremos cómo descubrir este identificador en nuestro dispositivo.

6.1.3. Despliegue del gateway LoRa-MQTT

El despliegue de este componente se tendrá que hacer de forma separada al conjunto anterior, ya que necesita ejecutarse en un hardware distinto (la TTGO LoRa 32), si bien, en última instancia, puede entenderse también como un componente auxiliar al Gateway.

Antes de hacer lo que se explica a continuación, es muy importante asegurarse de que la placa que vayamos a utilizar tiene su antena extraíble colocada y enroscada correctamente en su lugar, ya que hacer uso del módem LoRa sin la antena podría dañar los componentes electrónicos.

Instalación del firmware

Instalaremos en el dispositivo el firmware propio del gateway LoRa del proyecto Theengs OpenMQTTGateway, concretamente su versión compilada para el modelo de nuestro placa. Para ello, con la placa TTGO LoRa32 conectada a nuestro equipo de desarrollo por USB:

```
1 # Nos ubicamos en el directorio donde se ha descargado el firmware
2 cd platform/tb-gateway/lora-bridge/Theengs_OpenMQTTGateway/
3
4 esptool erase_flash # Limpiamos la memoria flash del dispositivo
5
6 # Flasheamos el firmware (sustitúyase /dev/ttyACM0 por la ruta del
  puerto serie virtual correspondiente)
7 esptool --port /dev/ttyACM0 --chip esp32 --baud 921600 --before
  default_reset --after hard_reset write_flash -z --flash_mode dout
  --flash_size detect 0xe000 boot_app0.bin 0x1000 ttgo-lora32-v21-
  bootloader.bin 0x8000 ttgo-lora32-v21-partitions.bin 0x10000 ttgo-
  lora32-v21-firmware.bin
```

Configuración

Una vez instalado el firmware, desconectaremos el dispositivo de nuestro equipo de desarrollo y lo volveremos a conectar a la alimentación, ya sea al mis-

mo equipo de desarrollo o a una alimentación cualquiera. En ese momento, el dispositivo creará una red Wifi propia llamada `OMG_ESP32_LORA`, a la cual nos podremos conectar desde nuestro smartphone, donde se nos abrirá una página para configurar la conexión del dispositivo.

Siguiendo las instrucciones disponibles en la página web del proyecto Theengs OpenMQTTGateway⁴, haremos los ajustes necesarios para que el gateway se conecte a nuestra red Wifi, desde la cual se conectará al broker MQTT. Continuando con nuestra configuración, estas serán las opciones que especificaremos:

```
(Credenciales Wifi)
SSID = (la de nuestra red Wifi local)
Password = (la de nuestra red Wifi local)
```

```
(MQTT)
mqtt server = my-iot-station.local
mqtt port   = 1884
mqtt user   = device
mqtt pass   = updatable
mqtt base topic = thingsboard/
```

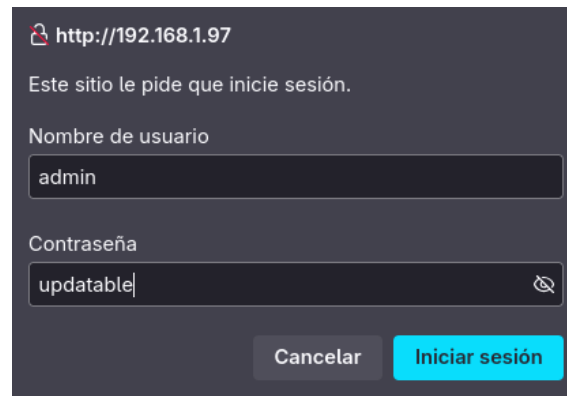
```
(WebUI login)
login = admin
password = updatable
```

Una vez guardada la configuración, se reiniciará el dispositivo, y ahora ya no generará su propia red Wifi, sino que se conectará a la que le hemos especificado, y empezará a realizar sus funciones como gateway LoRa-MQTT.

Sin embargo, aún falta configurar los parámetros de comunicación por LoRa. Nada más conectar el dispositivo a la alimentación, este empezará a mostrar por su pantalla algunos datos relativos al estado de la conexión y, en especial, mostrará su IP dentro de la red Wifi (en nuestro caso, 192.168.1.97). Al entrar con un navegador a la página HTTP servida en esa IP, deberá aparecer un cuadro emergente que nos pedirá unas credenciales, y deberemos introducirlas como se muestra en la figura 6.4.

Una vez dentro, configuraremos los parámetros de LoRa, teniendo en cuenta que habrán de estar en sintonía con el programa que lleve el dispositivo luego. De un modo compatible con el programa de ejemplo, haremos la configuración como se muestra en la figura 6.5.

⁴<https://docs.openmqttgateway.com/upload/portal.html#configuration-portal>



A login form displayed in a browser window. The address bar shows 'http://192.168.1.97'. The page text says 'Este sitio le pide que inicie sesión.' (This site asks you to log in). There are two input fields: 'Nombre de usuario' (Username) with the value 'admin' and 'Contraseña' (Password) with the value 'updatable'. At the bottom, there are two buttons: 'Cancelar' (Cancel) and 'Iniciar sesión' (Log in).

Figura 6.4: Cuadro emergente que aparece en el navegador. Si hemos configurado el gateway como se ha mostrado anteriormente, deberíamos poder entrar con el usuario *admin* y la contraseña *updatable*.



A configuration form titled 'Configure LORA'. It contains several settings, each with a dropdown menu or a checkbox:

- Frequency:** 868MHz
- TX Power:** 14 dBm
- Spreading Factor:** SF7
- Signal Bandwidth:** 125 kHz
- Coding Rate:** 4/5
- Preamble Length:** 8
- Sync Word:** 0x12
- CRC:** ☒
- Invert IQ:** ☐
- Only known:** ☐

At the bottom, there is a green 'Save' button.

Figura 6.5: Configuración del gateway LoRa-MQTT.

6.2 Configuración de la plataforma

Una vez desplegados todos los componentes de nuestra arquitectura que forman la infraestructura de comunicación, deberemos configurar adecuadamente la plataforma con tres objetivos fundamentales: que los dispositivos puedan es-

tablecer conexión con la plataforma a través de las tres formas de conectividad definidas; que se pueda realizar la actualización OTA también a través de los tres escenarios mencionados; y que exista una interfaz gráfica, pensada para un usuario común, donde gestionar los dispositivos y poder actualizarlos.

Llevaremos a cabo esta configuración desde la interfaz de Thingsboard, y deberemos tener a mano los recursos creados con tal propósito en el directorio `platform/resources` del repositorio.

6.2.1. Creación de perfiles

Los perfiles de dispositivos en Thingsboard permiten crear grupos de dispositivos que pueden ser diferenciados por la plataforma y tratados de forma particular. En este caso, crearemos dos perfiles diferentes: uno para los dispositivos con Micropython, y otro para los futuros dispositivos Linux. Esto nos permitirá diferenciar 2 tipos de paquetes de actualización, de manera que, por diseño, la plataforma no permitirá instalar en Linux un paquete de actualización pensado para Micropython, y viceversa.

Para crear estos perfiles, con la barra lateral iremos a la sección *Perfiles > Perfiles de dispositivo*, y añadiremos un nuevo perfil con la opción *Importar perfil de dispositivo*, tal y como se muestra en la figura 6.6. Así importaremos los archivos **`micropython_updatable_profile.json`** y **`linux_updatable_profile.json`**, los cuales contienen los perfiles titulados *Micropython-Updatable-profile* y *Linux-Updatable-profile*, respectivamente.

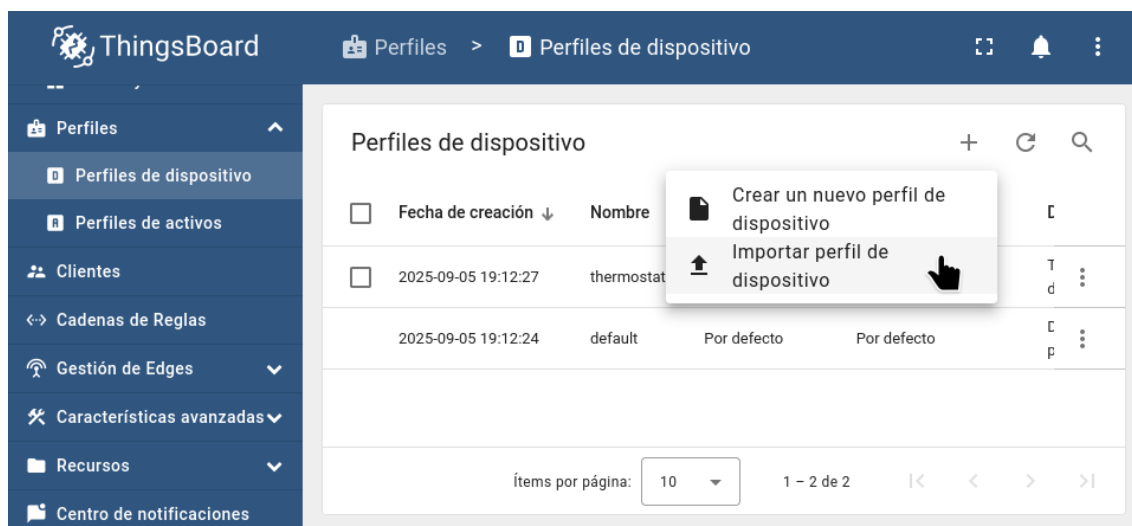


Figura 6.6: Importación de perfiles de dispositivo.

6.2.2. Dashboard

Para importar el Dashboard (o panel) que hemos diseñado, iremos a la sección *Paneles* con la barra lateral y agregaremos un nuevo panel con la opción *Importar panel*, tal como se ve en la figura 6.7. Así importaremos el archivo **`updatable_devices_dashboard.json`**, el cual contiene el Dashboard titulado *Updatable-Devices-dashboard*.

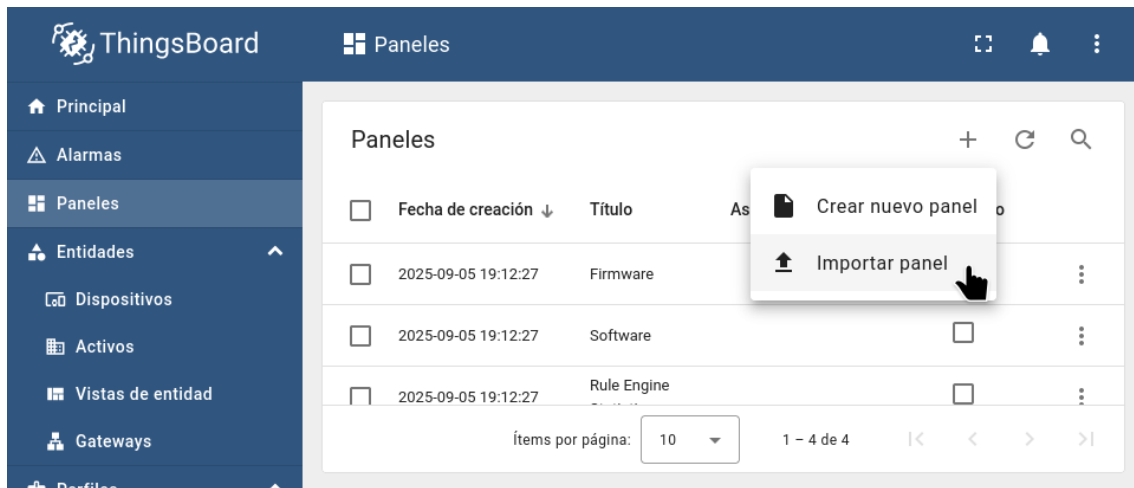


Figura 6.7: Importacion de Dashboards.

6.2.3. Cadena de reglas

Para importar la cadena de reglas desarrollada, que permite la delegación de la OTA en el servicio OTA Transfer para el escenario de BLE y LoRa, iremos con la barra lateral a la sección *Cadenas de reglas*, y agregaremos una nueva con la opción *Importar cadena de reglas*, como se muestra en la figura 6.8. Así importaremos el archivo **updatable_devices_rule_chain.json**, que contiene la cadena de reglas llamada *Updatable-Devices-rule-chain*. Después, es imprescindible establecerla como la cadena de reglas raíz, para que se encargue de manejar todos los mensajes entrantes y salientes de la plataforma. Para ello, haremos click en el icono con forma de lápiz dentro de la cadena recién creada, y pulsaremos la opción *Hacer la Cadena de reglas Raíz*, tal como muestra la figura 6.9.

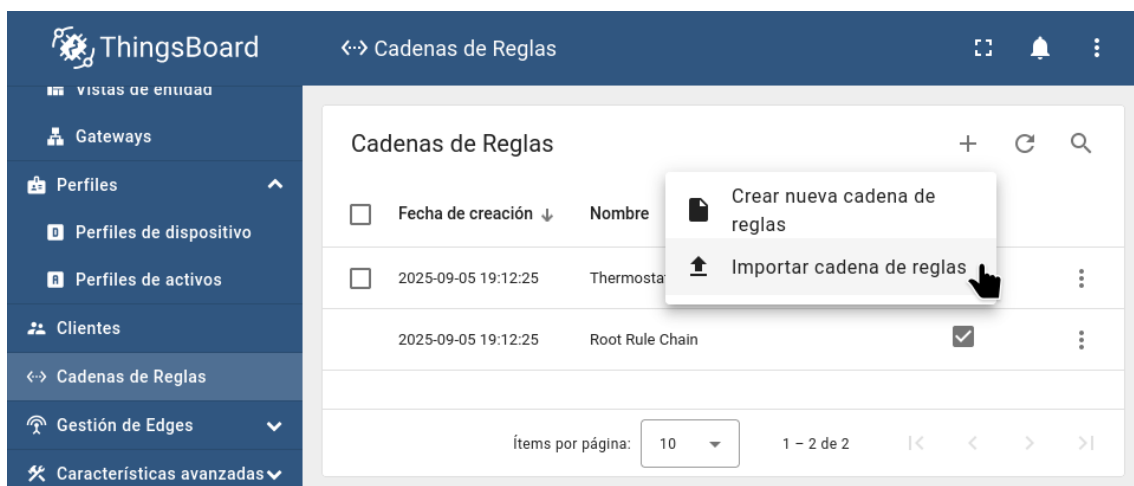


Figura 6.8: Importación de una cadena de reglas.

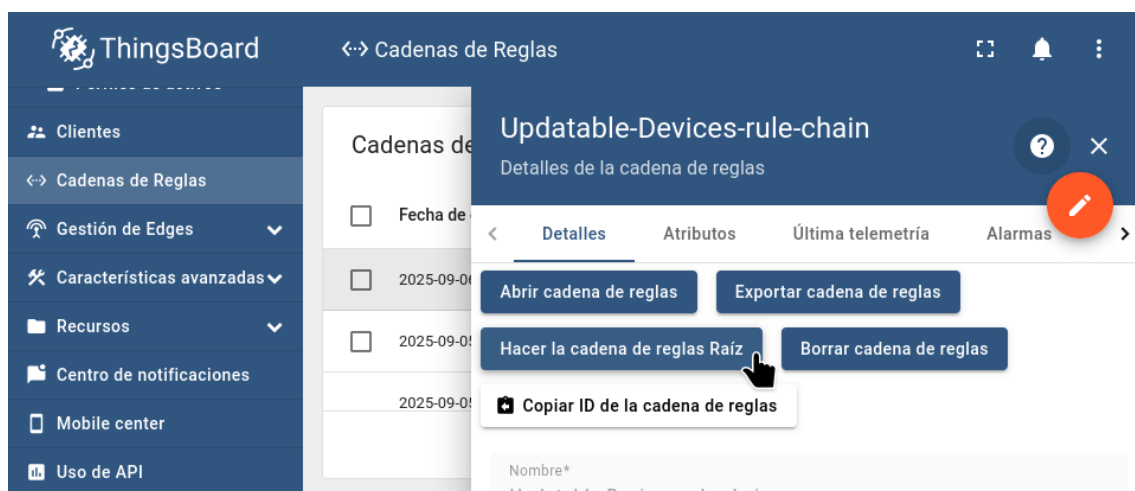


Figura 6.9: Forma de establecer *Updatable-Devices-rule-chain* como la cadena de reglas raíz.

6.2.4. Conector MQTT

Llegados a este punto deberemos crear y configurar adecuadamente los dos conectores que tendrá nuestro Thingsboard Gateway, empezando por el conector MQTT. Para ello, accederemos, como hicimos en el despliegue inicial del servidor, a la lista de Gateways, y entraremos en el nuestro (*Multi-Connection-gateway*), para seguidamente acceder a su opción *Connectors Configuration* y agregar un nuevo conector MQTT que nombraremos *LoRa-Bridge-MQTT-Connector*, tal como se muestra en la figura 6.10:

The screenshot shows the 'Add connector' modal in ThingsBoard. It has a title bar with a question mark and a close button. The form contains three main sections: 'Type' with a dropdown menu set to 'MQTT'; 'Name *' with a text input field containing 'LoRa-Bridge-MQTT-Connector'; and 'Logging level' with a dropdown menu set to 'INFO'. Below these is a checkbox labeled 'Fill configuration with default values' which is checked. At the bottom right are two buttons: 'Cancelar' and 'Agregar'.

Figura 6.10: Creación de un conector MQTT.

Después, desde la misma pantalla habremos de configurar el conector MQTT recién creado, tal como se explica en la figura 6.11.

Conste que esta configuración está pensada específicamente para admitir un dispositivo con nuestro identificador *f024f9b18b08* del ejemplo, además de admitir solo los atributos, telemetría y llamadas RPC que usa el programa de ejemplo. Sin embargo, siguiendo el mismo esquema, no es difícil adaptar esta configura-

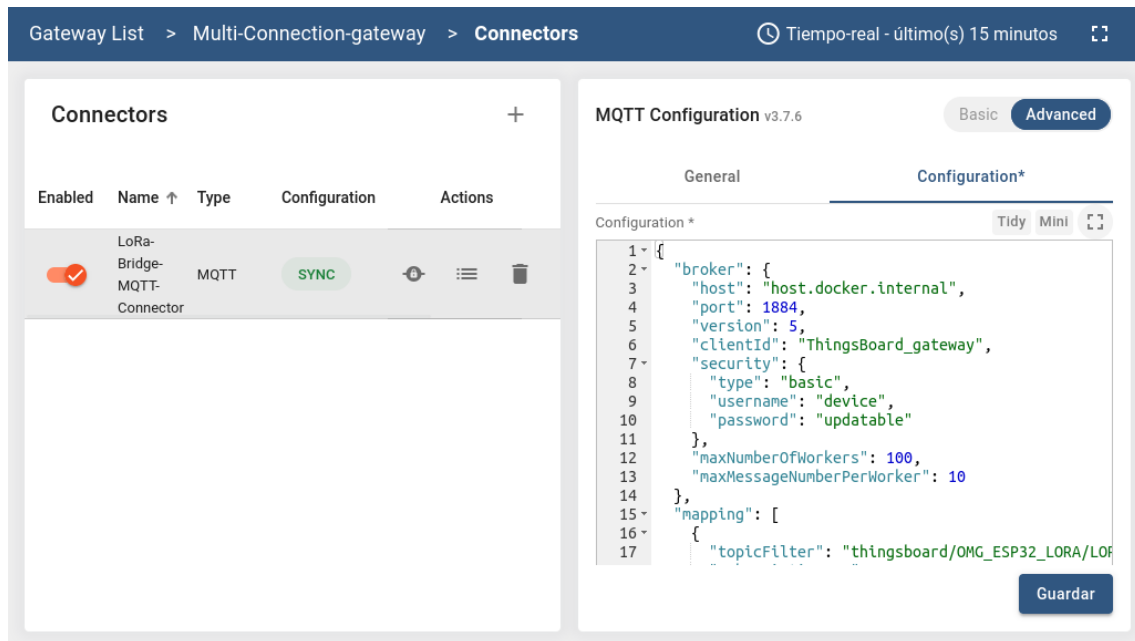


Figura 6.11: Seleccionaremos nuestro conector en la parte izquierda y, en la parte derecha (MQTT Configuration), seleccionaremos la vista *Advanced*. Entonces, en la pestaña *Configuration*, copiaremos el contenido del archivo **MQTT-connector-config.json**. La figura muestra el campo de texto donde deberemos copiar dicho contenido.

ción a otros escenarios, y más tarde veremos cómo averiguar el identificador del dispositivo para escribirlo en esta configuración.

6.2.5. Conector BLE

Del mismo modo que el conector MQTT, crearemos el conector BLE. A la hora de añadirlo, en el cuadro emergente seleccionaremos *BLE* en el campo *Type* y *My-BLE-Connector* en el campo *Name*. De un modo similar al anterior, copiaremos el contenido del archivo **BLE-connector-config.json** en la configuración, tal como se muestra en la figura 6.12

Se debe tener en cuenta, al igual que en el conector MQTT, que esta configuración está pensada para admitir solamente los atributos, telemetría y llamadas RPC que usa el programa de ejemplo. Además, también se encuentra en esta configuración la identificación del dispositivo BLE del ejemplo, que en este tipo de conectores es la MAC del módulo BLE del dispositivo. Posteriormente habremos de averiguar esa MAC y escribirla en esta configuración.

6.2.6. Creación de dispositivos

Para los dispositivos que se conectan por el Gateway, mediante los conectores, Thingsboard creará automáticamente una entidad Dispositivo asociada a cada uno de ellos en el momento en que se conecten. En la configuración de los conectores ya están definidos sus nombres y el perfil que les corresponde.

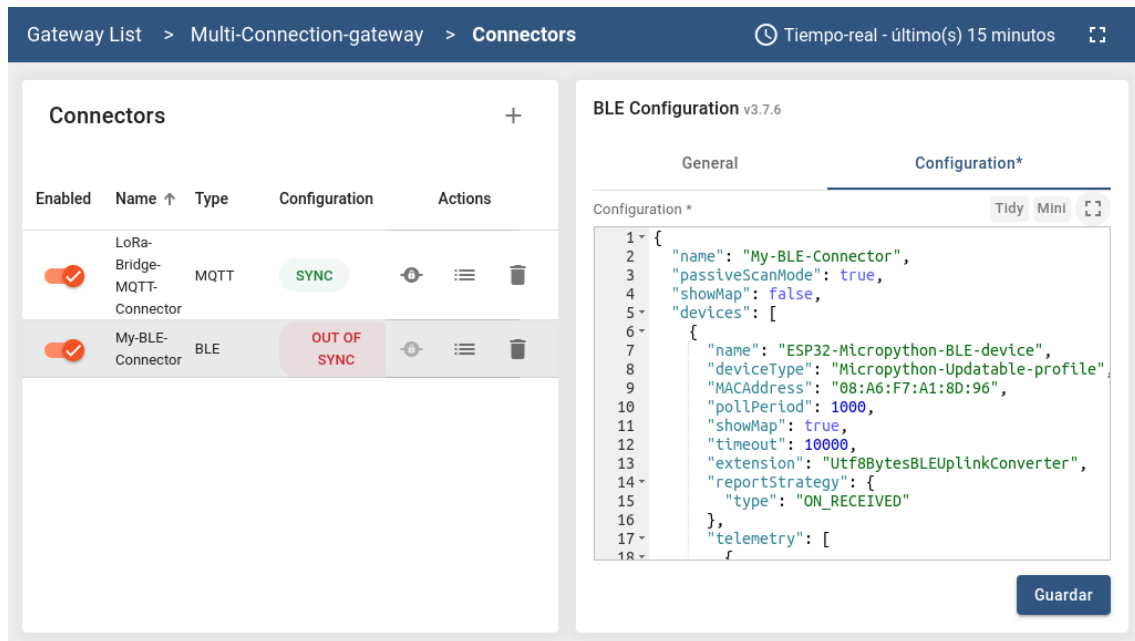


Figura 6.12: Configuración del conector BLE.

Sin embargo, para el dispositivo que se conecta directamente al servidor —es- te es el conectado por Wifi— hay que crear una entidad dispositivo asociada a él. Esto lo haremos manualmente, entrando con la barra lateral a la sección Dis- positivos y agregándolo con la opción *Agregar nuevo dispositivo*. Simplemente le daremos un nombre (en nuestro caso, *ESP32-Micropython-Wifi-device*) y le indi- caremos el perfil *Micropython-Updatable-profile*. Guardaremos su *access token* para usarlo en el programa Micropython del dispositivo del modo que se aprecia en la figura 6.13.



Figura 6.13: Se selecciona el dispositivo recién creado y se copia su *access token*.

6.3 Instalación en los dispositivos del borde

Para realizar las instalaciones que vamos a explicar, necesitaremos un equipo de desarrollo, que puede ser el mismo o uno distinto al que hemos usado como estación de trabajo para desplegar la infraestructura. En cualquier caso, deberemos tener acceso al mismo repositorio:

```
1 git clone https://github.com/p4blo0S/thingsboard-updatable-devices.  
   git  
2 cd thingsboard-updatable-devices/  
3 git submodule update --init --recursive
```

El primer paso para instalar el software desarrollado en los dispositivos del borde, común a todos ellos, consiste en instalarles el firmware de Micropython teniendo en cuenta cuál es su hardware. Para ello, con el dispositivo en cuestión conectado:

```
1 # Directorio donde se encuentran los binarios de Micropython que  
   hemos usado en el desarrollo  
2 cd devices/micropython/firmware  
3  
4 esptool erase_flash # Se limpia la memoria flash  
5  
6 # Flashear el firmware en la ESP32 WROOM (caso Wifi y BLE):  
7 esptool --baud 460800 write_flash 0x1000 ESP32_GENERIC-20241129-v1  
   .24.1.bin  
8  
9 # Flashear el firmware en la ESP32 TTGO LORA32 (caso LoRa):  
10 esptool --baud 460800 write_flash 0x1000 LILYGO_TTGO_LORA32-20250415-  
   v1.25.0.bin
```

Una vez que los dispositivos cuentan con el firmware de Micropython, podemos acceder a ellos mediante el programa de línea de comandos **mpremote**. Partiendo de ahí, el procedimiento para preparar un dispositivo capaz de conectarse a Thingsboard con los programas de ejemplo consiste en lo siguiente:

1. Configurar según nuestras necesidades algunos archivos de configuración ubicados el directorio `devices/micropython/config/`.
2. Instalar la biblioteca **thingsboard-ota-helpers**.
3. Instalar el programa de ejemplo correspondiente.

Entre los archivos de configuración hay uno que es común a todos, el llamado **ota_config.json**, cuyos campos se describen en la figura 6.14:

Figura 6.14: Descripción esquemática del fichero `ota_config.json`

```

1  {
2      "chunk_size": "", # Bytes de chunk (fragmento de transmisión
3                          en el paquete de actualización) usado en el caso de Wifi.
4                          La cadena vacía implica transmitir todo en un solo chunk.
5
6      "tmp_filename": "new_firmware.tar.gz", # Nombre del fichero
7                          temporal que albergará el paquete de actualización
8                          recibido.
9
10     "excluded_files": [ # Lista de ficheros excluidos. No podrán
11                          ser modificados ni borrados durante la actualización.
12                          "boot.py",
13                          "lib/aioable/",
14                          "lib/logging.mpy",
15                          "lib/tarfile/",
16                          "lib/tb_client_sdk/",
17                          "lib/thingsboard_ota_helpers/",
18                          "lib/utils.py"
19     ],
20
21     "clear_filesystem": false # Si es true, todos los ficheros del
22                               sistema salvo los excluidos se borrarán antes de instalar
23                               el paquete de actualización.
24 }

```

La instalación de la biblioteca `thingsboard-ota-helpers` también es común a todos los casos de conectividad, y se realiza instalando el paquete de *mip* descrito en el archivo **thingsboard-ota-helpers.json**, de la siguiente forma:

```

1  # Directorio donde se encuentran los paquetes mip.
2  cd devices/micropython/mip_packages
3
4  # Lanzamos la instalación.
5  mpremote mip install thingsboard-ota-helpers.json

```

A continuación se describen algunos aspectos a tener en cuenta en la instalación de cada dispositivo por separado.

6.3.1. Dispositivo conectado por Wifi

En este escenario de conectividad, el programa de ejemplo hará uso del archivo **wifi_config.json**, ubicado en la ruta `devices/micropython/config/wifi/`. El contenido de este archivo debe seguir una estructura como la que describe en la figura 6.15:

Figura 6.15: Descripción esquemática del fichero `wifi_config.json`

```

1 {
2     # Conexión Wifi
3     "SSID": "MiRedWifi",
4     "password": "abscdfefe",
5
6     # Conexión MQTT al servidor Thingsboard
7     "server_host": "192.168.1.126",
8     "server_port": "1883",
9     "device_access_token": "g10tiwbH0nGxkhIzgFLC",
10
11     # Periodo de comprobación de nuevos mensajes usado por el programa
12     # de ejemplo
13     "check_msg_period_ms": 1000
14 }
```

Cabe resaltar el campo "device_access_token", que debe corresponderse con el *access token* de la identidad dispositivo que obtuvimos en el punto 6.2.6 (Creación de dispositivos) para una correcta autenticación con el servidor Thingsboard.

Una vez realizada la configuración, podemos instalar el programa de ejemplo en el dispositivo de la siguiente forma:

```

1 cd devices/micropython/mip_packages # Directorio donde se encuentran
   los paquetes mip.
2 mpremote mip install example-wifi.json # Lanzamos la instalación.
```

6.3.2. Dispositivo conectado por BLE

En este escenario de conectividad, el programa de ejemplo no hará uso de ningún archivo de configuración especial. Podemos instalar el programa de ejemplo en el dispositivo de la siguiente forma:

```

1 cd devices/micropython/mip_packages # Directorio donde se encuentran
   los paquetes mip.
2 mpremote mip install example-ble.json # Lanzamos la instalación.
```

En estos momentos deberemos averiguar la MAC del módulo BLE del dispositivo para poder escribirla en la configuración del conector BLE de Thingsboard. Podemos leerla en los logs del dispositivo si accedemos a ellos con `mpremote`, estando el dispositivo conectado al equipo de desarrollo:

```

1 mpremote
2 # Logs:
3 # ...
4 # INFO:main| Dirección MAC del módulo BLE: \x08\xa6\xf7\xa1\x8d\x96
5 # ...
```

6.3.3. Dispositivo conectado por LoRa

En este escenario de conectividad, el programa de ejemplo hará uso del archivo `lora_config.json`, ubicado en la ruta `devices/micropython/config/lora/`.

Este archivo es un JSON que el dispositivo leerá directamente como un diccionario de Python, el cual pasará directamente como parámetro de configuración al driver de LoRa⁵. En nuestra configuración de ejemplo, que se muestra en la figura 6.16, estos parámetros son compatibles con los configurados en el gateway LoRa-MQTT (punto 6.1.3):

Figura 6.16: Fichero de ejemplo lora_config.json

```
1 {  
2     "freq_khz": 868000,  
3     "sf": 7,  
4     "bw": "125",  
5     "coding_rate": 5,  
6     "preamble_len": 8,  
7     "output_power": 14,  
8     "crc_en": true,  
9     "tx_ant": "PA_BOOST",  
10    "rx_boost": true,  
11    "lna_gain": 1,  
12    "lna_boost_hf": 3  
13 }
```

Una vez ajustada la configuración, podemos instalar el programa de ejemplo en el dispositivo de la siguiente forma:

```
1 cd devices/micropython/mip_packages # Directorio donde se encuentran  
   los paquetes mip.  
2 mpremote mip install example-lora.json # Lanzamos la instalación.
```

⁵Véase la documentación del driver de LoRa donde se describen los campos que pueden contener dicha configuración: <https://github.com/micropython/micropython-lib/tree/master/micropython/lora#modem-configuration>

CAPÍTULO 7

Pruebas

Si bien este trabajo no ha incluido la realización de pruebas exhaustivas con tests automatizados, queremos dejar constancia en este capítulo de las acciones que hemos tomado para garantizar una cierta calidad en el sistema desarrollado. Aunque una batería de pruebas automáticas es esencial para acompañar un proyecto de software de calidad profesional, la limitación temporal de este trabajo, en combinación con la gran producción de código que ha exigido, ha hecho imposible tal cosa. Sin embargo, sí ha habido otra clase de prácticas y pruebas manuales que nos permiten afirmar que el sistema desarrollado funciona como se espera, las cuales describiremos a continuación.

7.1 Técnicas de depuración

A lo largo del desarrollo hemos realizado múltiples pruebas de depuración manuales, usando distintas técnicas para identificar, aislar y corregir los fallos de código que surgen habitualmente en todo proceso de programación. A continuación se presentan las principales de estas técnicas que hemos usado.

7.1.1. Logging

Las aplicaciones de ejemplo desarrolladas utilizan la biblioteca *logging* de Micropython para ofrecer una visibilidad clara del comportamiento de cada programa, lo que ha ayudado a detectar, localizar y analizar errores. Usar una biblioteca como *logging*, configurable y con diferentes niveles de severidad supone una ventaja respecto a hacer simple *print debugging*. Además, en todo el código de Micropython se ha tratado de hacer un buen tratamiento de errores, capturando excepciones y registrando sus mensajes en el sistema de logs.

A modo de demostración, véase la figura 7.1.

7.1.2. REPL de Micropython

El REPL (*Read-Eval-Print Loop*) de Micropython, al que se puede acceder con *mpremote*, permite a los desarrolladores interactuar directamente con el microcontrolador, ejecutando comandos y obteniendo resultados de manera instantánea.

```

INFO:boot| Iniciando dispositivo
INFO:boot| Versión actual de firmware: micropython-LoRa-Node(v1)
INFO:boot| Paquete de actualización OTA ('new_firmware.tar.gz') no disponible. Continuando con el programa principal.
INFO:main| Iniciando programa principal
MAC Address: f024f9b18b08
INFO:updatable_lora_node| Modem lora = {'_preamble_len': 8, 'crc_errors': 0, 'crc_en': True, 'flags': (<ThreadSafeFlag object at 3ffda2f0>, <ThreadSafeFlag object at 3ffda320>), '_buf1': bytearray(b'\xa0'), '_buf2': bytearray(b'\xa6\x00'), '_sf': 7, '_dio0': Pin(26), '_rx_length': None, '_ant_sw': None, '_rx': False, '_irq_callback': <bound_method>, '_rf_freq_hz': 868000000, '_rx_continuous': False, '_last_irq': None, '_invert_irq': [False, False, False], '_bw_hz': 125000, '_tx': False, '_implicit_header': False, '_coding_rate': 5, '_cs': Pin(18), 'rx_crc_error': False, '_dio1': Pin(35), '_pa_boost': True, '_spi': SPI(id=1, baudrate=2000000, polarity=0, phase=0, bits=8, firstbit=0, sck=5, mosi=27, miso=19)}
DEBUG:updatable_lora_node| Callback de recepción de mensajes establecido.
DEBUG:updatable_lora_node| Enviando mensaje fiable: b'{"count": 0, "id": "f024f9b18b08/reliable/connect", "msg": {}}'
INFO:updatable_lora_node| A la escucha de mensajes LoRa
DEBUG:main| Iniciando envío periódico de telemetría (atributos mem_free, mem_alloc) cada 2 segundos.
DEBUG:updatable_lora_node| Paquete recibido (29 bytes, SNR=27, RSSI=-43, valid_CRC=True). Contenido: bytearray(b'{"id": "f024f9b18b08", "ack": 0}')
DEBUG:updatable_lora_node| Enviando mensaje fiable: b'{"count": 1, "id": "f024f9b18b08/reliable/attributes", "msg": {"lora_id": "f024f9b18b08", "ota_connectivity": "LoRa"}}'
DEBUG:updatable_lora_node| Paquete recibido (29 bytes, SNR=27, RSSI=-42, valid_CRC=True). Contenido: bytearray(b'{"id": "f024f9b18b08", "ack": 1}')
```

Figura 7.1: Esta captura muestra parte de la salida que emite la aplicación de ejemplo para LoRa si nos conectamos al dispositivo con *mpremote* y el nivel de log *debug* está habilitado.

Esta interactividad facilita la identificación de errores y la prueba de fragmentos de código en tiempo real, lo que acelera el proceso de depuración.

7.1.3. Pruebas de conectividad

A lo largo del desarrollo de la infraestructura de comunicación, sobre todo en lo relacionado los componentes intermediarios, hemos ejecutado pruebas de conectividad manuales con algunas herramientas que se mencionan a continuación.

mosquitto_pub y *mosquitto_sub*

Estas son dos herramientas de línea de comandos que proporciona el proyecto Eclipse Mosquitto junto a su broker MQTT. Son clientes MQTT que permiten suscribirse a temas y enviar mensajes, lo que los hace muy útiles para la depuración.

Por ejemplo, para comprobar que el Reliability Manager está activo y responde como se espera, podemos abrir dos terminales:

En la primera terminal, nos suscribiremos a un determinado tema:

```

1  mosquitto_sub -h my-iot-station.local -p 1884 -u device -pw
   updatable -t "thingsboard/OMG_ESP32_LORA/commands/MQTTtoLORA"
2  # Al mandar el mensaje desde la otra terminal, deberemos recibir aquí
   un mensaje como este:
3  # {"message": "{\"id\":\"f024f9b18b08\", \"ack\":64}"}
```


Luego, en otra terminal mandaremos un mensaje, el cual debería causar que el Reliability Manager envíe un ACK al tema suscrito por la primera terminal:

```
1 mosquitto_pub -h my-iot-station.local -p 1884 -u device --pw
  updatable -t thingsboard/OMG_ESP32_LORA/LORAtoMQTT/f024f9b18b08/
  reliable/prueba -m '{"count": 64}'
2 # (sustitúyase f024f9b18b08 con el identificador del dispositivo LoRa
  configurado)
```

MQTT Explorer

Esta aplicación de escritorio nos ha permitido monitorizar de forma gráfica el estado de las colas del broker MQTT.

Por ejemplo, para comprobar que el gateway LoRa-MQTT está activo y bien conectado con el broker MQTT, podemos abrir en MQTT Explorer la conexión creada con el broker, y podemos ver la estructura de temas que el gateway ha creado, la cual debe ser similar a la que se muestra en la figura 7.2:

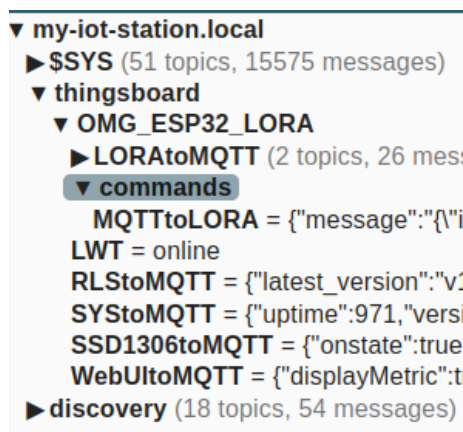


Figura 7.2: Estructura de temas creados por el gateway Lora-MQTT en el broker MQTT, tal como se ven en la aplicación MQTT Explorer.

HTTPIe

Hemos usado la aplicación de escritorio HTTPIe para depurar la API expuesta por el servicio OTA Transfer, elaborando peticiones HTTP de manera gráfica, lo que nos ha permitido verificar la correcta funcionalidad de los *endpoints*.

7.1.4. Depuración de la cadena de reglas en Thingsboard

La pantalla de edición de cadenas de reglas en Thingsboard cuenta con su propio sistema de depuración que, si está activo, permite visualizar el estado detallado de los mensajes entrantes y salientes en cada nodo. Esto ha sido fundamental para localizar los errores que iban surgiendo en los distintos nodos de la cadena y poder solucionarlos.

7.2 Prueba de integración

Se conoce como prueba de integración al proceso de verificar la interacción y el funcionamiento conjunto de diferentes componentes de un sistema.

El desarrollo de este trabajo ha estado en parte guiado por una gran prueba de integración manual, que desde el principio se ha establecido como una meta a superar para poder afirmar que el sistema cumple con sus propósitos. Se trata de una prueba, basada en las aplicaciones de ejemplo, que implica desplegar todos los componentes del sistema, y consiste en la realización de una actualización OTA con Thingsboard en la cual, usando el Dashboard desarrollado, se compruebe fehacientemente la efectividad de la OTA y la buena transmisión de datos entre plataforma y dispositivo.

La prueba puede (y debe) realizarse con las tres formas de conectividad propuestas para los dispositivos (Wifi, BLE, y LoRa), siendo el proceso casi idéntico en los tres casos.

A continuación se describe, paso por paso, el procedimiento para reproducir esta **prueba de integración**¹:

1. **Desplegar la infraestructura y configurar la plataforma adecuadamente.**

Debe hacerse según las indicaciones del capítulo 6 (*Implantación*).

2. **Instalar la aplicación de ejemplo en un dispositivo y ponerlo en marcha.**

Debe hacerse según las indicaciones del punto 6.3 de este trabajo. Haremos una instalación u otra según qué escenario de conectividad vayamos a probar: Wifi, BLE o LoRa.

3. **Abrir el Dashboard *Updatable-Devices-dashboard* y comprobar que el dispositivo establece conexión correctamente.**

El Dashboard se debería ver con una apariencia similar a la que muestra la figura 7.3, y se comprobarán los siguiente aspectos:

- a) El nombre del dispositivo aparece en la tabla *Estado de los dispositivos*. Significa que **el dispositivo ha entrado en contacto con la plataforma**.
- b) El widget que monitoriza la memoria del dispositivo se actualiza constantemente. Significa que **llega telemetría**.
- c) Si se pulsa el botón del Comando RPC "Garbage Collection", se observa un aumento de la memoria libre. Significa que **el comando RPC funciona**.

4. **Asignar un paquete de actualización OTA al dispositivo.**

Se hará de la siguiente forma:

¹Cabe aclarar que las capturas que se van presentando muestran solamente la prueba para un dispositivo conectado por Wifi. Sin embargo, para comprobar el funcionamiento integral del sistema, deben seguirse las indicaciones también para los dispositivos conectados por LoRa y BLE.

1. Con la barra lateral se entrará a la pantalla de actualizaciones OTA (*Características avanzadas > Actualizaciones OTA*).
2. Se añadirá un paquete de actualización al repositorio de paquetes como muestra la figura 7.4. Es importante que los campos *Título* y *Versión* que se escriban coincidan exactamente con los correspondientes en el fichero *FW_METADATA.json* dentro del paquete; de lo contrario, la actualización fracasará. Además, deberemos seleccionar el perfil de dispositivo *Micropython-Updatable-profile*. El paquete de actualización ha de ser de tipo parcial y ha de implementar una nueva característica en concreto: el envío de una nueva variable de telemetría llamada *cpu_frequency*.
3. Desde el Dashboard, concretamente desde el widget *Versiones de firmware*, se editará el llamado *firmware* (en nuestro caso es software) del dispositivo, tal como se muestra en la figura 7.5, y se asignará al dispositivo el nuevo paquete de actualización importado. Esta acción disparará la actualización OTA.

5. Comprobar que la actualización OTA ha sido exitosa.

Después de la actualización, se reanudará el envío de telemetría, y volverá a funcionar el comando RPC. El elemento *Status* del widget *Versiones de firmware* acabará mostrando el valor *UPDATED*, después de pasar por los estados *DOWNLOADING* y *VERIFIED*. Además, a causa de la nueva característica introducida con la actualización, dentro del widget *Frecuencia de CPU*, el dispositivo mostrará un valor en el campo *Frecuencia de CPU*, que previamente debía estar vacío. En suma, el Dashboard presentará un aspecto similar al que muestra la figura 7.6.

Al final del proceso de desarrollo de este trabajo, se consiguió superar esta prueba de integración para los dispositivos con los tres tipos de conectividad.

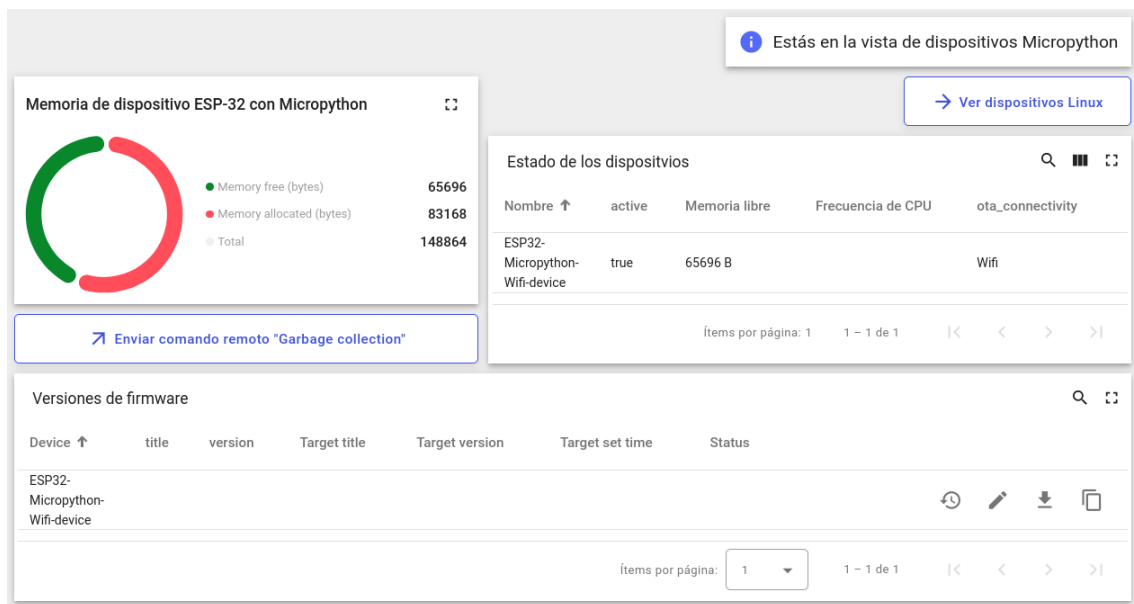


Figura 7.3: Estado visual del Dashboard cuando un dispositivo llamado *ESP32-Micropython-Wifi-device* se conecta a través de Wifi con la aplicación de ejemplo.

Añadir paquete
?
X

Título*
micropython-Wifi-MQTT-client

Versión*
v1

Tag de Versión
micropython-Wifi-MQTT-client v1

El tag de versión debe coincidir con la versión reportada por el dispositivo.

Perfil de dispositivo*
Micropython-Updatable-profile
X

El paquete subido, solamente estará disponible para los dispositivos con el perfil seleccionado.

Tipo de paquete*
Firmware

Una vez que el paquete se haya subido, no se podrá modificar el título, versión, perfil de dispositivo y tipo de paquete.

☒ Subir fichero binario
☐ Usar una URL externa

Package file*

Arrastrar y soltar un fichero o [Navegar fichero](#)
X

micropython-Wifi-MQTT-client_partial_v0_v1.tar.gz

Cancelar
Agregar

Figura 7.4: Cuadro emergente que aparece cuando se pulsa la opción *Añadir paquete* en la sección *Actualizaciones OTA*. Se sube un fichero binario llamado *micropython-Wifi-MQTT-client_v1.tar.gz*.

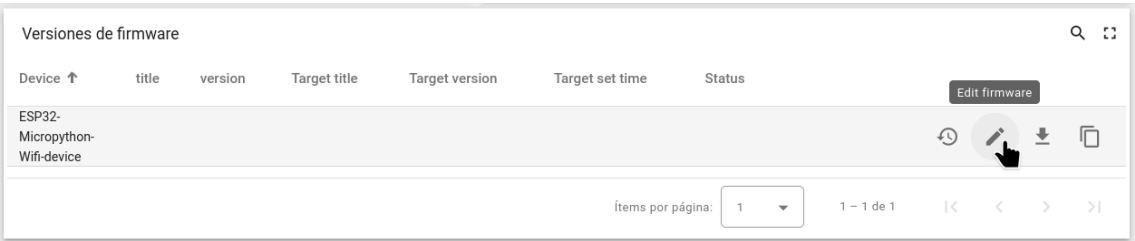


Figura 7.5: Se indica el icono con la acción *Edit firmware* para asignar una nueva versión al dispositivo.

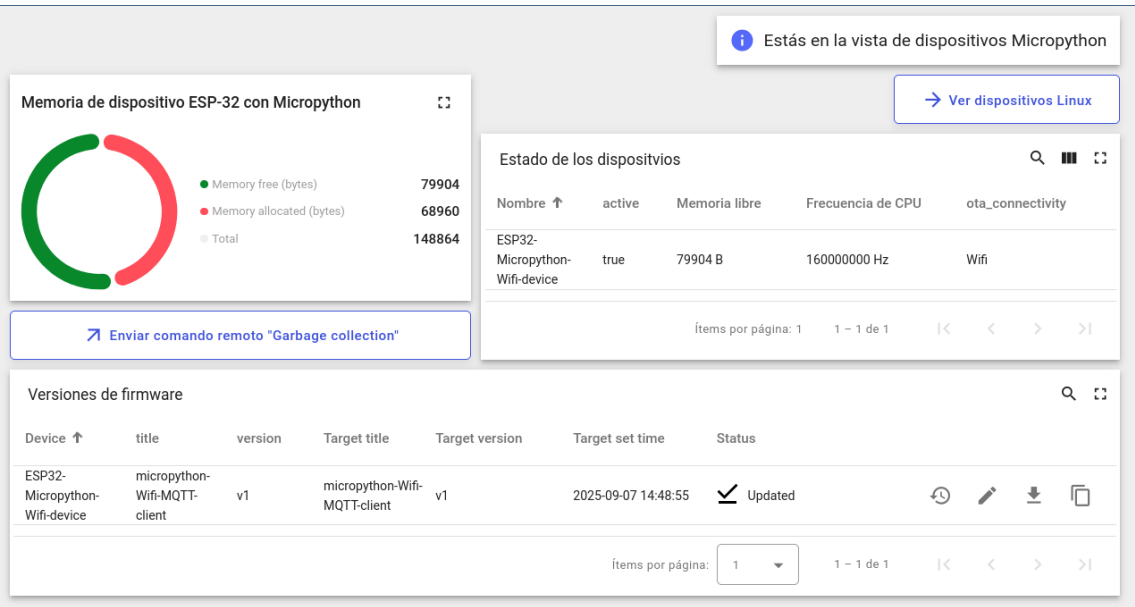


Figura 7.6: Estado visual del Dashboard después de una actualización OTA exitosa por parte del dispositivo llamado *ESP32-Micropython-Wifi-device*. Su versión actual es *micropython-Wifi-MQTT-client v1*.

CAPÍTULO 8

Conclusiones

A lo largo de esta memoria, hemos abordado la creación de un sistema de actualizaciones OTA, analizando primeramente su contexto en el panorama tecnológico actual, seleccionando las herramientas más convenientes a nuestro alcance, y continuando con el diseño y desarrollo de la propuesta, en todo momento guiada por conceptos y buenas prácticas aprendidas en la carrera, sin olvidar la implantación del sistema creado, junto a las pruebas para comprobar su funcionamiento.

Con el trabajo realizado, hemos aportado una solución que contribuye a una gestión de flotas más completa en proyectos IoT que se beneficien del código abierto. Gracias a las herramientas desarrolladas, quienes hagan uso de la popular plataforma Thingsboard, ahora podrán desplegar fácilmente un sistema capaz de actualizar el software de sus dispositivos Micropython, contando con dos tecnologías de red adicionales, como son LoRa y BLE, que Thingsboard no contemplaba hasta la fecha para la realización de OTAs.

Por lo tanto, podemos decir que hemos cumplido con la motivación inicial de este proyecto que, en esencia, era avanzar hacia una gestión de OTAs lo más híbrida posible, en el sentido de poder actualizar, desde un mismo punto, dispositivos de naturaleza diversa conectados mediante diferentes tecnologías de red inalámbricas. Si bien esa idea aún se puede expandir y abordar desde más frentes, hemos demostrado su viabilidad con un ejemplo práctico, asentando así una base para futuras líneas de trabajo.

8.1 Consecución de los objetivos

En la siguiente tabla 8.1 se muestran, a modo de recapitulación, los objetivos propuestos al comienzo del proyecto:

Objetivo	Cumplido
Interfaz común para la gestión de actualizaciones OTA heterogéneas	✓
Implementación en dispositivo con Micropython	✓
Implementación en dispositivo Linux	✗
Comunicación a través de MQTT sobre Wifi	✓
Comunicación a través de BLE	✓
Comunicación a través de LoRa	✓
Creación de componentes reutilizables	✓

Figura 8.1: Se indica para cada objetivo de los propuestos al comienzo si ha sido cumplido o no.

Como se puede observar, consideramos que todos los objetivos han sido resueltos satisfactoriamente a excepción de uno: la implementación de la OTA en dispositivos Linux. Debido a limitaciones de tiempo, este objetivo fue descartado, si bien su planificación ha quedado plasmada en los capítulos 2. *Estado del arte* y 3. *Propuesta*.

En cuanto al resto de objetivos, podemos afirmar que se han cumplido categóricamente, y hacemos caso por caso las valoraciones siguientes:

- a) **Interfaz común para la gestión de actualizaciones OTA heterogéneas:** Efectivamente lo hemos logrado, ya que, usando las capacidades de Thingsboard, hemos desarrollado un panel de administración desde el cuál se pueden consultar las versiones de los dispositivos y actualizarlos independientemente de la tecnología inalámbrica con la que hayan establecido conexión, ya sea a través de Wifi, BLE, o LoRa.
- b) **Implementación en dispositivo con Micropython:** Hemos desarrollado código Micropython que implementa la aplicación de un paquete OTA basado en una sobrescritura del sistema de archivos.
- c) **Comunicación a través de las tres formas de conexión propuestas:** En la parte de infraestructura, hemos realizado el despliegue de los componentes necesarios, y hemos desarrollado unos intermediarios extra para que puedan fluir los datos y los paquetes OTA a través de BLE y LoRa. En la parte del borde, hemos desarrollado una biblioteca capaz de manejar la comunicación bajo las tres formas de conexión.
- d) **Creación de componentes reutilizables:** Hemos desarrollado una serie de componentes que están disponibles para cualquiera que los quiera usar en su propio proyecto:
 - Biblioteca en Micropython `thingsboard-ota-helpers`
 - Programas configurables de ejemplo en Micropython para dispositivos conectables a través de Wifi, BLE y LoRa.
 - Scripts de utilidades `gen_ota_package.py` y `gen_partial_ota_package.py`.
 - Imágenes de docker para los componentes intermediarios desarrollados (*OTA Transfer Service* y *Reliability Manager* para LoRa)

- Automatización del despliegue de la infraestructura con Docker Compose.
- Recursos para Thingsboard (cadena de reglas, perfiles de dispositivo, configuración de conectores, y dashboard)

8.2 Trabajos futuros

A continuación se proponen una serie de trabajos futuros que pueden tomar este como base.

■ Mejoras del sistema desarrollado:

1. **Implementar una aplicación más fiable del paquete de OTA sobre el dispositivo Micropython.**

Aunque el proceso de aplicación del paquete verifica su integridad antes de instalarlo y reporta cualquier error inesperado que se produzca durante la instalación, no existe actualmente un método para asegurar que, tras la instalación, el sistema es funcional y conserva la conexión con la plataforma. Se podría implementar una estrategia de *fallback* para revertir a una versión anterior del software si se da el caso.

2. **Mejora de la biblioteca thingsboard-ota-helpers.**

Actualmente hay un tamaño máximo del paquete OTA que se puede recibir en el dispositivo Micropython, que es menor al de su memoria flash. Esto es así porque se almacena todo el paquete en memoria principal. Para corregirlo, habría que revisar los procesos de recepción, verificación e instalación del paquete.

3. **Realización de pruebas exhaustivas con tests unitarios en Micropython.**

Para asegurar un resultado más profesional, se deberían realizar tests unitarios para el código desarrollado en Micropython, cosa que no se ha hecho por limitación temporal.

4. **Probar el código Micropython con dispositivos distintos a la ESP-32.**

Aunque previsiblemente no deberían surgir apenas problemas, pues la naturaleza multiplataforma de Micropython y el hecho de que hemos evitado usar bibliotecas que dependan de hardware específico, sería conveniente ponerlo a prueba. En especial, se debería probar el manejo del driver LoRa, que tiene diferentes variantes según el modelo de módem.

■ Ampliación de características:

1. **Adaptación a Linux.¹**

Proponemos desarrollar el principal objetivo incompleto de este proyecto. Para ello, sugerimos la creación de una imagen de Linux con la herramienta Buildroot que contenga sólo lo indispensable, y tenga la

¹Se puede encontrar más información sobre esta línea de trabajo en los capítulos 2. *Estado del arte* y 3. *Propuesta*

capacidad de instalar paquetes de actualización de software con la herramienta RAUC. La forma más directa de manejar la comunicación con Thingsboard sería adaptar a Python el código Micropython desarrollado.

2. Implementación sobre otros protocolos de red.

Para ampliar el abanico de protocolos de red sobre los cuales se pueden conectar y actualizar los dispositivos, proponemos que se haga la implementación para algunos de los que también se suelen usar en IoT, como Zigbee o LoRaWAN,

3. Implementación en Zephyr.

Zephyr fue una de las tecnologías en las que se pensó para cubrir la parte de dispositivos basados en microcontrolador. Debido a su eficiencia y sus capacidades como sistema operativo en tiempo real (RTOS), puede ser una elección superior a Micropython en determinados casos.

4. Mejoras en la seguridad.

Para mejorar la seguridad del sistema, se propone: para los dispositivos conectados por MQTT, introducir el uso de TLS; para los dispositivos conectados por LoRa, sustituirla por la implementación en LoRaWAN mencionada anteriormente.

5. Versiones para otras plataformas IoT.

Se propone desarrollar un sistema similar también en otras plataformas diferentes a Thingsboard. Aunque una plataforma no soporte directamente determinados protocolos de red, en este trabajo hemos demostrado que es posible, con los intermediarios de comunicación necesarios, el intercambio de datos y paquetes OTA.

8.3 Relación del trabajo desarrollado con los estudios cursados

Este trabajo está profundamente relacionado con el grado de *Ingeniería Informática* que se ha cursado, y concretamente con la rama cursada, *Ingeniería de computadores*, sirviendo como demostración de los conocimientos obtenidos durante la carrera. Se han empleado gran parte de las enseñanzas aprendidas, en especial las atribuidas al ámbito de las redes, los sistemas distribuidos, los sistemas empujados y la programación.

El entendimiento de los distintos protocolos de red ha sido fundamental para comprender el contexto de una red IoT híbrida y poder aportar soluciones reales a los problemas de comunicación con que nos hemos enfrentado. La experiencia con varias arquitecturas de sistemas distribuidos ha sido clave para diseñar soluciones basadas en colas de mensajería, contenerización o APIs HTTP. Las bases adquiridas en el ámbito del hardware también han ayudado, especialmente al tratar con dispositivos basados en un microcontrolador de recursos limitados. En los problemas de programación, tener una perspectiva del bajo nivel de un sistema ha servido para, por ejemplo, controlar un driver de LoRa adecuadamente, así como controlar la eficiencia en situaciones donde ésta era crítica. Una perspectiva

experimentada de la programación a alto nivel nos ha permitido desarrollador código mantenible y reutilizable, empleando buenas prácticas como la programación orientada a objetos. Finalmente, los conceptos sobre el campo del IoT han sido los que han aportado el contexto y la base para este trabajo, el cual ha estado dirigido a contribuir en los proyectos de este ámbito.

Bibliografía

- [1] “Over-the-air update”, Wikipedia, https://en.wikipedia.org/wiki/Over-the-air_update (Consultado 6 sept., 2025).
- [2] “What are LoRa and LoRaWAN?”, The Things Network, <https://www.thethingsnetwork.org/docs/lorawan/what-is-lorawan/> (Consultado 6 sept., 2025).
- [3] “The Bluetooth Low Energy Primer”, Bluetooth Resources, <https://www.bluetooth.com/bluetooth-resources/the-bluetooth-low-energy-primer/> (Consultado 6 sept., 2025).
- [4] “ThingsBoard — Open-source IoT (Internet of Things) Platform”, ThingsBoard, <https://thingsboard.io/> (Consultado 6 sept., 2025).
- [5] Pablo Gimeno Hermán, “p4bloOS/thingsboard-updatable-devices”, Github, <https://github.com/p4bloOS/thingsboard-updatable-devices.git> (Consultado 6 sept., 2025).
- [6] “MicroPython - Python for microcontrollers”, MicroPython, <https://micropython.org/> (Consultado 6 sept., 2025).
- [7] “MicroPython latest documentation”, MicroPython, <https://docs.micropython.org/en/latest/> (Consultado 6 sept., 2025).
- [8] “OTA (Over-The-Air) vs FOTA (Firmware Over-The-Air) vs SOTA (Software Over-The-Air)”, Rajesh Kumar, <https://www.rajeshkumar.xyz/blog/ota-over-the-air-vs-fota-firmware-over-the-air-vs-sota-software-over-the-air/> (Consultado 6 sept., 2025).
- [9] “Over-the-Air (OTA) Updates in Embedded IoT Devices”, Tech4mint, <https://tech4mint.com/over-the-air-ota-updates-in-embedded-iot-devices/> (Consultado 6 sept., 2025).
- [10] “Zephyr Project”, Zephyr Project, <https://www.zephyrproject.org/> (Consultado 6 sept., 2025).
- [11] “FreeRTOS”, FreeRTOS, <https://www.freertos.org/> (Consultado 6 sept., 2025).
- [12] “Arduino - Home”, Arduino, <https://www.arduino.cc/> (Consultado 6 sept., 2025).

- [13] “The Yocto Project”, Yocto Project, <https://www.yoctoproject.org/> (Consultado 6 sept., 2025).
- [14] “PTXdist - Embedded Linux Build System”, PTXdist, <https://ptxdist.de/> (Consultado 6 sept., 2025).
- [15] “Buildroot - Making Embedded Linux Easy”, Buildroot, <https://buildroot.org/> (Consultado 6 sept., 2025).
- [16] “Raspberry Pi software”, Raspberry Pi, <https://www.raspberrypi.com/software/> (Consultado 6 sept., 2025).
- [17] “The solid foundation for IoT ecosystems — A lightweight yet powerful and scalable OS”, Fedora Linux, <https://fedoraproject.org/iot/> (Consultado 6 sept., 2025).
- [18] “Ubuntu Core”, Ubuntu, <https://ubuntu.com/core> (Consultado 6 sept., 2025).
- [19] “Alpine Linux”, Alpine Linux, <https://www.alpinelinux.org/>
- [20] “Over-the-air software updates for IoT devices”, Mender, <https://mender.io/> (Consultado 6 sept., 2025).
- [21] “Safe and Secure OTA Updates for Embedded Linux”, RAUC, <https://rauc.io/> (Consultado 6 sept., 2025).
- [22] “Your OTA for Embedded Linux and IOT”, SWUpdate, <https://swupdate.org/> (Consultado 6 sept., 2025).
- [23] “Mainflux Open Source IoT Platform”, Mainflux, <https://mainflux.com/> (Consultado 6 sept., 2025).
- [24] “Enterprise IoT Platform with Free Plan”, Kaa, <https://www.kaaiot.com/> (Consultado 6 sept., 2025).
- [25] “An Integrated IoT Platform-as-a-service”, Particle, <https://www.particle.io/> (Consultado 6 sept., 2025).
- [26] “Powerful IoT device management made simple”, Balena, <https://www.balena.io/> (Consultado 6 sept., 2025).
- [27] “High-level code and containers on the ESP32”, Toit, <https://toit.io/> (Consultado 6 sept., 2025).
- [28] “The ThingsBoard documentation”, ThingsBoard Community Edition, <https://thingsboard.io/docs/> (Consultado 6 sept., 2025).
- [29] “Docker: Accelerated Container Application Development”, Docker, <https://www.docker.com/> (Consultado 6 sept., 2025).
- [30] “JSON”, JSON, <https://www.json.org/json-es.html> (Consultado 6 sept., 2025).

- [31] “ESP32-DEVKITC-WROOM”, Zephyr Project Documentation, https://docs.zephyrproject.org/3.7.0/boards/espressif/esp32_devkitc_wroom/doc/index.html (Consultado 6 sept., 2025).
- [32] “TTGO LoRa32”, Zephyr Project Documentation, https://docs.zephyrproject.org/latest/boards/lilygo/ttgo_lora32/doc/index.html (Consultado 6 sept., 2025).
- [33] “LoRa gateway”, Theengs OpenMQTTGateway, <https://docs.openmqttgateway.com/use/lora.html> (Consultado 6 sept., 2025).
- [34] “Eclipse Mosquitto”, Eclipse Mosquitto, <https://mosquitto.org/> (Consultado 6 sept., 2025).
- [35] “esptool”, Pypi, <https://pypi.org/project/esptool/> (Consultado 6 sept., 2025).
- [36] “An all-round MQTT client that provides a structured topic overview”, MQTT Explorer, <https://mqtt-explorer.com/> (Consultado 6 sept., 2025).
- [37] “API testing client that flows with you”, HTTPie, <https://httpie.io/> (Consultado 6 sept., 2025).
- [38] “thingsboard/thingsboard-micropython-client-sdk”, GitHub, <https://github.com/thingsboard/thingsboard-micropython-client-sdk> (Consultado 6 sept., 2025).
- [39] “micropython-lib/micropython/bluetooth/aioble”, GitHub, <https://github.com/micropython/micropython-lib/tree/master/micropython/bluetooth/aioble> (Consultado 6 sept., 2025).
- [40] “micropython-lib/micropython/lora”, GitHub, <https://github.com/micropython/micropython-lib/tree/master/micropython/lora> (Consultado 6 sept., 2025).
- [41] “fastapi”, Pypi, <https://pypi.org/project/fastapi/> (Consultado 6 sept., 2025).
- [42] “uvicorn”, Pypi, <https://pypi.org/project/uvicorn/> (Consultado 6 sept., 2025).
- [43] “bleak”, Pypi, <https://pypi.org/project/bleak/> (Consultado 6 sept., 2025).
- [44] “paho-mqtt”, Pypi, <https://pypi.org/project/paho-mqtt/> (Consultado 6 sept., 2025).

APÉNDICE A

Archivos de interés

A.1 platform/tb-server/docker-compose.yml

```
1 services:
2   tb-server:
3     restart: always
4     image: "thingsboard/tb-postgres"
5     ports:
6       - "8080:9090" # HTTP
7       - "1883:1883" # MQTT
8     environment:
9       TB_QUEUE_TYPE: in-memory
10    volumes:
11      - tb-server-data:/data # Database
12      - tb-server-logs:/var/log/thingsboard # Logs
13
14 volumes:
15   tb-server-data:
16     name: tb-server-data
17   tb-server-logs:
18     name: tb-server-logs
```

A.2 platform/tb-gateway/docker-compose.yml

```
1 services:
2   # ThingsBoard IoT Gateway Service Configuration
3   tb-gateway:
4     image: thingsboard/tb-gateway:3.7-stable
5     container_name: tb-gateway
6     restart: always
7     privileged: true
8
9     # Necessary mapping for Linux
10    extra_hosts:
11      - "host.docker.internal:host-gateway"
12
13    environment:
14      - host=host.docker.internal
15      - port=1883
16      - accessToken=ODyKE7h3knqdIwdcIEcm
```

```

17
18 volumes:
19   - tb-gw-config:/thingsboard_gateway/config
20   - tb-gw-logs:/thingsboard_gateway/logs
21   - ./tb-gw-extensions:/thingsboard_gateway/extensions
22   - /var/run/dbus:/var/run/dbus #
23   - /dev:/dev # # Necesario para usar Bluetooth
24
25 ota-transfer:
26   build:
27     context: ./ota-transfer-service
28   container_name: ota-transfer
29   ports:
30     - "5000:5000"
31   volumes:
32     - tb-gw-config:/tb-gw-config/
33     - /var/run/dbus:/var/run/dbus #
34     - /dev:/dev # # Necesario para usar Bluetooth
35   # Necessary mapping for Linux
36   extra_hosts:
37     - "host.docker.internal:host-gateway"
38   depends_on:
39     - tb-gateway
40
41 mqtt-broker:
42   image: eclipse-mosquitto:latest
43   container_name: mqtt-broker
44   restart: always
45   volumes:
46     - ./lorabridge/mosquitto_config:/mosquitto/config
47     - mosquitto-log:/mosquitto/log
48   ports:
49     - "1884:1883"
50
51 lora-reliability-manager:
52   build:
53     context: ./lorabridge/reliability_manager
54   container_name: lora-reliability-manager
55   extra_hosts:
56     # Necessary mapping for Linux
57     - "host.docker.internal:host-gateway"
58   environment:
59     - mqtt_user=device
60     - mqtt_password=updatable
61     # Lista separada por comas de los identificadores de dispositivos
62     # LoRa en la red
63     # (se obtienen a partir de la MAC Wifi del dispositivo)
64     - device_id_list=f024f9b18b08
65   depends_on:
66     - mqtt-broker
67
68 # Volumes declaration for configurations, extensions and configuration
69 volumes:
70   tb-gw-config:
71     name: tb-gw-config
72   tb-gw-logs:
73     name: tb-gw-logs
74   tb-gw-extensions:
75     name: tb-gw-extensions

```

```
75 | mosquitto-log:  
76 |   name: mosquitto-log
```

A.3 devices/micropython/mip_packages/thingsboard-ota-helpers.json

```
1 | {  
2 |   "urls": [  
3 |     [  
4 |       "thingsboard_ota_helpers/__init__.py",  
5 |       "../src/lib/thingsboard_ota_helpers/__init__.py"  
6 |     ],  
7 |     [  
8 |       "thingsboard_ota_helpers/ota_installer.py",  
9 |       "../src/lib/thingsboard_ota_helpers/ota_installer.py"  
10 |    ],  
11 |    [  
12 |      "thingsboard_ota_helpers/updatable_mqtt_client.py",  
13 |      "../src/lib/thingsboard_ota_helpers/updatable_mqtt_client.py"  
14 |    ],  
15 |    [  
16 |      "thingsboard_ota_helpers/updatable_ble_peripheral.py",  
17 |      "../src/lib/thingsboard_ota_helpers/updatable_ble_peripheral.py"  
18 |    ],  
19 |    [  
20 |      "thingsboard_ota_helpers/updatable_lora_node.py",  
21 |      "../src/lib/thingsboard_ota_helpers/updatable_lora_node.py"  
22 |    ],  
23 |    [  
24 |      "tb_client_sdk/__init__.py",  
25 |      "../src/lib/thingsboard_ota_helpers/__init__.py"  
26 |    ],  
27 |    ["tb_client_sdk/umqtt.py",  
28 |     "../src/external/tb-client-sdk/umqtt.py"],  
29 |    [  
30 |      "tb_client_sdk/sdk_utils.py",  
31 |      "../src/external/tb-client-sdk/sdk_utils.py"  
32 |    ],  
33 |    [  
34 |      "tb_client_sdk/tb_device_mqtt.py",  
35 |      "../src/external/tb-client-sdk/tb_device_mqtt.py"  
36 |    ],  
37 |    [  
38 |      "tb_client_sdk/provision_client.py",  
39 |      "../src/external/tb-client-sdk/provision_client.py"  
40 |    ],  
41 |    "deps": [  
42 |      ["tarfile", "latest"],  
43 |      ["logging", "latest"],  
44 |      ["aioable-peripheral", "latest"],  
45 |      ["aioable-server", "latest"],  
46 |      ["lora-async", "latest"],  
47 |      ["lora-sx127x", "latest"]  
48 |    ]  
49 |  }
```

49

}

APÉNDICE B

Objetivos de Desarrollo Sostenible (ODS)

El presente proyecto guarda relación con varios de los Objetivos de Desarrollo Sostenible (ODS) definidos en la Agenda 2030 de las Naciones Unidas. A continuación, se detalla el grado de vinculación de este trabajo con cada uno de los ODS, valorando su contribución directa o indirecta al cumplimiento de dichos objetivos:

Objetivo de Desarrollo Sostenible	Alto	Medio	Bajo	No procede
ODS 1. Fin de la pobreza				✓
ODS 2. Hambre cero				✓
ODS 3. Salud y bienestar				✓
ODS 4. Educación de calidad				✓
ODS 5. Igualdad de género				✓
ODS 6. Agua limpia y saneamiento				✓
ODS 7. Energía asequible y no contaminante				✓
ODS 8. Trabajo decente y crecimiento económico				✓
ODS 9. Industria, innovación e infraestructuras	✓			
ODS 10. Reducción de las desigualdades				✓
ODS 11. Ciudades y comunidades sostenibles	✓			
ODS 12. Producción y consumo responsables				✓
ODS 13. Acción por el clima				✓
ODS 14. Vida submarina				✓
ODS 15. Vida de ecosistemas terrestres				✓
ODS 16. Paz, justicia e instituciones sólidas				✓
ODS 17. Alianzas para lograr los objetivos				✓

Este proyecto destaca especialmente en su alineación con los siguientes ODS:

- **ODS 9: Industria, innovación e infraestructuras.** La clase de sistemas IoT actualizables que se pueden construir gracias a este trabajo podrían aplicarse en entornos industriales.
- **ODS 11. Ciudades y comunidades sostenibles.** Como bien es sabido, el IoT es la piedra angular de las ciudades inteligentes. La combinación de monitorización y actualización que ofrece este trabajo a los dispositivos de un sistema IoT de este tipo podría suponer una característica diferenciadora.